Technical Report

OFDM Transceiver Prototype

An Adaptive Audio-Band OFDM Transceiver for Amateur HF Radio —
Article Reproduction, Sensitivity Extension to −17.9 dB,
Adaptive Link Layer, and a Fixed-Point RTL Reference Model

Author

Ivan Gubochkin igubochkin@gmail.com

August 27, 2026

Abstract

This report documents a complete software prototype of an audio-band OFDM digital communication system for amateur HF radio, developed in Python (NumPy/SciPy) as a faithful reproduction — and then a substantial extension — of the PHY layer published by bashkirtsevich [2]. The reproduction is bit-exact against every worked example in the article (44/44 checks: CRCs, Base38/QTH encodings, convolutional-code outputs, scrambler and interleaver sequences) and reaches $-7.2\,\mathrm{dB}$ sensitivity versus the article's $\approx -7.5\,\mathrm{dB}$ on the same channel model; the residual gap was diagnosed as synchronization loss, not the decode chain. Three synchronization defects found during the diagnosis — a Zadoff–Chu time–frequency ambiguity worth $\sim$1 dB, an SNR-dependent tone-detection metric, and a preamble transmitted 7 dB below data-symbol power — are analysed and corrected.

The waveform is then extended with SNR-adaptive link modes that scale the coherent symbol-tiling factor ($4\times$ / $16\times$ / $64\times$), lowering the sensitivity floor to $-17.9\,\mathbf{dB}$ in the 6 kHz band[1] (78 % of Shannon capacity at $-20\,\mathrm{dB}$); with a measured monotone LLR recalibration worth 1.5–2 dB at the sensitivity edge; with an IRA LDPC code, Gray-mapped 16-QAM, and CRC-gated chase-combining HARQ; and with a complete link layer: a 13-rung rate ladder spanning 7.8–1059 bit/s over $-17.9\ldots+4.7\,\mathrm{dB}$, receiver-driven rate adaptation through a 20-bit piggybacked link-control word, QoS with fragment-boundary preemption, simplex channel access, and closed-loop AFC frequency netting that pulls a $+284\,\mathrm{Hz}$ initial offset inside a 12 Hz deadband in five exchanges. An SSB RF-chain simulator with per-station reference oscillators makes carrier offsets emerge from oscillator physics rather than parameters, and an integer-only fixed-point twin of the receiver — Q15 arithmetic, block-floating-point FFT, NCO/CORDIC primitives, division-minimised channel estimation — serves as a golden reference for a future RTL implementation, matching the floating-point receiver within 0.5 dB and surpassing it at $-9/-10\,\mathrm{dB}$. All results are backed by self-verifying regression suites (44 article checks, 10 end-to-end cases, 24 fixed-point checks, whole-system simplex and AFC scenario tests) and by a blind-decoded WAV demonstration of a complete two-station session over the fading RF channel.

[1] Throughout this report, SNR follows the article's convention: total signal power versus total noise power in the full 6 kHz Nyquist band of the 12 kHz-sampled audio. The signal itself occupies only $\approx$2.1 kHz (300–2400 Hz), so the same operating point reads $\approx$4.5 dB higher against in-band noise only, and $\approx$3.8 dB higher in the ham-standard 2.5 kHz convention (-17.9 dB here ≈ -14.1 dB ham).

Contents

List of Tables

List of Abbreviations

Abbreviation	Definition
ACK	Acknowledgement
AFC	Automatic Frequency Control
AGC	Automatic Gain Control
ARQ	Automatic Repeat reQuest
AWGN	Additive White Gaussian Noise
BEC	Binary Erasure Channel
BER	Bit Error Rate
BFP	Block Floating Point
BPSK	Binary Phase-Shift Keying
BSC	Binary Symmetric Channel
CFO	Carrier Frequency Offset
CP	Cyclic Prefix
CRC	Cyclic Redundancy Check
DFT/FFT	(Fast) Discrete Fourier Transform
FEC	Forward Error Correction
FIR	Finite Impulse Response (filter)
HARQ	Hybrid ARQ (soft-combining retransmission)
RTP	Real-time Transport Protocol (IETF RFC 3550)
HF	High Frequency (3–30 MHz)
IRA	Irregular Repeat–Accumulate (LDPC construction)
LC	Link Control (word)
LDPC	Low-Density Parity-Check (code)
LFSR	Linear-Feedback Shift Register
LLR	Log-Likelihood Ratio
LO	Local Oscillator
LPF	Low-Pass Filter
MMSE	Minimum Mean-Square Error (equalizer)
NCO	Numerically Controlled Oscillator
NVIS	Near-Vertical-Incidence Skywave
OFDM	Orthogonal Frequency-Division Multiplexing
PAPR	Peak-to-Average Power Ratio
PER	Packet Error Rate
PHY	Physical Layer
ppm	Parts Per Million
QAM	Quadrature Amplitude Modulation
QoS	Quality of Service
QPSK	Quadrature Phase-Shift Keying
QSB	Signal fading (amateur-radio Q code)
QSO	Two-way radio contact (amateur-radio Q code)
RTL	Register Transfer Level
SNR	Signal-to-Noise Ratio
SQNR	Signal-to-Quantisation-Noise Ratio
SSB	Single-Sideband (modulation); USB = upper sideband
STF	Short Training Field (IEEE 802.11 preamble style)
TCXO	Temperature-Compensated Crystal Oscillator
ZC	Zadoff–Chu (sequence)
ZF	Zero-Forcing (channel estimate)

1 Introduction

1.1 Background

The starting point of this work is the Habr article *"Development of a digital amateur-radio protocol based on OFDM: the PHY layer"* by bashkirtsevich [2]: an audio-band OFDM modem designed to be fed into the microphone input of an ordinary SSB transceiver. The article specifies the complete physical layer — a 128-bin OFDM numerology in the 300–2400 Hz audio band, Zadoff–Chu pilots [5], a rate-1/3 $K = 7$ convolutional code [14] with puncturing, packet framing with CRC protection, a two-stage synchronization preamble, and a four-fold symbol-tiling scheme for coherent SNR gain — together with worked numeric examples, an air-channel simulator, simulated BER/PER curves, and a 14 km over-the-air test log.

The article is a rich but informal specification: several implementation details are omitted, one convention (the LLR sign) is stated inconsistently with the article's own code, and the published sensitivity figures depend on synchronization details the text does not fully pin down. Reproducing it independently is therefore a non-trivial exercise in reverse engineering as much as in DSP implementation.

1.2 System Overview

Figure 1 shows the system as it stands at the end of the work reported here. What began as a PHY reproduction has grown into a four-layer communication system, implemented as the pure NumPy/SciPy package `ofdm_phy` with no build step:

- **PHY** — the OFDM modem (`ofdm.py`), the frame transceiver (`transceiver.py`), and the bit pipeline (FEC, interleaver, scrambler, packets, CRC, PAPR reduction);
- **link layer** — a rate ladder and receiver-driven adaptation controller (`link.py`) and a full station with QoS queues, ARQ/HARQ, simplex channel access, and AFC netting (`station.py`);
- **RF/channel** — the article's audio-domain channel simulator (`channel.py`) and an SSB RF chain with per-station reference oscillators (`rf.py`);
- **fixed-point twin** — an integer-only reimplementation of the receiver (`ofdm_phy/fixed/`) structured as an RTL datapath, cross-validated against the float model.

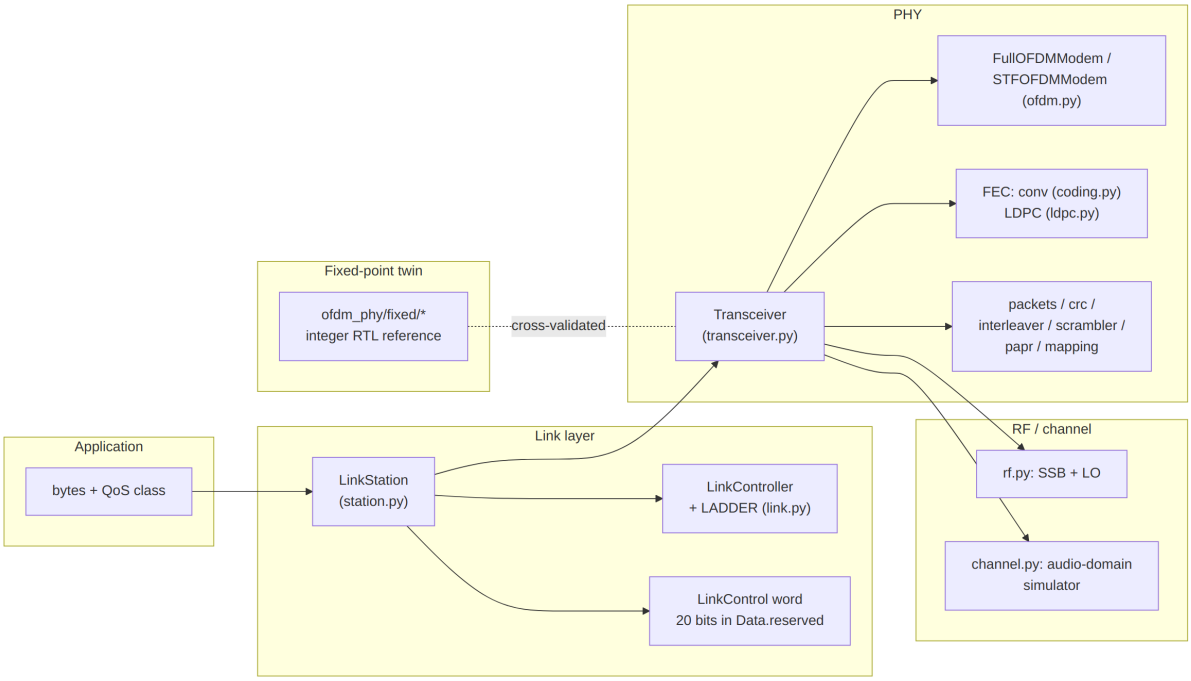


Figure 1: Layered architecture of the prototype. The simulation-side layers (`channel.py`, `rf.py`) are exactly what a real deployment replaces with a soundcard and an SSB transceiver.

Three design principles shaped every layer. First, *faithful defaults*: the plain `Transceiver` reproduces the article's behaviour exactly (convolutional FEC, raw LLRs), and every improvement — LLR recalibration, LDPC coding, HARQ, AFC — is opt-in and enabled at the link layer, so the article-reproduction experiments remain valid permanently. Second, *self-describing frames*: each link mode owns a distinct Zadoff–Chu preamble root, so the receiver identifies the mode from the preamble itself and a mode switch can never deadlock the link. Third, *everything measured feeds back*: per-frame SNR drives rate adaptation, per-frame CFO drives AFC netting, failed decodes export their LLRs for HARQ combining, and every ACK or timeout nudges learned per-rung sensitivity offsets.

1.3 Scope of This Report

This report documents the complete prototype in `ofdm_transceiver_proto`: the article reproduction and its verification, the synchronization analysis that closed the sensitivity gap, and every subsequent extension. All quantitative claims are backed by scripts in `experiments/` (Section 11); the measured artefacts (curves, JSON sweeps, WAV recordings) live in `results/`. Developer-oriented documentation with the same structure exists as Markdown with Mermaid diagrams in `docs/`.

1.4 Contributions

- **Bit-exact article reproduction** — 44/44 worked examples verified; BER/PER curves (article Figures 23–24) reproduced on the article's channel model; measured sensitivity $-7.2\,\mathrm{dB}$ versus the article's $\approx -7.5\,\mathrm{dB}$, with the residual attributed to synchronization by a genie-synchronized control measurement.

- **Synchronization analysis and repair** — three defects identified and fixed: the Zadoff–Chu time–frequency ambiguity (∼1 dB), an SNR-dependent tone-detection metric, and a preamble transmitted 7 dB below data power. An alternative 802.11-style preamble (proposed in the article's comment thread) was implemented and shown statistically equivalent.
- **SNR-adaptive modes** — 16× and 64× tiling modes with per-symbol residual-CFO hypothesis search, group-coherent ZC detection, and per-mode preamble roots, extending the sensitivity floor to −17.9 dB (78 % of Shannon capacity at −20 dB).
- **LLR recalibration** — a measured monotone reliability map correcting a shape (not temperature) miscalibration of the demodulator's LLRs, worth 1.5–2 dB at the BPSK/QPSK sensitivity edge.
- **Coding and modulation extensions** — an IRA LDPC code (N=768, K=256) with normalized min-sum decoding, Gray-mapped 16-QAM extending the ladder to +4.7 dB and 1059 bit/s, and CRC-gated chase-combining HARQ.
- **Adaptive link layer** — a 13-rung measured rate ladder, receiver-driven adaptation over a 20-bit piggybacked link-control word, QoS with fragment-boundary preemption, stop-and-wait ARQ, and simplex channel access with carrier sense; validated by a whole-system two-station test with a mid-transfer fade.
- **AFC frequency netting** — closed-loop carrier netting through the LC word (+284 Hz initial offset converged inside a 12 Hz deadband in five exchanges), with trim-budget and anchor guard rails against common-mode frequency drift.
- **RF-chain simulator** — SSB modulator/product detector at a 48 kHz IF rate with one shared reference oscillator per station, making CFO emerge physically from ppm errors and thermal drift (validated to 0.1 Hz).
- **Fixed-point RTL reference model** — an integer-only receiver covering the full frame family (conv+LDPC, BPSK/QPSK/16-QAM, all modes, HARQ, calibrated LLRs), within 0.5 dB of the float receiver at −7 dB and ahead of it at −9/ − 10 dB.

1.5 Signal Profile at a Glance

Table 1 summarizes the on-air appearance of the waveform in the style of a signal-identification database entry [1], for quick comparison with established sound-card HF modes such as VARA HF.

Table 1: Signal identification profile (sigidwiki-style summary).

Field	Value
Frequency range	any SSB voice channel (the waveform lives in the 300–2400 Hz audio passband); amateur HF in the source article's context
Mode	USB (audio passband, AGC off)
Modulation	OFDM, 23 subcarriers (16 data + 7 Zadoff–Chu pilots), BPSK / QPSK / 16-QAM
FEC	convolutional K=7 rate 1/3 punctured to 1/2, 2/3, 3/4 (soft Viterbi); optional IRA LDPC
Bandwidth	≈2.1 kHz occupied (bins 3–25 at 93.75 Hz spacing)
Symbol rate	22.1 / 5.8 / 1.46 symbols/s (NORMAL / ROBUST / EXTREME tiling)
ACF	10.7 ms (intra-symbol tile); 45.3 / 173 / 685 ms (symbol, mode-dependent)
Net data rate	7.8–1059 bit/s, 13-rung adaptive ladder (Table 6)
Sensitivity	down to −17.9 dB SNR (noise in the 6 kHz Nyquist band; ≈ −14 dB re 2.5 kHz)
Identification	three-part preamble: two Newman-phased tone combs, then a tiled Zadoff–Chu symbol whose root (17/19/21) labels the link mode; the EXTREME bootstrap is heard as a ≈21 s steady warble
Duplex / access	simplex; carrier sense, stop-and-wait ARQ (selective-repeat burst extension in the C implementation, Section 12)
Status	research prototype (this work): bench and virtual-channel operation; not a deployed on-air standard
Short description	audio-band adaptive OFDM data modem for SSB transceivers, reproducing and extending the PHY of [2]

1.6 Report Structure

Section 2 describes the PHY layer as implemented. Section 3 presents the synchronization analysis that closed the sensitivity gap. Section 4 describes the SNR-adaptive link modes. Section 5 covers LLR quality measurement and recalibration. Section 6 describes the LDPC, 16-QAM, and HARQ extensions. Section 7 presents the link layer, and Section 8 the AFC netting loop. Section 9 documents the RF-chain and channel simulators. Section 10 describes the fixed-point reference model. Section 11 consolidates the validation and benchmark results. Section 12 documents the portable C implementation, its microcontroller feasibility, and the demonstration infrastructure. Section 13 discusses limitations and future work, and Section 14 concludes.

2 PHY Layer

2.1 Numerology

The waveform is designed to pass through an ordinary SSB transceiver's audio path (AGC off). Table 2 lists the parameters, all taken from the article [2].

Table 2: OFDM numerology.

Parameter	Value
Sample rate	12 kHz
FFT size	128 bins → 93.75 Hz subcarrier spacing
Occupied band	300–2400 Hz → bins 3–25, 23 subcarriers
Pilots	7, Zadoff–Chu root 3, bins 3, 6, 10, 14, 17, 21, 25
Data carriers	16
Cyclic prefix	32 samples (25 %)
Symbol tiling	4× / 16× / 64× by link mode (+6 dB per 4×, coherent)
CFO tolerance	±375 Hz design, ±300 Hz specified

Symbol *tiling* is the article's central trick for negative-SNR operation: each OFDM symbol body is repeated back-to-back `sym_tile` times, and the receiver accumulates the repeats coherently before the FFT. Each 4× factor buys ≈6 dB of effective SNR at a 4× rate cost.

2.2 Frame Structure

Every frame (Figure 2) carries a two-stage preamble — Newman-phased tone combs [11] for detection and coarse CFO, then Zadoff–Chu symbols for sample-exact timing — followed by a header and the data block. The header (25 bits: `ver(2)` | `typ(3)` | `mod(2)` | `spd(2)` | `len(8)` | `CRC-8`) is always sent in the most robust configuration (BPSK, rate-1/3, six tiled symbols = 93 coded bits); its `mod`/`spd`/`len` fields drive the data-block demodulation, and `ver=2` marks an LDPC-coded data block (Section 6). Preamble bins carry deliberate gain (×2 for ZC symbols, $\times\sqrt{5.75}$ for tone combs) so the whole frame has uniform per-sample power — Section 3 shows why detection sensitivity depends on this.

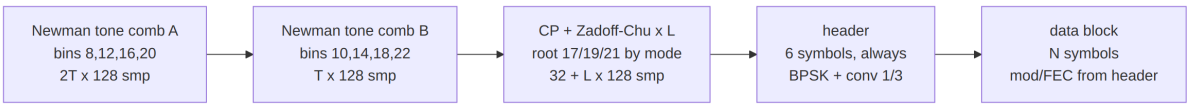


Figure 2: Frame structure. T is the mode's tone-tiling factor and L the mode's Zadoff–Chu symbol count.

2.3 Streamed Bursts

The preamble and header are a fixed cost paid *per frame*: 0.637 s in NORMAL, 2.49 s in ROBUST, 9.92 s in EXTREME (Table 3). For a full 27-byte frame that is ≈24 % of the air time in every mode; for a short frame at the top rung it is 74 %. Extended frames (Section 6) attack this by making one frame carry more. Streaming attacks it from the other side: one preamble and one header for N blocks,

$$[\text{preamble}][\text{header}][\text{blk}_0][\text{ZC}][\text{blk}_1]\ldots$$

with the preamble's own Zadoff–Chu block re-emitted every fourth block to refresh timing and residual CFO. All blocks share the packet type, size, modulation and code rate

— that is precisely what allows a single header to describe them — and the block *count* is not carried in the waveform at all: the link layer signals it, or the receiver decodes until the samples run out.

Table 3: Fixed per-frame cost by mode, measured.

Mode	Tone field	ZC	Header	Total
NORMAL	0.320 s	0.045 s	0.272 s	0.637 s
ROBUST	1.280 s	0.173 s	1.040 s	2.49 s
EXTREME	5.120 s	0.685 s	4.112 s	9.92 s

Note that the header is comparable to the tone field, so dropping only the preamble buys 1.14×; dropping preamble *and* header buys 1.30×. Measured against the same packets sent as independent frames over the article channel (`experiments/stream_mode.py`, 300 bursts = 6 000 blocks per SNR point), 20 × 27-byte NORMAL packets take 16.23 s streamed against 28.16 s per-frame — 1.73× the throughput — for a sensitivity cost of 0.08 dB (Figure 3).

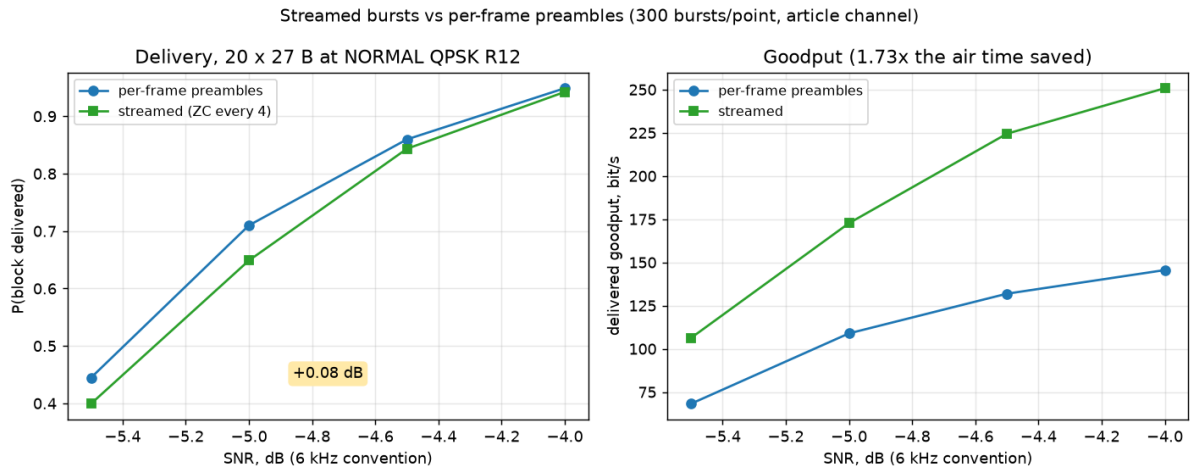


Figure 3: Streamed bursts against per-frame preambles (`experiments/stream_mode.py`). Delivery is scored per block, so the comparison is packets delivered rather than frames sent.

Three properties took measurement rather than reasoning to establish. First, **the Zadoff–Chu resync is not what holds the stream together**: with resync disabled entirely a 24 s stream decodes at the same packet-error rate, because the per-symbol frequency search and the pilot channel estimate already do all the tracking. The resync earns its place on sample-clock offset — a 32-sample cyclic prefix slips in ≈133 s at 20 ppm — and as a re-entry point for a receiver that missed the opening preamble. Second, **failures do not cascade**: block offsets are deterministic, so a block that fails its CRC costs exactly that block, and its raw LLRs are retained for chase combining on the retransmission. Third, **the cost grows with burst length**: a 5-block EXTREME burst measured ≈0.35 dB for only 1.21×, which is why streaming is recommended at NORMAL rungs and not at the modes where decibels are most expensive.

One invariant is easy to violate: the clip-and-filter threshold is a *whole-waveform* RMS, so a burst is not the concatenation of separately built frames. A one-block burst with no

resync is bit-identical to an ordinary frame, and both the float and fixed-point test suites assert exactly that.

2.4 Transmit Chain

Figure 4 shows the transmit chain (`Transceiver.build_frame`). Packet bits (with CRC) are FEC-encoded, zero-padded to a whole number of OFDM symbols *before* interleaving, interleaved over the 16 data carriers, scrambled by a 15-bit LFSR, mapped (BPSK/QPSK/16-QAM Gray), and assembled into OFDM symbols with the ZC pilots and a Hermitian mirror so the time-domain signal is real. After tiling and CP insertion the preamble is prepended and a clip-and-filter stage (clip at RMS+6 dB, low-pass at 3 kHz) bounds the PAPR.

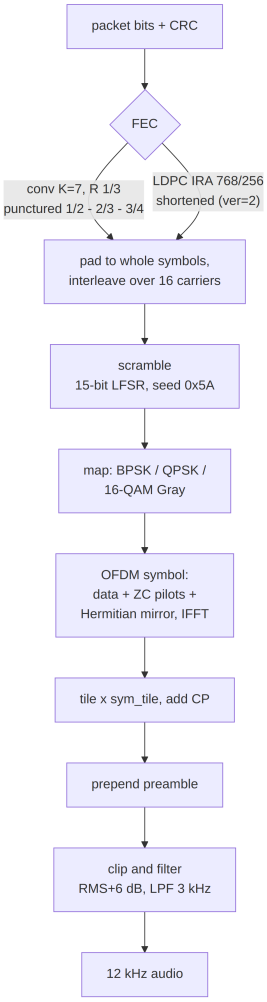


Figure 4: Transmit chain.

Two ordering invariants are easy to break and worth stating: TX applies interleave *then* scramble, so RX must descramble *then* deinterleave; and because the FEC block is padded to whole symbols before interleaving, RX must crop the deinterleaved stream back to the codec's block length before decoding.

2.5 Receive Chain

Figure 5 shows the receive chain (`Transceiver.demod_frame`). After the analytic-signal transform, synchronization runs in three stages (detailed in Section 3); the demodulator then processes each tiled symbol: CP removal and coherent tile accumulation (with per-symbol phase tracking), FFT, pilot-based channel estimation (zero-forcing estimate at the pilots, Wiener smoothing with an exponential frequency-correlation model of length 8 bins, MMSE equalization), and LLR formation weighted by the per-carrier symbol SNR, clipped to ±20. One convention is load-bearing across four modules: LLR sign is **positive = logical 1**, consistent with the article's BPSK mapping table (the article's prose states the opposite of its own code).

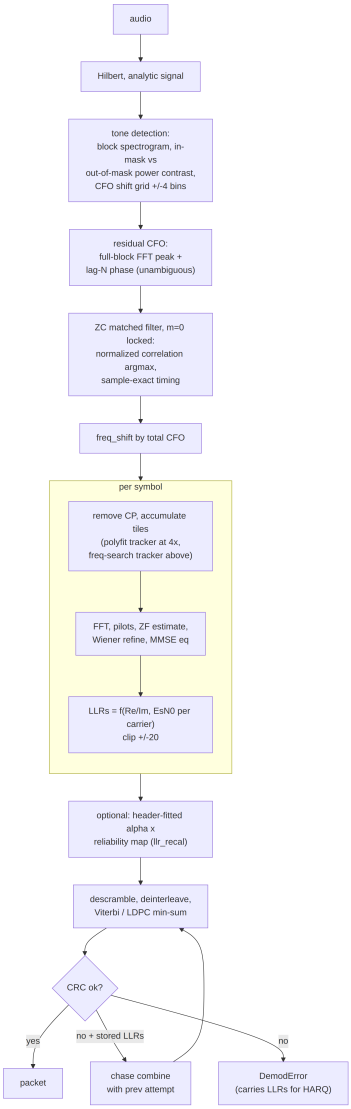


Figure 5: Receive chain, including the opt-in recalibration stage (Section 5) and the HARQ path (Section 6).

A CRC failure does not simply discard the frame: the `DemodError` exception carries the decoded header and the data-block LLRs, which is what makes the chase-combining HARQ of Section 6 possible without any extra signalling.

2.6 Soft Demodulation: from Symbols to LLRs

This subsection states exactly what the demodulator computes (`OFDMModem.demodulate_symbol_cp_so`). After tile accumulation and FFT, each data carrier k of a symbol obeys

$$Y_k = H_k S_k + N_k, \qquad N_k \sim \mathcal{CN}(0, \sigma^2), \tag{1}$$

where H_k is the channel coefficient and S_k the transmitted constellation point. With $\hat{H}_k$ the Wiener-smoothed pilot estimate, the MMSE equalizer output is

$$\hat{S}_k \;=\; \frac{Y_k\,\hat{H}_k^{*}}{|\hat{H}_k|^2 + \hat{\sigma}^2}. \tag{2}$$

The noise power $\hat{\sigma}^2$ is tracked decision-directed: the mean-square error of $\hat{S}_k$ against the nearest constellation point (per axis for BPSK/QPSK; against the sliced 16-QAM grid for $\mu = 4$), referred back through the channel by the mean $|\hat{H}_k|^2$, and smoothed with an exponential filter ($\alpha = 0.1$) across symbols. The per-carrier symbol SNR

$$\rho_k \;=\; \frac{|\hat{H}_k|^2}{\hat{\sigma}^2} \tag{3}$$

weights every LLR, so bits on faded carriers enter the decoder as weak evidence rather than as confident errors.

The log-likelihood ratio of a bit b is defined here as

$$L(b) \;=\; \ln \frac{P(b=1 \mid \hat{S})}{P(b=0 \mid \hat{S})}, \tag{4}$$

i.e. **positive LLR means logical 1** — the load-bearing sign convention noted above. Under the max-log approximation $\ln \sum_i e^{x_i} \approx \max_i x_i$, the general form

$$L(b_i) \;\approx\; \rho_k \Big(\min_{s \in \mathcal{S}_0^{(i)}} |\hat{S}_k - s|^2 - \min_{s \in \mathcal{S}_1^{(i)}} |\hat{S}_k - s|^2 \Big) \tag{5}$$

collapses to closed forms per modulation (as implemented; overall scale is immaterial to the scale-invariant Viterbi and min-sum decoders, but the *shape* across carriers and bit positions is what Section 5 calibrates):

$$\text{BPSK:} \quad L = 4\,\rho_k \operatorname{Re} \hat{S}_k, \tag{6}$$

$$\text{QPSK:} \quad L_I = 2\,\rho_k \operatorname{Re} \hat{S}_k, \qquad L_Q = 2\,\rho_k \operatorname{Im} \hat{S}_k, \tag{7}$$

$$\text{16-QAM (Gray, unit } a = g/\sqrt{10}\text{):} \quad L_{b_0} = 2\,\rho_k \operatorname{Re} \hat{S}_k / g, \qquad L_{b_1} = 2\,\rho_k \big(2a - |\operatorname{Re} \hat{S}_k|\big)/g, \tag{8}$$

with the imaginary-axis pair L_{b_2}, L_{b_3} analogous, and g the per-symbol gain estimate. The 16-QAM forms are the piecewise-linear max-log metrics of the Gray mapping: the outer bit (b_0) is the axis sign, the inner bit (b_1) discriminates inner from outer amplitude levels, changing sign at $|\operatorname{Re} \hat{S}| = 2a$. All LLRs are clipped to ± 20 before decoding.

3 Synchronization and the Sensitivity Gap

The decode chain reproduced the article bit-exactly almost immediately; the measured *sensitivity* did not. This section documents the diagnosis — the most instructive part of the reproduction — and the resulting synchronizer.

3.1 Diagnosis Method

The tool was a *genie-synchronized* control receiver: the same demodulator fed the true timing offset and CFO from the channel simulator. With genie sync the chain measured -7.6 dB (BPSK rate-1/3, article channel, PER $\leq 10\,\%$)[2]; the real receiver initially sat several dB worse. Every improvement below was validated by closing part of that gap, and the final residual is $\sim$0.3 dB (Section 11).

3.2 Three Defects and Their Fixes

Preamble power. Transmitting the tone combs and ZC symbols at nominal bin amplitude leaves the preamble region $\sim$7 dB below the frame-average power (4 tones versus 23 loaded carriers). Detection then fails at SNRs where the payload would decode. The fix is the gain scaling of Section 2: $\times\sqrt{5.75}$ on tone bins and $\times 2$ on ZC bins equalizes per-sample power across the frame.

Tone-detection metric. The article's tone detector thresholds the in-mask energy *fraction* of a block spectrogram. That fraction is SNR-dependent — it fails below $\approx$0 dB regardless of threshold — and divides by zero on noise-free signals. The implemented metric is a *contrast ratio*: mean in-mask bin power over mean out-of-mask bin power, evaluated over a grid of $\pm$4-bin CFO shifts of the mask, with the denominator regularized by 1 % of the median block power. The regularizer matters: on clean signals the out-of-mask power is $\approx$0 and the unregularized argmax degenerates into a lottery among CFO hypotheses, producing exact one- and two-bin CFO errors on clean WAV recordings.

Zadoff–Chu time–frequency ambiguity. The ZC matched filter selects the best *normalized* correlation over a search window after the tone preamble (the raw-peak argmax locks onto the periodic tiled data symbols). The subtler defect was scanning the ZC correlation over $\pm$1-bin frequency hypotheses: a Zadoff–Chu sequence's frequency-shifted replica correlates almost as well at a *shifted time* (the classic ZC time–frequency ambiguity), so at low SNR the wrong hypothesis wins, injecting a one-subcarrier (93.75 Hz) CFO error plus a $\sim$30-sample timing error. Locking the composite detector to the zero-CFO hypothesis — legitimate because the tone stage already leaves a residual well below half a bin — recovered $\approx$1 dB of sensitivity by itself.

Residual CFO. The tone-stage residual CFO is estimated by a full-length FFT peak over the first tone block (unambiguous over $\pm$1.5 bins), refined by a lag-N phase estimate. A lag-N estimate alone is ambiguous modulo one subcarrier spacing; during development a lag-16 variant produced 276 Hz errors because the lag was not a multiple of the tone-comb period — the general rule (enforced in the code) is that a delay-and-correlate lag must be a multiple of the comb's time-domain period.

[2] All sensitivities in this report use the article's SNR convention: noise counted across the full 6 kHz Nyquist band, of which the signal occupies $\approx$2.1 kHz. Add $\approx$4.5 dB to compare against in-band noise only, $\approx$3.8 dB for the ham 2.5 kHz convention.

3.3 The STF Preamble Variant

A commenter on the article proposed replacing the Newman tone comb with 802.11-style short-training tones every 8 bins. This was implemented as `STFOFDMModem`: tone combs on bins $[8, 16, 24]$ and $[4, 12, 20]$ make the tone block periodic with 16 samples, so the residual CFO comes from a plain Schmidl–Cox-style delay-and-correlate [12] (lag 16 refined by lag 128), unambiguous over ± 375 Hz. An A/B sweep over the full ± 300 Hz CFO range and all eight modulation/rate configurations shows the two preambles statistically equivalent: identical -7.2 dB sensitivity, all eight per-configuration sensitivities within 0.14 dB, every PER point within 2σ binomial noise (Figure 6). The article-faithful `FullOFDMModem` remains the default.

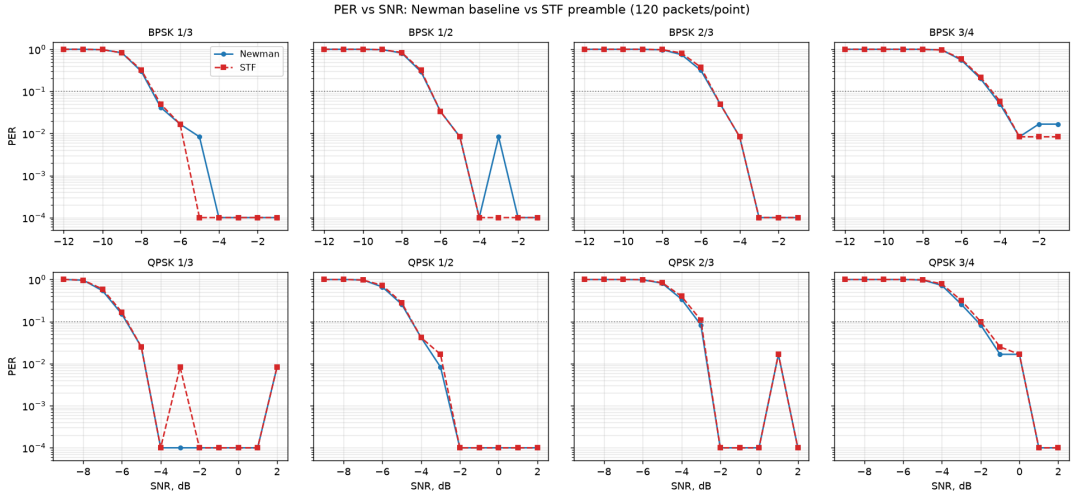


Figure 6: PER versus SNR for the Newman-tone and STF preamble variants over the article channel (same channel realizations per arm).

4 SNR-Adaptive Link Modes

The article's waveform operates around -7 dB. The first major extension scales the coherent-accumulation tile factor (and the preambles with it) into three link modes (Table 4), trading bitrate for sensitivity.

Table 4: Link modes (sensitivities at PER $\leq 10\,\%$, recalibrated receiver, article channel, 6 kHz-band SNR).

Mode	Tiles	ZC root	User rate (BPSK 1/3)	Sensitivity
NORMAL	4×	17	118 bit/s	−7.6 dB
ROBUST	16×	19	31 bit/s	−11.7 dB
EXTREME	64×	21	7.8 bit/s	−17.9 dB

Shannon capacity [13] of the 2100 Hz occupied band at -20 dB (6 kHz reference) is ≈30 bit/s; EXTREME's channel rate of 22 bit/s is 78 % of capacity, so the theoretical wall for this bandwidth is -20.7 dB and the measured -17.9 dB floor sits ≈3 dB from it.

4.1 What Makes the Low-SNR Modes Work

Each of the following was found necessary empirically; removing any one of them collapses the EXTREME mode.

Per-symbol residual-CFO hypothesis search. A 64× tiled symbol integrates coherently over 0.69 s and cannot tolerate even a few Hz of residual CFO across the accumulation window; per-tile phase tracking (the NORMAL-mode polynomial fit) is pure noise at −19 dB. The tiled demodulator instead derotates the whole symbol over a grid of frequency hypotheses, accumulates tiles under each, and keeps the hypothesis with maximum in-band energy — spending the symbol's *entire* energy (≈19 dB E/N_0 even at −20 dB SNR) on the frequency decision. A slew-limited tracker (full grid on the first symbol, ±2 grid steps thereafter) suppresses per-symbol argmax noise; the search range must cover the worst-case preamble CFO error (±25 Hz for EXTREME).

Finer detection FFT. ROBUST and EXTREME tone detection uses 512-point spectrogram blocks: 4× more tone energy per bin, and a 23.4 Hz coarse-CFO grid whose residual fits a plain lag-N estimate.

Group-coherent ZC correlation. The ZC stage correlates kernels of 2–4 concatenated ZC symbols (magnitudes summed across groups) with a small fractional-CFO hypothesis grid (±15 Hz) — single-symbol correlation is not selective enough at −18 dB, and Section 3's ambiguity argument forbids a wide frequency scan.

Per-mode preamble roots. The receiver cannot read the tile factor from the header, because it needs the tile factor to demodulate the header. The mode is therefore signalled by the preamble itself: each mode owns a distinct ZC root (17/19/21), a wrong-root matched filter simply does not lock, and `demod_frame_auto` tries the modes in turn (Figure 7). Detection thresholds were tuned against noise-only false alarms (0/20 on pure noise, all modes).

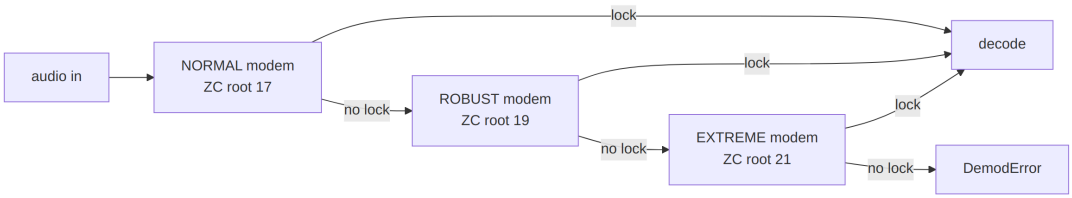


Figure 7: Blind mode detection: the preamble root *is* the mode signal. A consequence used throughout the link layer: a mode switch can never deadlock the link.

4.2 Constellations Across the Operating Range

Figure 8 makes the adaptation policy visible in one picture: for eight SNRs spanning the whole operating range (−17.5 to +20 dB), it shows the *equalized received constellation of the rung the rate controller actually selects* at that SNR (highest rung whose measured sensitivity clears a 1 dB margin). Each panel is produced by the real per-symbol receive chain — coherent tiling accumulation, per-symbol frequency search where the mode uses one, and MMSE equalization (`experiments/figures.py`). At −17.5 dB the EXTREME

BPSK decision variable barely emerges from the noise — which is precisely why rung 0 pairs 64× tiling with the rate-1/3 code and soft decisions; the QPSK quadrants firm up through the mid-ladder, and the 16-QAM grid the top rungs rely on only becomes usable above ≈+3 dB, matching the ladder sensitivities of Table 6.

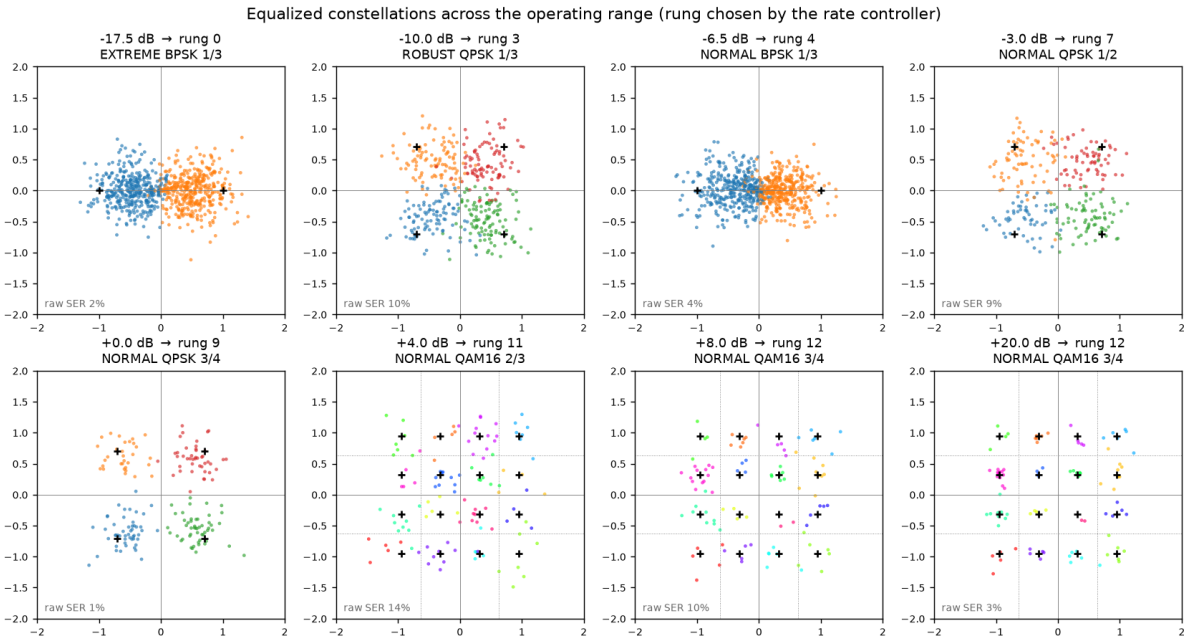


Figure 8: Equalized constellations at the controller-selected rung, from the EXTREME sensitivity floor to a clean channel (article channel, AWGN + CFO). Black crosses mark the ideal constellation points; each received point is colored by the *transmitted* symbol, so raw pre-FEC errors appear as points of the wrong color inside a neighbouring decision region (the per-panel "raw SER" annotation counts them), and the dotted subgrid on the 16-QAM panels marks the decision boundaries between the 16 cells.

4.3 Measured Waterfalls

Figure 9 shows the three modes' PER waterfalls on the article channel. The function `select_mode` implements the resulting adaptation policy (mode + modulation + code rate for an expected SNR), later subsumed by the link-layer ladder of Section 7.

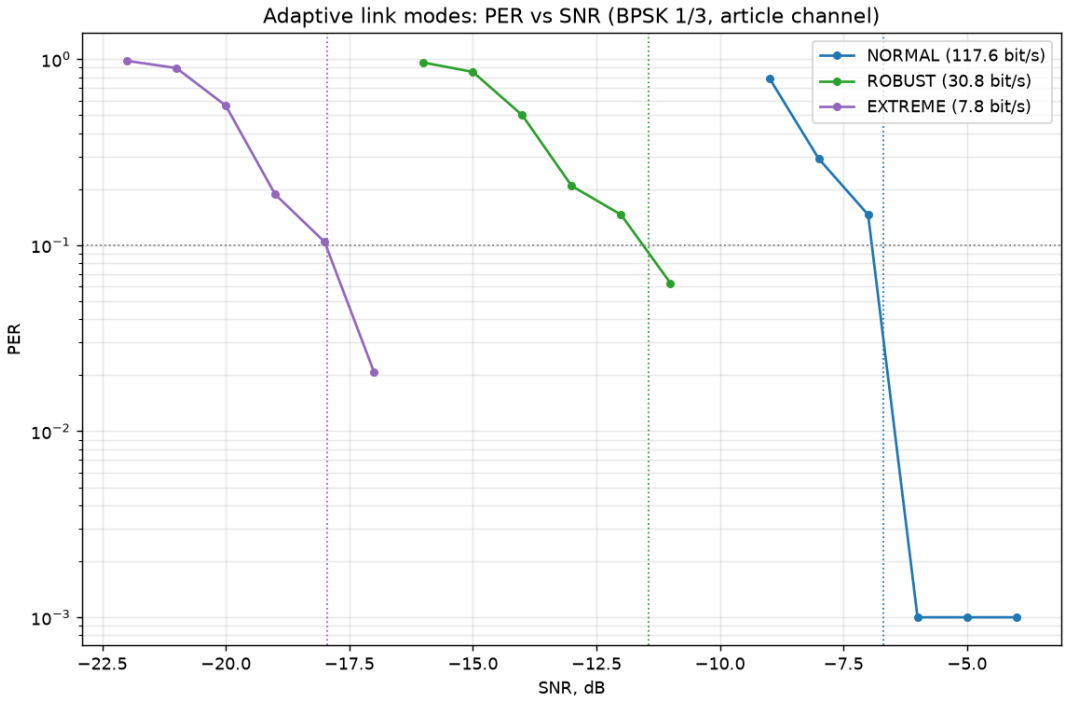


Figure 9: PER versus SNR for the three link modes (`experiments/adaptive_modes.py`).

5 LLR Quality and Recalibration

A code-review pass over the finished system asked a simple question: are the demodulator's LLRs actually calibrated — does $|L|$ mean what it claims? Measuring them against ground truth (`experiments/llr_calibration.py`) found the single largest untapped gain in the system.

5.1 The Miscalibration Is Shape, Not Temperature

Two terms are used throughout this section and deserve definition. *Temperature scaling* is the standard one-parameter recalibration family

$$L' \;=\; \alpha \cdot L, \qquad \alpha > 0, \tag{9}$$

one global gain applied uniformly to every LLR. The name comes from machine-learning calibration [7], where a network's logits are divided by a scalar "temperature" T before the softmax ($\alpha \equiv 1/T$) — itself borrowed from the Boltzmann distribution, in which temperature controls how sharply probability concentrates on the most likely outcome: $\alpha < 1$ softens overconfident beliefs, $\alpha > 1$ sharpens underconfident ones. It is the natural first candidate here because it preserves signs and ordering, costs one multiply, and can be fitted blind on a per-frame basis: a correctly calibrated Gaussian LLR ensemble satisfies the consistency condition $\mathrm{Var}[\ell] = 2\,\mathbb{E}[\ell]$ (for $\ell = L \cdot \tilde{x}$, the LLR signed by the true bit), so the moment estimator $\alpha = 2\,\mathbb{E}[\ell]/\,\mathrm{Var}[\ell]$ recovers the gain that restores consistency. By contrast, *shape* refers to any monotone but non-linear correction $L' = f(|L|)\cdot \mathrm{sgn}\, L$ — one that can treat weak and strong LLRs differently. The finding of this section is that the front end's miscalibration is of the second kind, so the first kind cannot repair it.

The raw LLRs ($4 \cdot \mathrm{Re} \cdot E_s/N_0$ per carrier, clipped to ± 20) are *shape*-miscalibrated: bits with $|L| < 2$ err 11 % empirically versus the 39 % their value implies — the decision-directed noise estimation and per-carrier weighting over-spread confidence downward — while the clipped top of the scale overstates. A single temperature cannot fix this: the moment fit is dragged by the clipped mass, and the fitted $\alpha < 1$ while the weak LLRs are empirically *under*confident, a contradiction only a shape correction resolves. Figure 10 shows the measured shape. In the left panel, the measured error rate starts *below* the ideal curve $P(\mathrm{err}) = 1/(1 + e^{|L|})$ (weak LLRs are better than they claim) and crosses it around $|L| \approx 5$, after which errors run orders of magnitude *above* the claim — and beyond $|L| \approx 9$ the measured reliability actually *decreases* as the claimed confidence rises, all the way into the clipped mass at $|L| \approx 19$. The right panel replots the same data as empirical log-odds versus nominal $|L|$: a calibrated front end would sit on the identity line, but the measured curve rises above it at small $|L|$ (wanting $\alpha > 1$) and falls far below at large $|L|$ (dragging any global fit to $\alpha < 1$ — here $\alpha = 0.67$). No straight line through the origin can be on the correct side of both regions simultaneously; the red curve itself, applied as a lookup, is the monotone reliability map of the next subsection. Note where the measured curve peaks: ≈ 7.5 empirical log-odds at $|L| \approx 9$ — the deployed map's cap of 7.4 is that peak, re-measured independently. This is also the figure that explains the LDPC result of Section 6: exact sum–product consumes these numbers as *absolute* probabilities and pays for every lie, while min-sum and Viterbi only compare them.

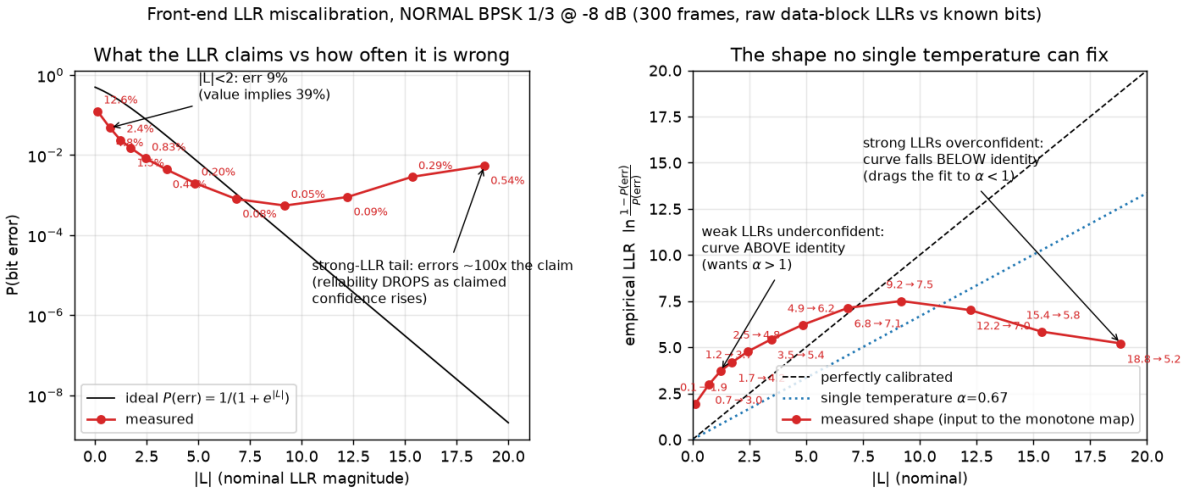


Figure 10: The measured LLR miscalibration shape (NORMAL BPSK 1/3 at −8 dB, 300 frames, raw data-block LLRs against known transmitted bits; reproduce with `experiments/llr_shape.py`, bin table and parameters in `results/llr_shape.json`). Left: claimed versus actual reliability, each bin annotated with its measured error rate. Right: the same data as empirical log-odds, each point annotated with its mapping (nominal → empirical) — above the identity line at small $|L|$, below it at large $|L|$, so no single temperature α can correct both ends.

5.1.1 Measurement Procedure

The data behind Figure 10 (and behind the deployed map of the next subsection) is produced by a self-labeling protocol — the transmitter's bit pipeline is deterministic, so every raw LLR can be paired with the coded bit it was supposed to carry:

1. **Stimulus.** 300 NORMAL-mode frames, BPSK rate 1/3, 27-byte random payloads; each passes the channel simulator with a random start offset (300–1200 samples), a random CFO drawn uniformly from ±60 Hz, and AWGN at the target SNR (−8 dB here, 6 kHz convention). Collection runs as parallel 30-frame chunks with per-chunk seeds, so the run is reproducible and its result is recorded in `results/llr_shape.json`.

2. **Reception.** Each frame goes through the *production* receive chain — preamble detection, CFO correction, tile accumulation, channel estimation, MMSE, LLR formation with the ±20 clip (Section 2.6) — and the raw data-block LLRs are captured *before* any decoding or recalibration. Frames that fail preamble detection are skipped; no selection is made on CRC, so decoding failures do not bias the sample.

3. **Ground truth.** The known payload is run through the TX bit pipeline (FEC encode → pad → interleave → scramble), giving the transmitted coded bit for every LLR position. With $\tilde{x} = \pm 1$ ($+1 \Leftrightarrow$ bit 1, the sign convention), each sample is reduced to the signed product $\ell = L \cdot \tilde{x}$: $\ell < 0$ means the LLR votes for the wrong bit, and $|\ell| = |L|$ is its claimed confidence.

4. **Binning.** Pooled samples are binned by $|\ell|$ (edges $0, 0.5, 1, 1.5, 2, 3, 4, 6, 8, 11, 14, 17, 20$); bins with fewer than 80 samples are dropped. Per bin, the empirical error rate is $\hat{p} = \text{mean}(\ell < 0)$, clamped to $[\,0.5/N,\ 1 - 0.5/N\,]$ (a half-count continuity correction that keeps the log-odds finite when a bin has no errors), and the bin's abscissa is its mean $|\ell|$.

5. **Derived quantities.** The empirical log-odds $\hat{L} = \ln\big((1-\hat{p})/\hat{p}\big)$ give the right panel; the reference curve is $P(\text{err}) = 1/(1 + e^{|L|})$. The single-temperature baseline is the moment fit

$$\alpha \;=\; \frac{2\,\mathbb{E}[\ell]}{\text{Var}[\ell]}, \tag{10}$$

   the consistency condition of a well-calibrated Gaussian LLR ($\text{Var} = 2\,\mathbb{E}$ under the standard approximation $L \sim \mathcal{N}(\pm\sigma^2/2, \sigma^2)$) — the same estimator the runtime header-based temperature fit uses, which is exactly why the figure's $\alpha = 0.67$ is representative of what deployment would apply.

The harness is `experiments/llr_calibration.py` (collection and console reliability tables) and `experiments/llr_shape.py` (the figure and its JSON record); re-running either regenerates all numbers from scratch.

5.2 The Monotone Reliability Map

The correction is a measured monotone map: each $|L|$ bin is replaced by the log-odds of its empirical error rate (10 interior bin edges, saturating at the data-justified maximum of ≈7.4). Capping LLRs at what the data supports also defuses confidently-wrong bits from BSC flips and fades. Measured at the sensitivity edge (NORMAL BPSK 1/3, 40 trials/point):

Table 5: Decodes out of 40 with and without recalibration.

SNR	Raw LLRs	Recalibrated
−8 dB	33/40	38/40
−9 dB	6/40	**29/40**

This is **≈1.5–2 dB of sensitivity** for the existing convolutional chain (Viterbi decoding is scale-invariant but not shape-invariant), and it is what finally lets LDPC sum-product decoding work on this front end (9/40 → 24/40 at −9 dB). The accumulation-limited ROBUST/EXTREME rungs are unaffected *at their operating points* (re-measured at 300 frames/point in Section 6: the map's gain at EXTREME decays to zero by −18 dB, though ≈0.5 dB survives at the deep edge below the rung), and 16-QAM *regresses* under the BPSK-trained map at the ladder rates (re-measured at 300 frames/point in Section 6: −0.5 dB at rate 2/3, though at rate 1/3 the map actually helps — the regression is rate-dependent), so the map is gated to modulations with $\mu \leq 2$ bits/symbol.

5.3 Why the Map Helps Viterbi (and Temperature Cannot)

It is worth being precise about *which* part of the calibration each decoder consumes, because the two decoder families fail in opposite ways:

- **Temperature is provably useless for Viterbi.** The path metric $\mathrm{PM} = \sum_e \beta_e$ is linear in the LLRs (Section 6), so scaling every LLR by any $\alpha > 0$ scales every competing path metric by the same α — the arg-max, and therefore every decoded bit, is unchanged. Figure 11 (left) shows this empirically: the temperature-only curve overlays the raw curve *exactly*, frame for frame, at every SNR. Exact sum–product LDPC is the opposite case: its tanh nonlinearity consumes absolute LLR values, so the scale error alone costs it dB.

- **Shape is what Viterbi actually consumes.** The decoder compares *sums* of LLRs along competing paths, so the decisive quantity is the relative weight of one strong bit against several weak ones. The measured shape (Figure 10) breaks exactly that ratio: a raw $|L| = 9$ bit claims to outweigh four $|L| = 2$ bits, but its empirical log-odds are ≈ 6 while each weak bit is really worth ≈ 4. Two failure modes follow. A single confidently-wrong strong bit (fade, impulse) can outvote several correct weak bits along a path — the map's cap at the data-justified ≈ 7.4 defuses this. And underweighted weak bits waste real information — the map restores their influence in the path sums.

- **HARQ combining needs calibrated addends.** Chase combining adds the LLR streams of two receptions; if the two frames sit on different miscalibrated scales (different SNR, different noise-estimate transients), the sum weights the frames wrongly. Mapping both to honest log-odds first makes addition the theoretically correct combining rule.

- **Puncturing is calibration-neutral.** Depuncture zeros mean "no opinion," and any odd monotone map keeps 0 at 0, so the inserted positions are unaffected by construction.

Figure 11 makes the asymmetry measurable in one sweep (300 frames per point): the same raw LLRs from each received frame are decoded three ways. Temperature-only reproduces the raw decisions bit-exactly at every point — 2 700 frames without a single divergence; the generating script asserts this equality, so scale invariance is enforced, not eyeballed. The monotone map shifts the decode waterfall by ≈1.5 dB: at −9 dB, 63/300 raw versus 216/300 mapped; at −9.5 dB, 15/300 versus 165/300 — consistent with Table 5.

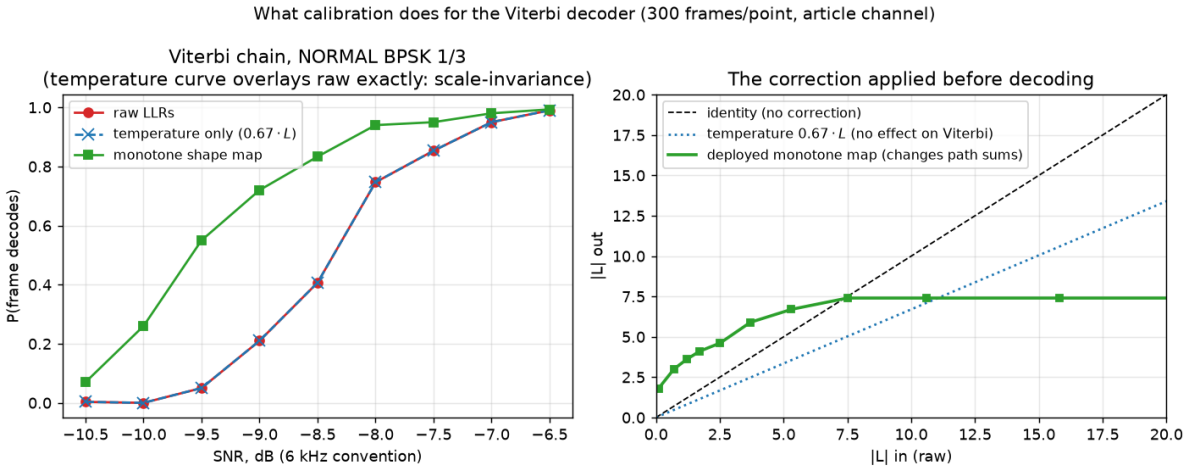


Figure 11: Calibration and the Viterbi decoder (reproduce with `experiments/viterbi_recal.py`; parameters and per-point counts in `results/viterbi_recal.json`). Left: decode probability versus SNR for the same received frames decoded from raw LLRs, temperature-scaled LLRs ($0.67 \cdot L$ — overlays raw exactly), and shape-mapped LLRs. Right: the deployed monotone map against the identity and the temperature line: it boosts weak LLRs above identity and caps the top at the data-justified ≈ 7.4.

5.4 Deployment

Alongside the static map, a per-frame temperature $\alpha = 2\,\mathbb{E}[Lx]/\mathrm{Var}[Lx]$ is fitted from the 96 known bits of each decoded header (a Gaussian-consistency fit), which keeps LLR scales consistent across frames — a prerequisite for the HARQ combining of Section 6. Following the faithful-defaults principle, recalibration is **off** in the plain `Transceiver` and enabled by the link-layer station (`llr_recal="auto"`). The entire rate ladder was re-measured with recalibration enabled; Section 7 carries the resulting sensitivities (mid-ladder BPSK/QPSK rungs gained 0.7–1.4 dB).

6 Coding and Modulation Extensions

6.1 Baseline FEC

The article's FEC — reproduced bit-exactly — is the $K = 7$ convolutional code with generators 133/171/165 (octal) at rate 1/3, punctured to 1/2, 2/3, and 3/4, decoded by a vectorized soft-decision Viterbi [14]. It remains the default and the header is always convolutionally coded, so every extension below is backward compatible.

6.1.1 Encoder and Puncturing

For input bits $u[e]$ (zero-tailed with $K-1=6$ flush bits), the three coded streams are

$$c_p[e] \;=\; \bigoplus_{j=0}^{6} g_p^{(j)}\, u[e-j], \qquad p \in \{0,1,2\}, \tag{11}$$

with taps g_p the binary expansions of $133_8, 171_8, 165_8$. Puncturing transmits only the positions where the rate's mask is 1 (mask column $= e \bmod P$, period P):

$$M_{1/2} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad M_{2/3} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad M_{3/4} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \tag{12}$$

keeping 4/6, 3/6 and 4/9 of the mother-code bits respectively. Rate 1/3 is the all-ones mask.

6.1.2 Soft-Decision Viterbi

The receiver *depunctures* by inserting $\lambda = 0$ ("no opinion") LLRs at the punctured positions — consistent with $L = 0 \Leftrightarrow P(1) = P(0)$ — and runs a max-log Viterbi over the 64-state trellis. With the sign convention of Section 2.6, the expected symbol for a transition from state s' with input u is $x_p(s',u) \in \{-1,+1\}$ ($+1 \Leftrightarrow$ coded bit 1), the branch metric is the soft correlation

$$\beta_e(s',u) \;=\; \sum_{p=0}^{2} \lambda_p[e]\; x_p(s',u), \tag{13}$$

and the path metric recursion is

$$\mathrm{PM}_{e+1}(s) \;=\; \max_{(s',u)\,\to\, s} \big[\, \mathrm{PM}_e(s') + \beta_e(s',u) \,\big], \tag{14}$$

with the winning predecessor stored for traceback (one bit per state per step in the C port — the predecessor of s is determined by which of its two candidates won). Because both β and the LLRs are linear in the demodulator's scale, the decoder is scale-invariant: only the *relative* weighting of carriers and bit positions matters, which is why the front-end shape calibration of Section 5 pays off while a global gain does not.

6.2 LDPC

An LDPC alternative (`ofdm_phy/ldpc.py`) was built as an irregular repeat–accumulate code [10]: $N = 768$, $K = 256$ (rate 1/3), information-node degree 4, girth ≥ 6, linear-time accumulator encoding, and shortening to the payload size. Frames carry it via header `ver=2`, so the receiver needs no negotiation. The construction history is itself a result: a random (4,6) matrix *lost* to the convolutional code (short cycles), and exact sum-product decoding lost several dB end-to-end because it is sensitive to the front end's miscalibrated LLR scale. The shipped decoder is normalized min-sum [4] ($\alpha = 0.8$ float, $\alpha = 0.75$ as $x - (x \gg 2)$ in the integer kernel) — deliberately, because min-sum is scale-invariant. Measured: +0.7 dB over the convolutional code on AWGN at BLER 10 %, compressing to ≈parity end-to-end at −8 dB — the front-end LLR quality, not the code, is the binding constraint (Section 13).

6.2.1 Structure, Encoding, and Decoding

The parity-check matrix has the IRA form $\mathbf{H} = [\,\mathbf{H}_u \mid \mathbf{H}_{\text{acc}}\,]$, where $\mathbf{H}_u$ (512×256, column weight 4, girth ≥ 6 by construction) connects information bits to checks and $\mathbf{H}_{\text{acc}}$ is the dual-diagonal accumulator. Check i therefore reads

$$\bigoplus_{v\in A(i)} u_v \;\oplus\; p_{i-1} \;\oplus\; p_i \;=\; 0, \tag{15}$$

which gives linear-time encoding as a running XOR:

$$p_i \;=\; p_{i-1} \;\oplus\; \bigoplus_{v\in A(i)} u_v, \qquad p_{-1} \equiv 0. \tag{16}$$

Shortening to a k-bit packet ($k \leq 255$) fixes the unused $256 - k$ information positions to zero; the corresponding LLRs enter the decoder as $+\infty$-grade priors.

Decoding is iterative normalized min-sum message passing. With $m_{v\to c}$ the variable-to-check and $m_{c\to v}$ the check-to-variable messages, initialized from the channel LLRs L_v:

$$m_{c\to v} \;=\; \alpha \cdot \Big(\prod_{v'\in N(c)\setminus v} \operatorname{sgn} m_{v'\to c}\Big) \min_{v'\in N(c)\setminus v} \big|m_{v'\to c}\big|, \tag{17}$$

$$m_{v\to c} \;=\; L_v \;+\; \sum_{c'\in N(v)\setminus c} m_{c'\to v}, \tag{18}$$

with hard decisions taken on the total $L_v + \sum_c m_{c\to v}$ and iteration stopping early on a zero syndrome $\mathbf{H}\hat{\mathbf{x}}^{\mathsf{T}} = \mathbf{0}$ (60 iterations maximum; good frames converge in a few). The normalization α compensates min-sum's overestimate of check reliability versus exact sum–product; crucially, Eq. (17) is homogeneous in the LLR scale, which is why min-sum survives the front end's miscalibrated scale where exact sum–product (whose tanh nonlinearity consumes *absolute* LLRs) measurably lost several dB.

Figure 12 is the LDPC counterpart of the Viterbi calibration sweep (Figure 11): the same raw LLRs from each received frame are decoded four ways — both kernels, each on raw and on shape-mapped inputs (the exact-SPA kernel is reconstructed for this measurement; the shipped code retains only min-sum), 300 frames per point. The asymmetry the equations predict is exactly what the sweep measures. Exact sum–product on raw LLRs collapses ≈1.5–2 dB *below* min-sum (at −9 dB: 0/300 versus 112/300; at −8.5 dB: 19/300 versus 192/300) because every overconfident bit in Figure 10 is taken literally by the tanh rule. The shape map fully rescues it (0/300 → 234/300 at −9 dB), landing on par with mapped min-sum — once the LLRs stop lying, the exact rule has nothing left to be hurt by. Min-sum, meanwhile, is *helped* but not gated by the map (≈1 dB: 112/300 → 231/300 at −9 dB): its check update ignores absolute scale, but the map's cap still defuses confidently-wrong bits in the variable-node sums, the same mechanism as in the Viterbi path metrics.

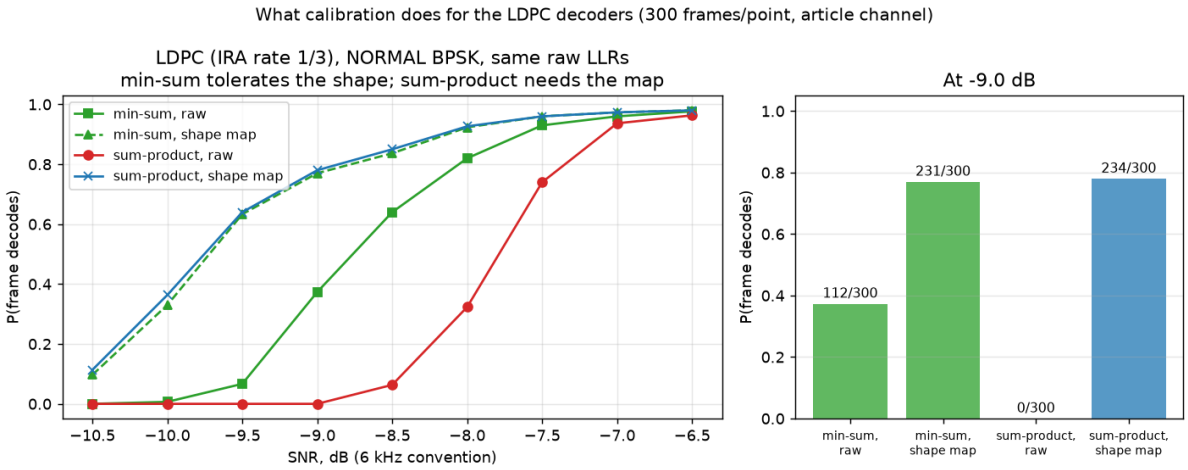


Figure 12: Calibration and the LDPC decoders (reproduce with `experiments/ldpc_recal.py`; parameters and per-point counts in `results/ldpc_recal.json`). The same received frames decoded by normalized min-sum and by exact sum–product, each from raw and from shape-mapped LLRs. Left: decode probability versus SNR. Right: the four variants at −9 dB. Sum–product is unusable on the raw front end and fully recovered by the map; min-sum tolerates the miscalibration and still gains ≈1 dB from it.

6.2.2 The Code Families Head-to-Head, Calibrated

With the front end fixed, the remaining question is which *code* is better — the comparison that the raw front end had rendered moot ("≈parity end-to-end"). Figure 13 puts the fully calibrated chains side by side: convolutional + Viterbi versus LDPC under both kernels, all on shape-mapped LLRs, identical channel-draw sequences per variant, 300 frames per point (binomial uncertainty $\leq \pm 3\,\%$ near the waterfall midpoint). Two conclusions:

- **The LDPC code gain finally materializes end-to-end:** ≈0.3–0.4 dB at the waterfall (at −9.5 dB: 193/300 versus 151/300, i.e. 64 % versus 50 %; at −10 dB: 116/300 versus 83/300). This is consistent with the +0.7 dB AWGN code-level measurement — the front-end miscalibration was masking it, not the code failing to deliver it.
- **Calibrated min-sum ≈ calibrated sum-product:** above −9 dB the two LDPC curves are identical to within one frame in 300. A small exact-SPA edge survives only at the extreme low end (at −10.5 dB: 54/300 versus 40/300; ≈0.1 dB), far below the rung's operating point — so the integer min-sum kernel deployed in the fixed-point and C receivers gives up nothing measurable in the operating region.

The remaining distance to a standards-grade LDPC (≈2 dB theoretical) is now attributable to the code construction itself (short block, $K = 256$, hand-rolled IRA), not to the decoder or the front end — which is precisely the open thread recorded in Section 13.

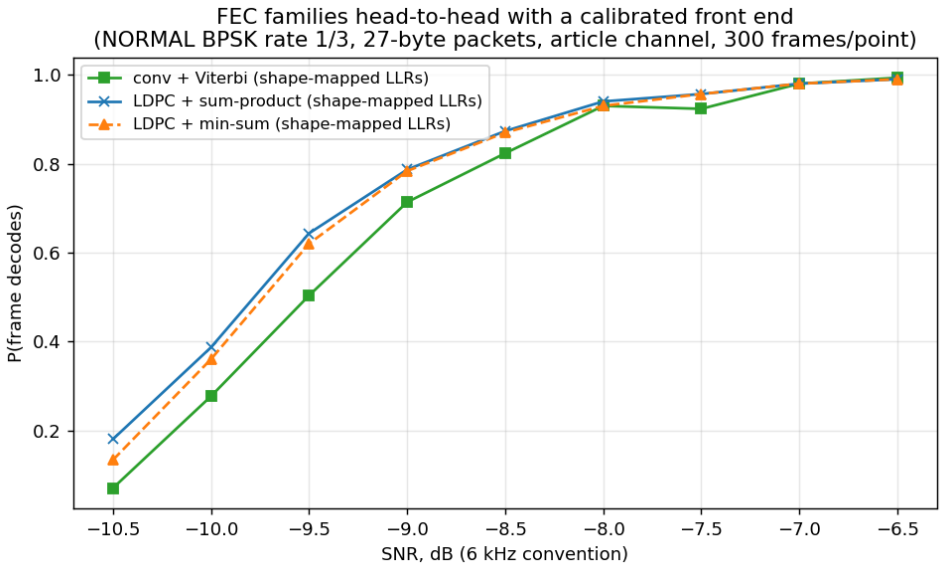


Figure 13: The two FEC families with a calibrated front end (reproduce with `experiments/fec_comparison.py`; every parameter and per-point count is recorded in `results/fec_comparison.json`): convolutional + Viterbi versus LDPC with exact sum–product and with normalized min-sum, all decoding shape-mapped LLRs. LDPC holds ≈0.5 dB over the convolutional chain, and the two LDPC kernels are indistinguishable once calibrated.

6.2.3 The Same Study at the EXTREME Edge

Figure 14 repeats the entire calibration study at the opposite end of the ladder — EXTREME mode (64× tiling, BPSK rate 1/3, the −17.9 dB rung-0 regime), SNR −20.5 to −16.5 dB, 300 frames per point, with the *NORMAL-trained* shape map applied unchanged. Four findings:

- **Scale invariance holds, as it must:** raw and temperature-only Viterbi decodes are bit-identical across all 2 700 conv frames (asserted by the harness).
- **The map transfers, but only below the operating point.** It buys ≈0.5 dB at the deep edge (at −20.5 dB: 138/300 versus 81/300; at −20 dB: 209 versus 157) and the gain decays to *zero* by −18 dB (279 versus 279) — right where the rung actually operates. This refines Section 5's "accumulation-limited rungs are unaffected": accurate at the operating point, where acquisition and accumulation dominate, but not a statement about the region below it.
- **Sum–product's raw-LLR penalty is far milder here** (at −19.5 dB: 152/300 raw versus min-sum's 215, compared with 0/300 versus 112/300 in the NORMAL study): the 64× coherent accumulation reshapes the LLR distribution and shrinks the overconfident tail the tanh rule chokes on. The map still closes the gap completely, and calibrated min-sum and calibrated sum–product are again identical to the frame.
- **The code families reach parity at EXTREME.** The calibrated head-to-head shows no consistent winner (largest deviation 21 frames in 300, alternating in sign) — NORMAL's ≈0.3–0.4 dB LDPC advantage does not replicate. At the deep edge

the loss budget is dominated by acquisition (detection failures count against every variant equally), so the code choice matters correspondingly less.

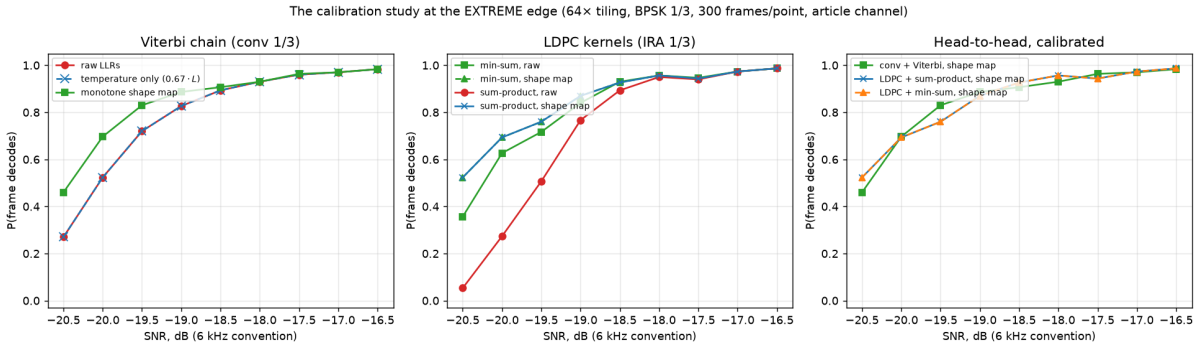


Figure 14: The calibration study repeated at the EXTREME edge (reproduce with `experiments/extreme_recal.py`; parameters and per-point counts in `results/extreme_recal.json`). Left: the Viterbi three-way. Middle: the LDPC kernels. Right: the calibrated head-to-head — parity, unlike Figure 13's NORMAL result.

6.2.4 . . . and for 16-QAM: the Regression Is Rate-Dependent

Figure 15 completes the tour with 16-QAM (NORMAL mode; rate 1/3 in the first two panels for code parity with the LDPC code, whose only rate that is). The result overturns the simple version of the "the map regresses on 16-QAM" statement and replaces it with a sharper one:

- **At rate 1/3 the BPSK-trained map *helps* 16-QAM** by ≈0.4 dB (71/300 → 109/300 at −3.5 dB; 148 → 177 at −3 dB) — the cap's protection against confidently-wrong bits still pays when the code has redundancy to spare.
- **At the ladder rate (2/3) it *regresses*** by ≈0.5 dB, consistently across the waterfall (148/300 → 108/300 at +1.5 dB; 190 → 155 at +2 dB; still 278 → 262 at +4 dB). Under heavy puncturing each surviving bit carries more weight, and a correction trained on BPSK's single LLR class mis-weights 16-QAM's two classes (axis-sign versus inner/outer, Eq. (8)) faster than the thinner code can absorb. The deployment gate ($\mu \leq 2$) protects exactly the rungs that exist — all ladder 16-QAM rungs are rate 1/2 and above — so it remains correct, but for a rate-dependent reason, not a modulation-dependent one.
- **Calibrated min-sum *beats* calibrated sum–product here** (215/300 versus 198/300 at −3 dB; 140 versus 115 at −3.5 dB) — the first configuration in these studies where the two calibrated kernels separate. The mechanism is the same one that sank raw-LLR SPA at BPSK: the BPSK-trained map leaves a *residual* miscalibration on 16-QAM's bit classes, sum–product consumes it as truth, min-sum ignores it. Min-sum is not merely "as good as" the exact rule — it is robust to imperfect calibration in a way the exact rule cannot be.
- The code families otherwise repeat the NORMAL-BPSK pattern at rate 1/3: LDPC min-sum holds a small edge over conv+Viterbi on raw LLRs (185/300 versus 148/300 at −3 dB, ≈0.2 dB), and raw-LLR sum–product shows only a mild penalty (168/300)

— nothing like the BPSK collapse, since at these operating points the 16-QAM LLR distribution rarely reaches the overconfident clipped regime.

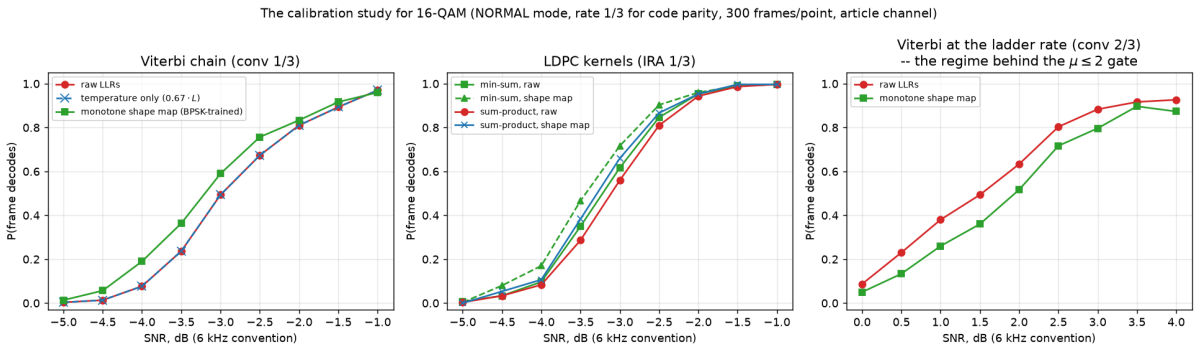


Figure 15: The calibration study for 16-QAM (reproduce with `experiments/qam16_recal.py`; parameters and per-point counts in `results/qam16_recal.json`). Left: Viterbi three-way at rate 1/3 — the map *helps*. Middle: LDPC kernels — calibrated min-sum beats calibrated sum–product for the first time. Right: Viterbi at the ladder rate 2/3 — the map *regresses*, the measured basis of the $\mu \leq 2$ deployment gate.

6.3 16-QAM

Gray-mapped 16-QAM with max-log LLRs (separate inner/outer bit metrics) fills the article's own "QAM could be applied" gap above 0 dB, adding ladder rungs 10–12: 706 bit/s at +0.7 dB, 941 bit/s at +2.6 dB, and 1059 bit/s at +4.7 dB (PER $\leq 10\,\%$, re-measured sensitivities). Figure 16 shows received constellations across that operating range: at +1 dB the sixteen clusters are washed together — which is exactly why the bottom rung pairs the modulation with the rate-1/2 code and soft LLRs rather than hard decisions — while at +5 dB the MMSE equalizer resolves the 4×4 grid cleanly enough for rate 3/4.

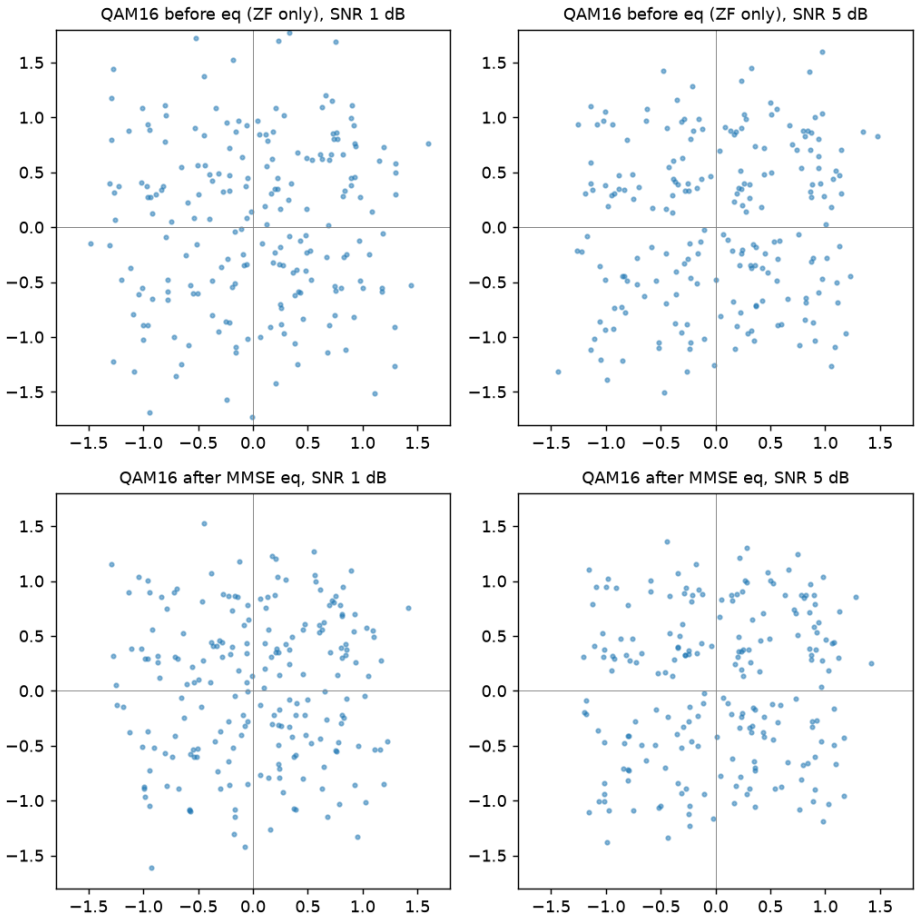


Figure 16: 16-QAM received constellations over the article channel (`experiments/figures.py`): before equalization (ZF pilot estimate only, top) and after Wiener-refined MMSE equalization (bottom), at the bottom (+1 dB) and top (+5 dB) of the 16-QAM operating range.

6.4 Chase-Combining HARQ

Because a CRC failure exports the data-block LLRs (Section 2), retransmissions can be soft-combined in the Chase manner [3]: the station stores the failed attempt's LLRs, and on the next decode tries the fresh frame alone, then fresh + stored (LLR sum), accepting whichever passes CRC (Figure 17). The combining is *blind* — the ARQ sequence number lives inside the undecodable payload, so the receiver cannot know the retransmission is the same frame — and *safe*, because the CRC arbitrates. Measured at −8.5 dB: second-attempt success 15/20 versus 10/20 without combining, worth ≈3 dB on a same-rung retransmission in the noise-limited regime. Fade losses benefit less: they are mostly synchronization-level failures that leave no LLRs to combine.

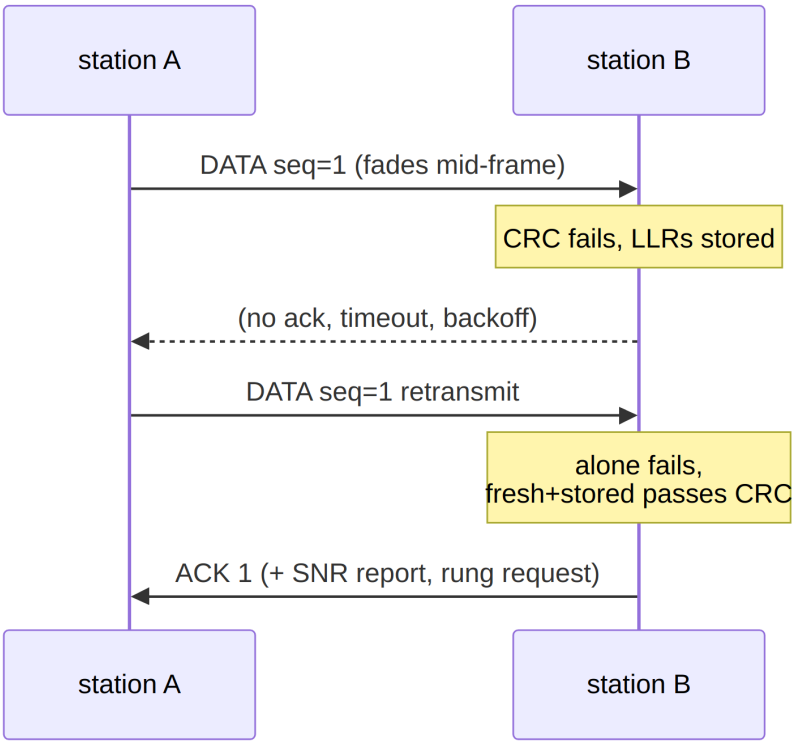


Figure 17: Chase-combining HARQ. No extra signalling: the stored LLRs come from the failed decode itself.

7 Link Layer

Two half-duplex stations form *two directed links*, each governed by its receiver — only the receiver knows its own noise floor. There is no shared "link mode" to negotiate; each side requests what it can hear.

7.1 The Rate Ladder

Thirteen operating points (mode × modulation × code rate, dominated rungs removed) span **7.8–1059 bit/s** over $-17.9 \ldots +4.7$ **dB** (Table 6). Every sensitivity is measured, not derived (`experiments/ladder_sweep.py`, 48 packets/point, recalibrated receiver; full PER curves in `results/ladder_recal.json`). Requests and grants are ladder indices carried in the link-control word. One quirk is documented: rung 1 (ROBUST BPSK 1/3) is dominated by rung 2 (faster, equally sensitive) and is kept only for ladder-index stability — it is never selected.

Table 6: The measured rate ladder (PER $\leq 10\,\%$ sensitivities, recalibrated receiver, 6 kHz-band SNR).

#	mode/mod/FEC	bit/s	dB	#	mode/mod/FEC	bit/s	dB
0	EXTREME BPSK 1/3	7.8	−17.9	7	NORMAL QPSK 1/2	353	−5.3
1	ROBUST BPSK 1/3	31	−11.7	8	NORMAL QPSK 2/3	471	−3.8
2	ROBUST BPSK 1/2	46	−11.8	9	NORMAL QPSK 3/4	529	−2.2
3	ROBUST QPSK 1/3	62	−11.3	10	NORMAL QAM16 1/2	706	+0.7
4	NORMAL BPSK 1/3	118	−7.6	11	NORMAL QAM16 2/3	941	+2.6
5	NORMAL BPSK 1/2	176	−7.3	12	NORMAL QAM16 3/4	1059	+4.7
6	NORMAL QPSK 1/3	235	−7.0				

7.2 The Link-Control Word

Every frame — data, ACKs, everything — piggybacks a 20-bit word in `Data.reserved`: `seq(2) | ack(2) | req_rung(4) | snr_report(4) | freq_corr(5) | flags(3)`. The SNR report uses 2 dB steps; the frequency-correction field carries ±120 Hz in 8 Hz steps (Section 8); flags are last-fragment, no-data (ACK-only), and priority-stream.

7.3 Rate Adaptation: Three Feedback Paths

Figure 18 shows the controller. On the receive side, per-frame SNR estimates pass through a fade-aware filter (the second-lowest of recent measurements, age-windowed to 60 s) before driving the rung request: upshift requires the target rung's sensitivity plus 2.5 dB of held margin, downshift triggers below 1 dB (the hysteresis band prevents ping-pong), and inbound *silence* decays the request and purges the measurement history — without the purge, stale pre-fade measurements resurrect a high rung the moment one frame gets through.

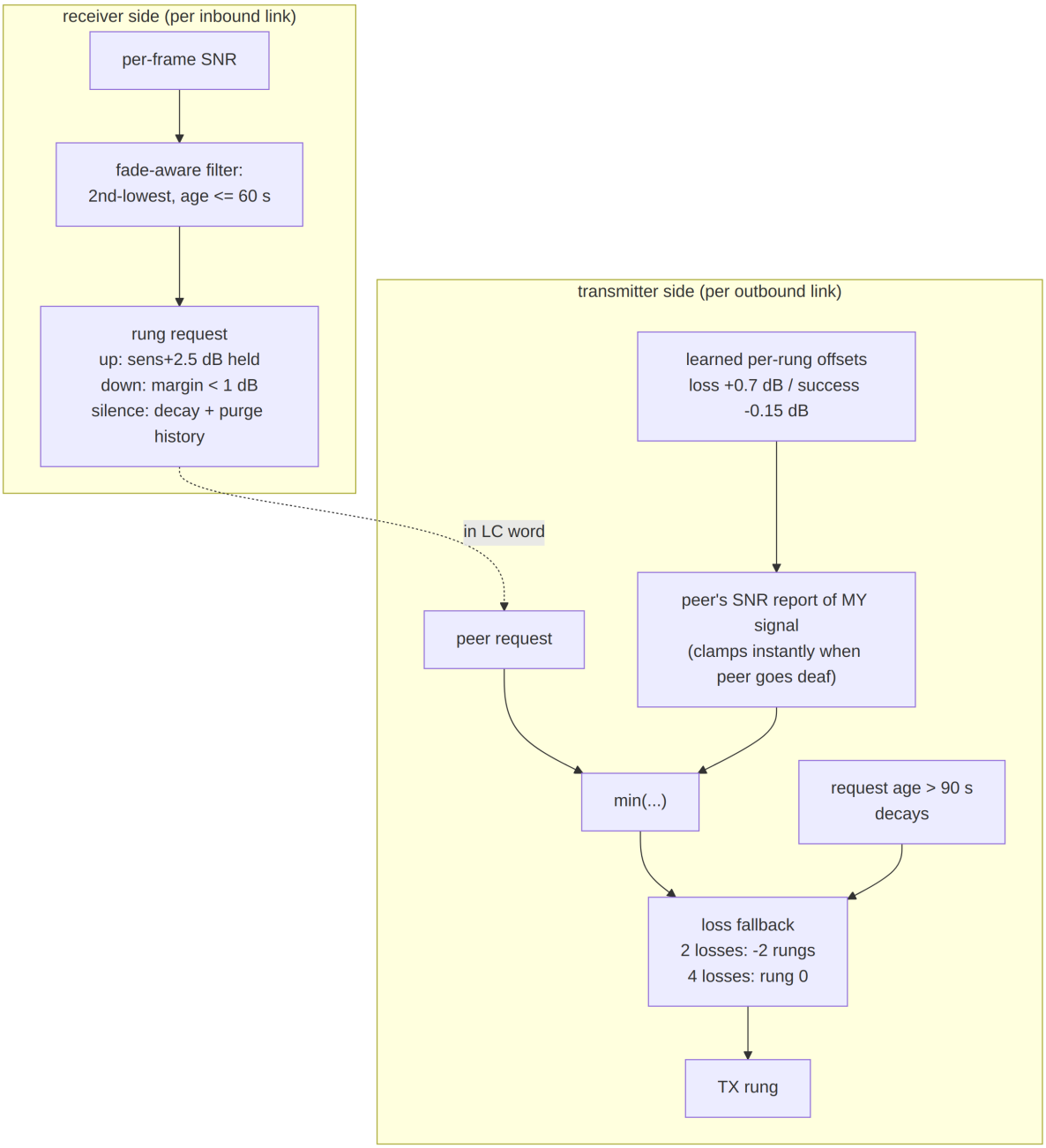


Figure 18: Receiver-driven rate adaptation. The three feedback paths have three different latencies: the peer's explicit request (one round trip), the peer's SNR report of *your* signal (instant clamp when the peer goes deaf), and loss-driven fallback (no feedback at all).

On the transmit side, the rung is the minimum of the peer's request and a cap derived from the peer's SNR report of our own signal, adjusted by *learned per-rung offsets* (+0.7 dB on loss, −0.15 dB on success — correcting the static table per deployment), with loss fallback (2 consecutive losses → −2 rungs, 4 → rung 0) and staleness decay. Bootstrap is trivial by construction: both sides start at rung 0 (EXTREME), which works whenever anything works, and one decoded exchange carries enough measurement to jump directly to the SNR-indicated rung.

7.4 ARQ, QoS, and Preemption

ARQ is stop-and-wait with piggybacked ACKs (the 2-bit sequence number suffices), enhanced by the HARQ combining of Section 6. An invariant worth recording: sequence numbers 0–7 are all legitimate, so "nothing received yet" is represented by `None`, never a magic value — the initial implementation used 7 as a sentinel and deadlocked the moment a real frame carried sequence 7.

QoS is policy, not machinery: three classes (control > interactive > bulk) with strict priority; per-class air-time caps size fragments (a 27-byte EXTREME frame is 43 s of air time, so latency budgeting happens at segmentation); control and ACK traffic rides one rung below bulk for extra margin. The priority stream *preempts* an in-progress bulk message at fragment boundaries — the priority-stream LC flag selects one of two reassembly buffers — so an urgent message injected mid-transfer arrives ∼23 s before the bulk completes instead of after it.

7.5 Burst Windows and Streaming

Bulk transfers use selective repeat: a window of fragments goes out back-to-back, answered by one bitmap acknowledgment (NACK-by-omission). Where the PHY offers streaming (Section 2.3), the whole window is sent behind a *single* preamble and header rather than one preamble per fragment. The packets are byte-identical to per-frame burst fragments — one spare bit in the sub-header's index byte marks a fragment as streamed — so a receiver without streaming support still decodes the first block of every burst as an ordinary fragment. That property is what makes the fallback safe rather than fatal.

The window is chosen once, at engage, as min(operator ceiling, buffer cap, fragment count), then clipped by an air-time cap so that a single transmission can never outlast a plausible fade — everything in a streamed window is exposed before any of it is acknowledged. Sizing the window to the transfer is what makes a short transfer cost exactly one acknowledgment, which is the deferred-NAK arrangement of LTP and CFDP arrived at through a constant rather than new wire format. Measured on a 14 kB file over the virtual channel: window 4 takes 14 transmissions, window 8 takes 9, window 16 takes 6.

Resizing the window *during* a transfer was implemented and then reverted on the evidence. Halving it on each streamed-window timeout, rather than striking out of streaming, collapsed the window $16 \to 8 \to 4 \to 2 \to 1$ with no path back — it grows only on a fully acknowledged window, which never happens inside a fade — and produced 196 transmissions with 72 timeouts against 134 and 9 for the strike-out path. On a clean channel the two were indistinguishable. When a fade is what breaks a burst, the correct response is to stop streaming, not to stream less.

Three conditions end streaming for a transfer, and they are not the same kind of statement. A bitmap that acknowledges at most one fragment of a window of three or more is the signature of a peer decoding only the first block: that is a statement about the *peer*, and it is remembered across transfers, because a receiver unable to follow a stream will not learn to between them. Repeated timeouts and a transmitter-side build refusal describe the *channel* and the local buffers respectively, and deliberately do not stick — a fading channel raises timeouts against a peer that streams perfectly well, and making that sticky would disable streaming for the session on a capable link. The peer verdict is not permanent either, since a deep fade can forge the one-fragment signature: streaming is probed again after eight transfers, which bounds a wrong verdict to one wasted window per retry period while a genuinely incapable peer costs two windows per period instead

of two per transfer.

7.6 The Reply Timer

A station that has transmitted and expects an answer must decide how long to wait. The budget splits along what is knowable. The reply's *air time* is computable exactly from the rung and the expected reply length, and it swings by a factor of forty across the ladder, so smoothing it would be meaningless. Everything else — the peer's turnaround, its decode time, its wait for the channel to clear, its scheduling, and the error in our guess of which rung it will answer at — is latency that can only be measured.

The air term is therefore computed and the overhead is estimated in the manner of RFC 6298: a smoothed mean and variance ($\alpha = 1/8$, $\beta = 1/4$) with the budget set at $\mathrm{srtt} + 4\,\mathrm{rttvar}$. Karn's rule keeps ambiguous exchanges out of the estimator — a reply that follows a retransmission cannot be attributed to a particular transmission — and a timeout doubles the timer until a clean exchange clears the backoff. Against a peer that consistently takes 0.40 s, the 2.30 s bootstrap guess converges to 0.40 s.

This replaces a fixed margin that had already caused a misdiagnosis. When the host could not sustain the demonstration harness's 25× time compression, the two stations' sample-derived clocks drifted apart — the mostly-transmitting station stayed current while the one decoding three receivers fell behind — and the transmitter timed out before the receiver had finished the burst in its own timeline. Every timeout landed on one station and none on the other, with the reply arriving immediately after each: a signature that reads exactly like a protocol failure and is not one.

7.7 Simplex Channel Access

Both stations share one frequency and are deaf while transmitting. Figure 19 shows the access state machine. The load-bearing rules: carrier sense before keying; the listen window is sized from the *expected reply's* air time at the peer's rung; a timeout on a *busy* channel is not a loss (the peer may be replying at a slower rung); random 1–6 s backoff breaks the symmetry of two stations keying together; and a station replies only to frames that carried data — no ACK-of-ACK loops.

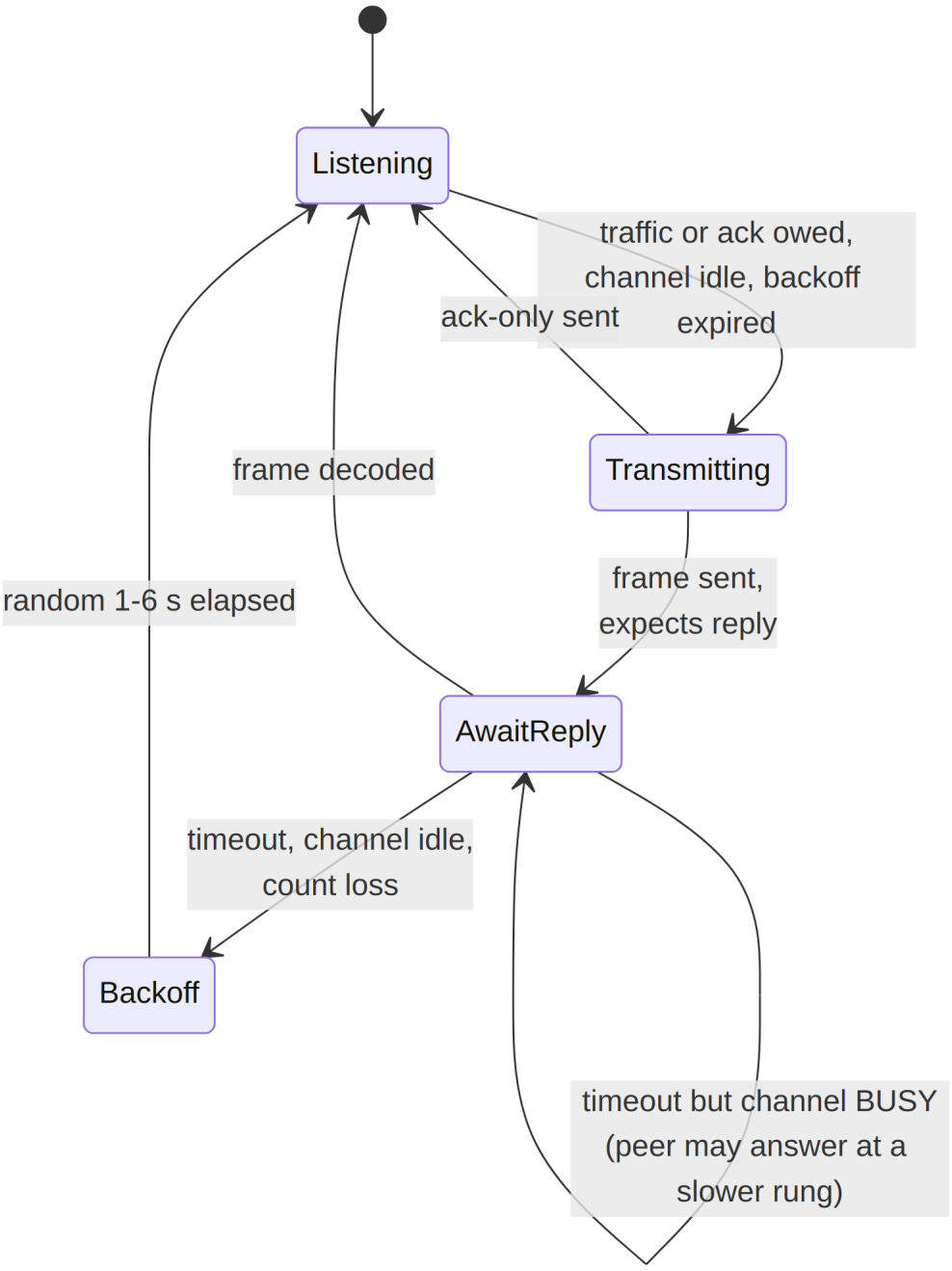


Figure 19: Simplex channel-access state machine (`LinkStation.poll_tx`).

The whole-system test (`experiments/simplex_session.py`) runs two stations with bidirectional QoS traffic and a -14 dB fade in the middle of the bulk transfer: all messages deliver bit-exact in QoS priority order, the fade costs four timeouts and one fallback-and-recover cycle, and the session terminates without deadlock at 11–22 bit/s effective on the shared channel. Figure 20 shows the controller-level simulation: bootstrap at EXTREME, a fast climb to the QPSK rungs, loss-driven fallback within two frames of a sudden fade, and recovery.

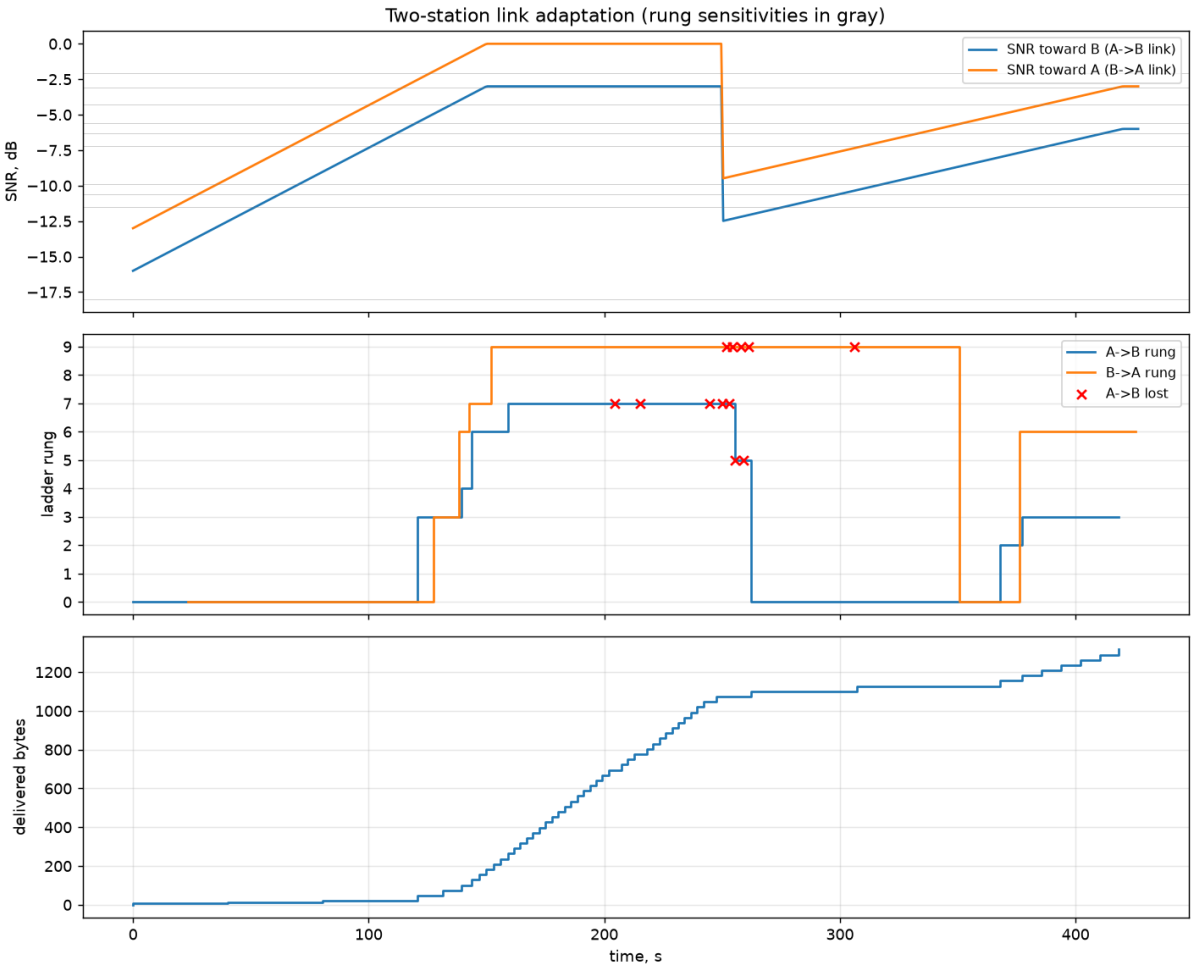


Figure 20: Controller-level two-station simulation (`experiments/link_adaptation.py`): asymmetric channel, −16 dB start, sudden fade; 1290/1500 bytes delivered at 24 bit/s effective goodput including ACK overhead and turnarounds.

7.8 Broadcast: Delivery Without Acknowledgment

Speech and telemetry cannot wait for a retransmission, so they need a mode in which nothing is repeated and the receiver's only contribution is a report that it was heard. The transport for this already exists: a broadcast is a streamed burst (Section 2.3) with the acknowledgment machinery subtracted — no acknowledgment request, no window, no selective repeat, no reply timer. What has to be added is the one thing a burst does not need. A burst has exactly one listener, who was already there; a broadcast must let a receiver that tunes in mid-transmission join. Every group therefore re-sends the preamble and opens with a SYNC frame carrying the stream descriptor, so a late listener acquires at the next group boundary rather than waiting for the broadcast to end.

Framing is two bytes per frame: a SYNC bit, an EOS bit, a six-bit sequence number used *only* for loss statistics, and the count of valid payload bytes in that frame. The obvious model to copy would be RTP, and copying it would be a mistake: its twelve-byte fixed header is 91 ms of air time at the top rung, so a 20 ms-packetised stream would carry 1.75 bytes of payload behind 12 bytes of header. What survives the transplant is what a receiver cannot work out for itself — sequence, stream boundaries and payload type. What does not is the timestamp (this link's delay is deterministic, so a frame's position

in the stream *is* its timestamp), the synchronisation source identifier, and the per-packet payload type, which rides the SYNC frame instead.

Carrying the length in *every* frame rather than only in the EOS frame costs about 4 % at 26-byte frames and buys a property that only matters when nothing is retransmitted: every frame is self-delimiting, so losing the one frame that would have carried the length does not mis-size the payload.

The rate ladder decides what can be carried at all. Codec2 at 700 bit/s fits only rungs 11–12 (941 and 1 059 bit/s user rate, +2.6 and +4.7 dB); Codec2 450 fits rungs 8–9; below rung 8 this mode carries telemetry only, down to EXTREME's 7.8 bit/s. That envelope is set by the waveform, not by the protocol above it.

Table 7: Broadcast delivery, 200-byte payload in groups of four (`experiments/broadcast_demo.py`, 20 trials per point). Nothing is retransmitted, so "recovered" is payload bytes reassembled against bytes sent.

	all recovered	≥90 %	breaks down
speech, rung 12 (16-QAM 3/4)	+9.5 dB	+5.0 dB	+2 dB
telemetry, rung 7 (QPSK 1/2)	+1.5 dB	−3.0 dB	−6 dB

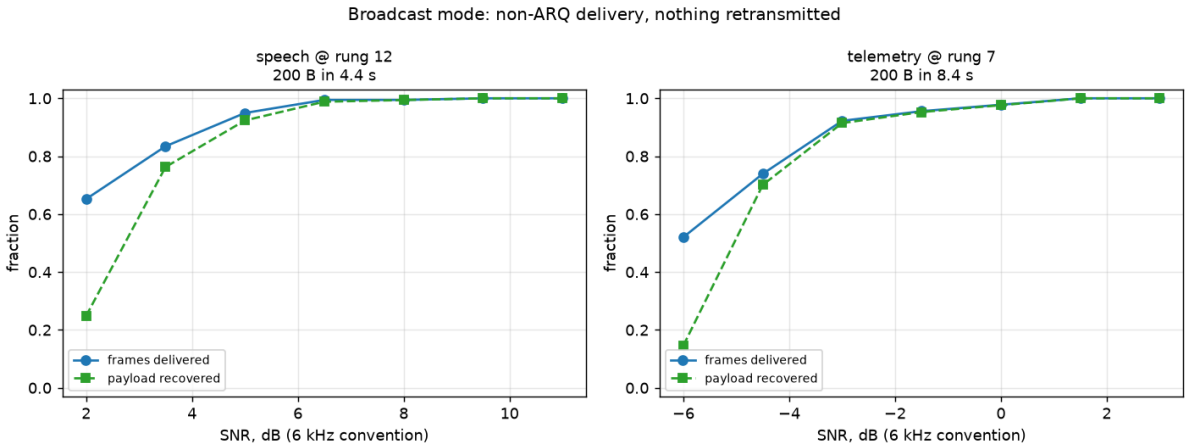


Figure 21: Non-ARQ delivery against SNR. Frames delivered runs above payload recovered at the low end because a frame lost from the middle of a group costs its own bytes while still being counted in the span.

Telemetry reaches 90 % delivery at ≈ −3.1 dB against rung 7's −5.3 dB acknowledged-mode sensitivity. That ≈2 dB is the honest price of non-ARQ: a frame that fails is simply gone, and a group whose preamble is missed takes four frames with it. A receiver that tuned in half-way through the first group of seven recovered 90 % of the remainder — it cannot, of course, recover what it never heard.

Two findings from building this generalise beyond broadcast.

The first concerns the detector. `detect_preamble` takes a *global* argmax over whatever slice it is handed, so a slice containing several groups makes it lock the strongest preamble rather than the first, and every group before that one is silently lost — 300 samples of lead-in were enough to skip a whole group. The slice must therefore be bounded on *both* sides so that it holds exactly one preamble. Starting the slice on a preamble is not

sufficient; nor can the far edge be found by searching forward for the next preamble, since from inside a group the detector false-locks on data. The bound is geometry — preamble plus header plus blocks — and because the first decode necessarily runs before that geometry is known, the receiver learns the extent from it and then restarts the walk with the bound in place. Anything that scans a recording containing multiple frames has the same exposure.

The second concerns LLR calibration (Section 5), which matters more here than under ARQ for a simple reason: a frame the decoder cannot recover is not retried, it is lost. Enabling the measured reliability map moved telemetry from 30.6 % to 69.8 % of payload recovered at −4.5 dB and from 3.4 % to 12.0 % at −6 dB, while changing nothing above −3 dB where delivery is already near perfect. It is therefore on by default in this mode. The map is gated to $\mu \leq 2$ because it was trained on BPSK/QPSK statistics, and the speech column of the sweep came back *bit-identical* with the map enabled — an incidental but clean confirmation that the gate does what it claims.

What broadcast gives up is the feedback path. The rate-adaptation mechanisms of Section 7 all depend on the peer replying, and here it does not. Either the sender schedules periodic listening gaps in which receivers return the statistics above, at the cost of the continuous transmission that makes broadcast cheap, or it does not adapt at all: announce a rung in the descriptor and hold it. STANAG 5066's non-ARQ delivery mode takes the second course, and it is the right default — a broadcast has no single listener to adapt to.

8 AFC Frequency Netting

The receiver measures the peer's carrier offset on every decoded frame anyway (`stats.cfo_hz`), so the link closes the loop: the 5-bit LC field carries "shift your carrier by $-X$ Hz" requests (±120 Hz per frame, 8 Hz steps, ±12 Hz deadband), and the peer trims its *shared* reference oscillator. Trimming the shared reference moves TX and RX together — which is exactly why asking the peer to correct beats retuning locally (Section 9). Corrections are applied at **half gain**: both sides act on measurements that are one frame stale, and full-gain corrections would swap the offset between the stations each turn instead of converging (Figure 22).

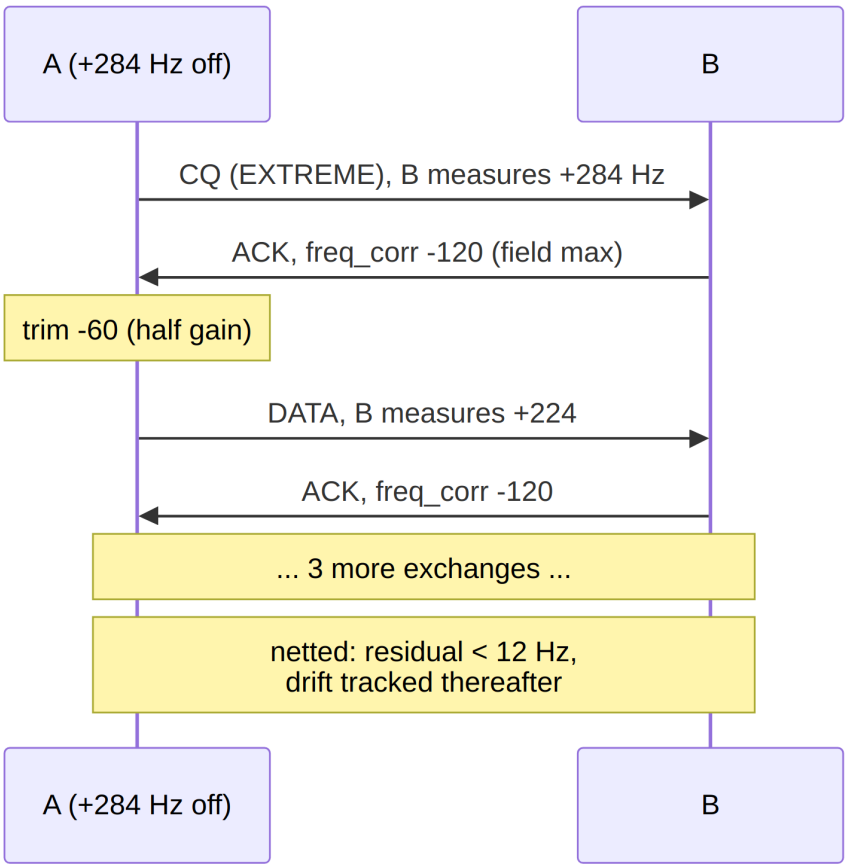


Figure 22: AFC netting from a +284 Hz initial offset.

8.1 Guard Rails

The loop observes only the *differential* offset, so without bounds the pair's common frequency random-walks over a long session — out of the channel slot, into bystanders' passbands, or off the rig's trim range. Two mechanisms bound it:

- `afc_max_trim_hz` (default ±150 Hz) hard-clamps each station's *cumulative* trim. Requests beyond the budget are partially honoured or refused, the peer's own headroom carries the rest, and any residual is left to the modem's ±300 Hz CFO tolerance — the link keeps working, just without full netting.
- `afc_anchor=True` designates a station (e.g. the CQ caller) that never trims, pinning the absolute frequency entirely; the peer does all the correcting.

8.2 Measured Behaviour

`experiments/afc_netting.py` runs two worn-TCXO rigs on 14.2 MHz starting +284 **Hz apart** (near the modem's ±375 Hz limit) with opposing thermal drifts: netted to within the deadband in ≈5 correction exchanges (≈104 s — EXTREME bootstrap frames dominate the wall clock), steady-state $|\mathrm{CFO}| \leq 11$ Hz under continuing 0.09 Hz/s relative drift, 400/400 bytes delivered. The guard scenarios behave as designed: default budgets net fully (final |CFO| 3 Hz, trims clamped at −150/+144 Hz); deliberately tight ±40 Hz budgets stop at their limits leaving a 222 Hz residual *by design*, with the link still delivering

on CFO tolerance alone; an anchored station A forces B to absorb the full +292 Hz and the pair's absolute frequency never moves. Beyond robustness margin, a netted link could shrink EXTREME's per-symbol frequency-search range (compute) and eases mask-grid detection at the extremes.

9 RF Layer and Channel Models

Two channel models coexist. The audio-domain simulator (`channel.py`) is the article's model — timing offset, CFO with quadratic drift, optional Rayleigh fading, multipath $[1, 0, 0.4, 0, 0, 0.2]$, AWGN, BSC block flips ($p = 10^{-3}$) and BEC block erasures ($p = 2 \cdot 10^{-2}$) — and drives most sweeps. The RF layer (`rf.py`) models the actual transceiver chain, so carrier offsets *emerge from oscillator physics* instead of being injected as parameters.

9.1 The SSB Chain

Simulation runs at a 48 kHz IF rate with a 12 kHz carrier standing in for the RF carrier; LO errors are specified in ppm of the nominal RF frequency (e.g. 7.1 MHz), so audio-band offsets come out physically correct (Figure 23).

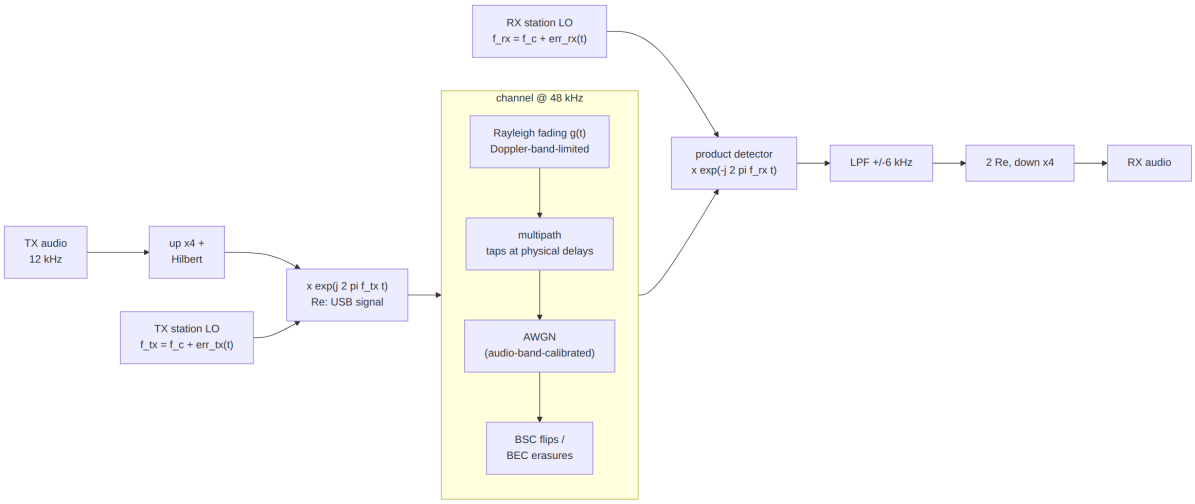


Figure 23: The RF chain: USB modulator, 48 kHz channel (fading, multipath, AWGN, BSC/BEC), product detector.

9.2 The LO Model

Each `StationRF` has *one* reference oscillator shared by its TX and RX, as in a real transceiver:

$$\mathrm{err}(t) \;=\; f_{\mathrm{rf}} \cdot \mathrm{ppm} \cdot 10^{-6} \;+\; \mathrm{drift}_{\mathrm{Hz/s}} \cdot t \;+\; \mathrm{trim}_{\mathrm{Hz}},$$

and the audio CFO between two stations is $\mathrm{err}_{tx}(t) - \mathrm{err}_{rx}(t)$ (Figure 24). Validated against the modem's own CFO measurements to 0.1 Hz at +99.4, −106.5, and +284.0 Hz scenarios. `trim_hz` is the AFC hook of Section 8.

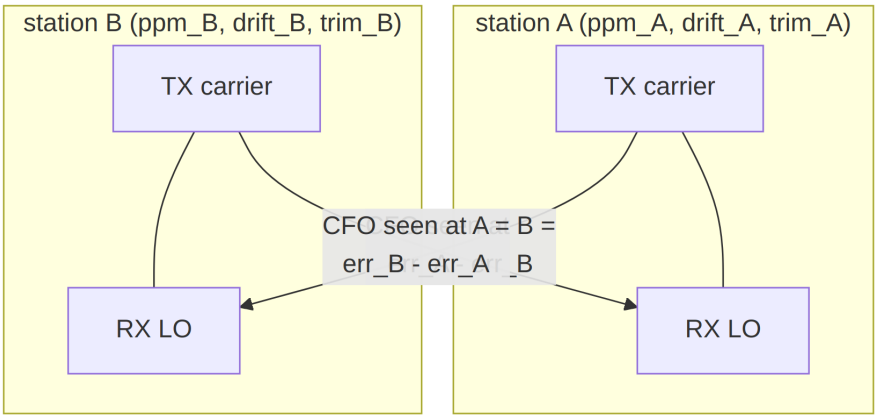


Figure 24: Per-station shared-reference LO model: where CFO comes from.

9.3 Calibration Subtleties

Three traps are documented in code comments because each silently costs dB if gotten wrong:

- **AWGN sizing.** The product detector's Re(·) halves *uncorrelated noise* power but not the coherent signal, so the RF noise variance factor is $2 \cdot P_{\text{sig}} \cdot 10^{-\text{SNR}/10}$ — not the naive bandwidth ratio, which is 3 dB off. With the correct factor the audio-band SNR convention matches the audio-domain simulator, so all measured sensitivities carry over (9/10 decodes at −5 dB through the full RF chain).
- **Rayleigh fading.** Complex Gaussian band-limited to the Doppler bandwidth (0.1–0.5 Hz, typical HF QSB), unit mean power, applied to the analytic signal; AWGN is sized from the *average* faded power, so instantaneous SNR swings around the nominal value.
- **Multipath at RF.** The audio-domain tap delays are physical, so taps map to interpolation-factor-spaced positions at the RF rate.

9.4 The Recorded Demonstration

`experiments/demo_wav.py` records a complete two-station session through this chain: station A at +6 ppm and B at −8 ppm on 7.1 MHz (A→B CFO ≈ +99.4 Hz, handled invisibly by the protocol). The output WAV is the audio of a *passive monitor* receiver with its own +2 ppm LO — A's frames appear at ≈ +28 Hz and B's at ≈ −71 Hz in the blind-decoded transcript (Appendix C), per-station CFO fingerprints with the thermal drift visible across the session.

10 Fixed-Point RTL Reference Model

`ofdm_phy/fixed/` is an integer-only twin of the modem, structured the way an FPGA/ASIC datapath would be. It is the golden reference a future RTL implementation verifies against; `experiments/fixed_point.py` (24 checks, Appendix B) cross-validates it against the float model continuously.

10.1 Receiver Datapath

Figure 25 shows the complete integer receive path, from int16 audio to decoded packet. The model covers the *full* frame family: convolutional and LDPC FEC, BPSK/QPSK/16-QAM, all three link modes, HARQ chase combining, and the optional calibrated-LLR mode — on the transmit side as well: `build_frame(fec="ldpc")` emits `ver=2` frames (the LDPC accumulator encoding is already pure integer, so only the header/codec switch was needed), and the Q15 constellations cover BPSK, QPSK, and Gray 16-QAM (per-axis levels $\{\pm 1, \pm 3\}/\sqrt{10}$).

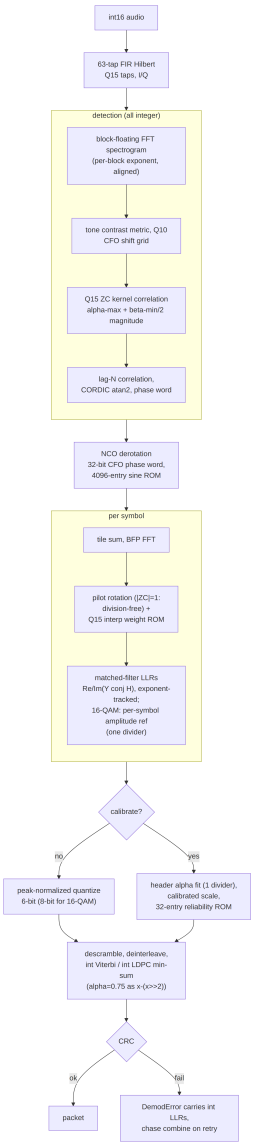


Figure 25: Fixed-point receiver datapath (all integer).

10.2 Primitive Blocks

Table 8: Fixed-point primitives and their measured quality.

Block	Structure	Quality
FFT	radix-2 DIT, Q15 twiddle ROM, per-stage $\gg 1$ (built-in $1/N$), BFP wrapper	59 dB SQNR
Hilbert	63-tap type-III FIR, Q15	61 dB SQNR
NCO	32-bit phase accumulator, 4096-entry Q15 sine ROM	-67 dBc
CORDIC	16-iteration vectoring, angle in phase-word units	< 0.001 rad
Viterbi	6-bit LLRs (8-bit for 16-QAM data), ± 1 expected symbols (adds only), int32 metrics	exact
LDPC	integer normalized min-sum, $\alpha = x - (x \gg 2)$	matches float min-sum

10.3 RTL-Oriented Conventions

- **CFO is a 32-bit phase-increment word** end-to-end (one turn $= 2^{32}$); Hz exists only at API boundaries.
- **Divisions are minimised.** Channel estimation is division-free ($|\text{ZC pilot}| = 1$, so the estimate is a rotation); pilot interpolation is a Q15 weight ROM. The remaining dividers: the calibration α fit (one per frame), the 16-QAM amplitude reference (one per symbol), and the detection metrics.
- **Scale tracking.** BFP exponents are carried through every energy and LLR comparison (scale $\propto 2^{2 \cdot \text{exp}}$); ZC correlation magnitude uses the α-max + β-min/2 approximation.

Two intentional deltas versus the float model: NORMAL-mode tone detection uses a 256-point FFT (float: 128) so the coarse CFO grid is fine enough for an unambiguous lag-N residual; and the demodulator uses the slew-limited frequency-search tracker for *all* tile factors — the float model's polynomial phase fit is not RTL-friendly, and measurably weaker at the edge anyway.

10.4 Measured Results and One Surprise

The fixed TX waveform correlates 0.99998 with the float waveform; the full fixed/float TX×RX cross-matrix decodes; NORMAL-mode PER is within $\approx$0.5 dB of the float receiver, and ROBUST/EXTREME decode at $-11/-17$ dB. The surprise: the default 6-bit peak-normalized LLR quantization already acts as a *compressive recalibration* (Section 5), and combined with the frequency-search tracker the fixed receiver **matches or beats the recalibrated float chain at $-9/-10$ dB** (22 vs. 20 and 11 vs. 5 of 30 trials). The calibrated-LLR mode (integer header α fit plus a 32-entry reliability ROM measured on the fixed chain itself) therefore buys a *stable cross-frame LLR scale* — what makes HARQ combining legitimate — rather than extra PER.

The 16-QAM rungs were swept separately (`experiments/fixed_qam16_sweep.py`, 48 packets/point, fixed TX with identical samples into both receivers). The first sweep exposed a ∼2 dB gap on the puncture-weak high rates (+4.6/+6.3 dB versus float +2.6/+4.5 at rates 2/3 and 3/4): 16-QAM max-log LLRs span a much wider dynamic range than BPSK (inner/outer bits times per-carrier gain), and the 6-bit peak-normalized quantization compresses them. The fix is a *per-modulation quantizer*: data blocks with $\mu = 4$ use an 8-bit peak-normalized quantizer (±127), while $\mu \leq 2$ keeps the 6-bit path, whose compression is precisely what helps at the BPSK edge. Peak and mean normalization measured equivalent at 8 bits — the win is the resolution — so the RTL cost is two extra LLR bits on the 16-QAM data path only. Re-swept, the fixed receiver measures +1.5/ + 2.9/ + 5.3 dB for rates 1/2, 2/3, 3/4: within 0.3–0.8 dB of the float receiver everywhere.

Robustness note: the tiled chain survives up to 65 % contiguous audio erasure (the 4× tiles keep partial symbols alive); the HARQ path is validated by a deterministic test with two 75 % complementary erasures whose union covers every sample at least once.

10.5 Gated Two-Stage Frequency Search

The first-symbol residual-CFO search — 275 full-symbol hypotheses at EXTREME, the demodulator's dominant burst in a firmware budget — was restructured as coarse-then-fine: a quarter-length coarse pass (tiles/4 accumulation, whose 4× wider frequency mainlobe tolerates a 4× coarser grid) ranks hypotheses, and only the top-3 ±5-step windows run at full precision, ≈5.5× fewer derotated samples. The coarse metric is ∼6 dB noisier than the full symbol, and a hard coarse decision measurably costs ∼0.5 dB at the EXTREME sensitivity edge — so the fast path is taken only when the coarse top-1/median contrast clears a measured gate (2.25×; edge frames sit ≤ 2.2×, frames with 5 dB of margin ≥ 2.7×), and below it the exhaustive grid runs unchanged. The same gated structure was then back-ported to the *float* demodulator's per-symbol frequency search — where it pays on every symbol, not just the first, because the float chain has no slew-limited tracker. Figure 26 shows the noise-sensitivity A/B (`experiments/coarse_search_ab.py`, 36 packets/point, identical channel realizations per arm), including the ablation that justifies the gate: the quarter-length coarse pass *without* the gate (dotted) shifts the EXTREME waterfall right by 0.5–1 dB (fixed RX at −18 dB: PER 0.31 vs. 0.11), while the gated search (dashed) is *identical* to the exhaustive grid at every measured point, both models — lossless by construction, with the full grid reduced to a low-margin fallback. ROBUST tolerates even the ungated coarse pass at parity: its coarse stage (4 of 16 tiles) retains enough detector SNR at that mode's shallower edge, so the gate specifically protects the deepest mode.

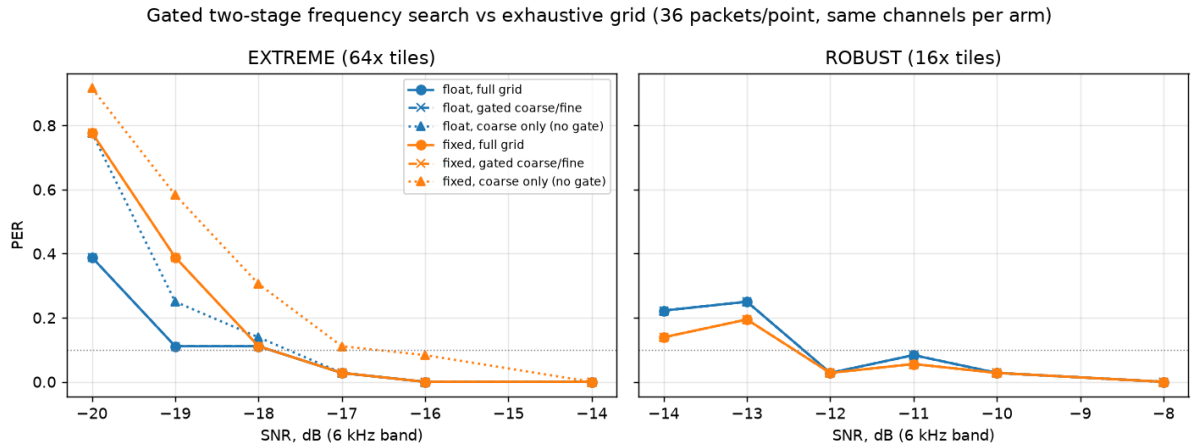


Figure 26: PER versus SNR for the exhaustive frequency grid (solid), the gated two-stage search (dashed), and the ungated coarse-only ablation (dotted); float and fixed-point receivers, same channels per arm. Gated overlays full exactly everywhere; the ablation's edge loss in the EXTREME panel is what the contrast gate prevents.

10.6 Integer SNR Estimator and Full-System Integration

Rate adaptation is driven by a per-frame SNR estimate the fixed receiver originally lacked. The added estimator is data-aided and divider-free: per-column first/second moments of the LLR stream against the known ± 1 bits (the re-encoded header always, plus the decoded data block for $\mu \leq 2$), an integer $\log_2$ via bit-length and a 16-entry mantissa ROM, minus the tiling gain, plus one measured global calibration constant ($-7.2\,\mathrm{dB}$, constant across all modes and modulations). Two design points were forced by measurement. First, grouping moments *per column* removes the multipath $|H|^2$ spread across carriers. Second, symbol rows must be *gain-weighted* by their mean |LLR|: a tiled EXTREME frame spans several QSB fade cycles, and both naive poolings failed — unweighted moments count the fading itself as noise (measured 5–15 dB pessimistic, which parked the rate ladder at rung 0), and equal-weight gain-normalized pooling yields a harmonic-mean-flavored estimate still dominated by fade valleys. Gain weighting reproduces the signal-energy-weighted average the float estimator reports — the statistic the ladder was tuned against — and reduces to the plain estimator when the gain is constant. Final accuracy: $\leq 1.5\,\mathrm{dB}$ error over $-17 \ldots +8\,\mathrm{dB}$, every mode and modulation (slightly pessimistic on the tiled modes, the safe direction).

With the estimator in place, a thin adapter (`FixedPHY`) gives the fixed TX/RX the float `Transceiver`'s link-facing interface — including blind mode auto-detection by trying the three per-mode receivers — and `LinkStation` accepts it via a `phy` parameter. The complete adaptive system then runs on the integer pipeline end to end: `demo_wav.py --phy fixed` records the two-station QSO of Section 11 with both stations *and* the blind monitor decoding on fixed point. At $-2/-1\,\mathrm{dB}$ the session bootstraps at EXTREME, netting and climbing exactly as the float system does, and the monitor blind-decodes 37 of 47 frames; at $+5\,\mathrm{dB}$ the ladder climbs into 16-QAM (rungs 9–10 on the air, requests up to rung 11) and the monitor decodes 26 of 40 (Appendix D).

10.7 Coarse-CFO Ambiguity and the Two-Lag Unwrap

The integer detector carried a defect that cost ≈8 % of *all* acquisitions and stayed hidden for a long time because acknowledged mode simply retransmits what it loses. Broadcast (Section 7.8) has no retransmission, so it surfaced there as whole groups vanishing, and the diagnosis is worth recording in full because the symptom pointed three layers away from the cause.

The observable was a timing outlier: the fixed receiver is sample-accurate on 22 of 24 noisy trials, and on the other two it locks +33 or +34 samples late. Thirty-three is one more than the 32-sample cyclic prefix, so those frames die of inter-symbol interference — and the rate was flat against SNR, which is the signature of a discrete decision going wrong rather than of noise.

The chain of causation runs backwards from there:

1. The coarse mask-shift search is a near-*tie* between adjacent frequency bins. Measured on a failing trial, the correct shift scored 36.7×10^6 and its neighbour 38.0×10^6 — 4 % apart, so noise picks the wrong bin often enough to matter. On its own this is harmless: the residual estimator exists to absorb exactly this.

2. It cannot. The residual is the lag-N phase of the tone field, which wraps beyond $\pm f_s/2N = \pm 46.9\,\text{Hz}$ — and that is precisely *one coarse bin*. With a one-bin coarse error the residual must supply +49.9 Hz, so it wraps to −43.8 Hz and the total lands 93.75 Hz ($= f_s/N$, one subcarrier) away.

3. A frequency error *cyclically shifts* a Zadoff–Chu correlation in time. The 93.75 Hz error moves the ZC peak by ≈34 samples, and the fine stage duly reports it: the ZC stage was faithfully answering for the frequency it had been handed.

So the defect is a **phase-unwrap failure**, not aliasing and not peak selection. The float chain never had it because it resolves the residual with a full-block FFT peak (unambiguous over ±1.5 bins) before refining with the lag-N phase; the integer model had only the lag-N step.

The fix restores the missing disambiguation at integer cost: take the phase at lag $N/2$ as well, which is unambiguous over $\pm f_s/N$ — wide enough to cover a one-bin coarse error — and use it to choose which lag-N cycle is the correct one,

$$\hat{\omega} = \omega_N + \text{round}\left(\frac{\omega_{N/2} - \omega_N}{f_s/N}\right) \cdot \frac{f_s}{N}.$$

One extra correlation, no FFT, no new tables. Applied identically to `ofdm_phy/fixed/rx.py` and `cport/src/rx_detect.c`.

Table 9: Effect of the two-lag unwrap on the integer detector.

	before	after
CFO on the two failing trials (truth +3.0 Hz)	−90.9, −87.0 Hz	+3.8, +5.7 Hz
timing outliers, 24 trials × 3 SNRs	6 at +33/ + 34	**none**
locks within ±16 samples	21–23 of 24	**24 of 24**

The golden vectors were regenerated against the corrected model and the C detector re-verified: that `test_rx` still passes is the meaningful check, since it asserts the two

implementations agree bit-exactly with the new unwrap rather than merely that both work.

End to end, the fix closes a gap that had looked like a sensitivity difference. Broadcast delivery (Section 7.8, 200-byte payload, 12 trials per point) ran 15–20 percentage points behind the float chain before it, because one lost acquisition costs a whole group of four frames; afterwards the integer chain matches float everywhere and is *ahead* at the two lowest points — the same compressive 6-bit quantization advantage reported above, no longer masked by lost acquisitions.

Two coincidences make this defect what it is, and both are visible in Figure 27. The first is numerical: the coarse bin spacing is $f_s/B = 46.875\,\text{Hz}$ and the lag-N wrap boundary is $f_s/2N = 46.875\,\text{Hz}$ — *the same number*, because $B = 2N$. A one-bin slip therefore demands a residual sitting exactly on the wrap, where the phase is at $\pm\pi$ and the decision is a coin toss. That is why the failures cluster at multiples of the bin spacing and vanish in between: 20–25 % of locks ruined at 0 and 46.9 Hz, none at all between 10 and 40 Hz.

The second coincidence is operational, and it is the one that matters. CFO ≈ 0 is not an arbitrary point on that axis — it is where the AFC netting loop of Section 8 drives a linked pair. The worst-affected operating point was the one the system actively steers toward, so a *well-netted* link paid the highest price. In acknowledged mode that price is invisible in the delivery statistics and shows up only as extra transmissions: at zero offset, 1.25 per delivered frame before the fix against 1.00 after, or 25 % of the link's transmissions spent re-sending frames the detector had mistimed.

The left panel measures the cause and the right the effect, and at zero offset they are exact complements — 20 % of locks off by more than half a cyclic prefix, 20 % of frames lost — so every mistimed lock cost precisely one frame. Note also what an A/B at a single frequency would have concluded: at 20 Hz the two curves are identical, and the first throughput measurement taken for this section, which happened to use that offset, reported no improvement whatsoever.

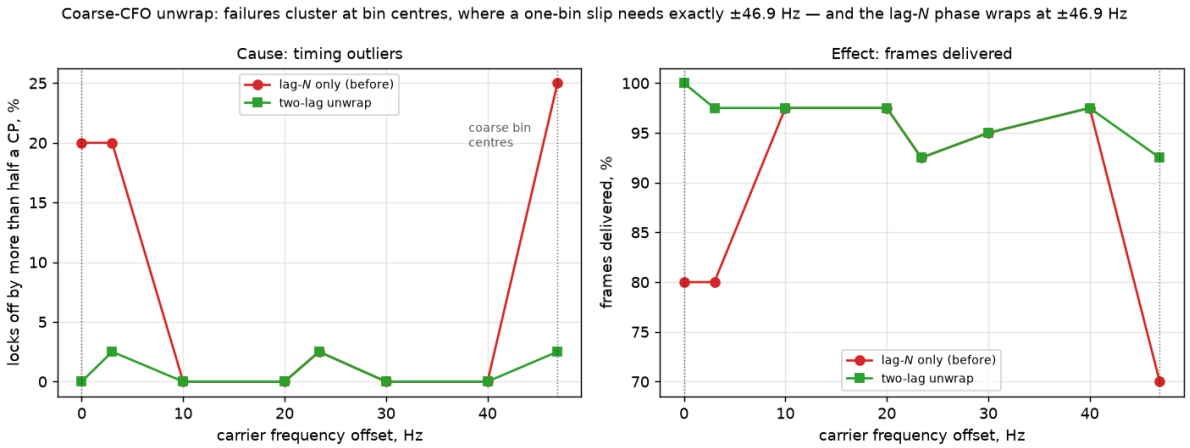


Figure 27: The coarse-CFO unwrap swept across one bin (`experiments/cfo_unwrap.py`, QPSK 1/2 at −3.5 dB, 40 frames per point). Dotted lines mark the coarse bin centres.

Table 10: Broadcast payload recovered, before and after the unwrap fix.

SNR	float	fixed (before)	fixed (after)
−4.5 dB	76.2 %	60.2 %	**87.1 %**
−3.0 dB	98.0 %	78.8 %	**100 %**
−1.5 dB	100 %	86.1 %	**100 %**
0.0 dB	100 %	90.1 %	**100 %**
+3.0 dB	100 %	100 %	100 %

10.8 Where the Fixed Model Differs from the Float Chain

The integer model is not a transliteration; several places diverge deliberately, and a caller that mixes the two must know which.

Table 11: Deliberate float/fixed divergences.

	float	fixed
Detection FFT (NORMAL)	128 bins	256 bins, so the coarse grid is fine enough for an unambiguous lag-N residual
Residual CFO	full-block FFT peak, refined by lag-N phase	lag-$N/2$ phase selects the lag-N cycle (Section 10.7)
CFO representation	Hz throughout	32-bit phase-increment word everywhere; Hz only at the boundaries
Analytic signal	`scipy.signal.hilbert`, zero delay	63-tap FIR with a **31-sample group delay** — detector positions are in analytic index space, not sample space
LLR quantization	float LLRs, ±20 clip	6-bit peak-normalized for $\mu \leq 2$; 8-bit for $\mu = 4$, whose wider dynamic range the 6-bit path compressed
LLR recalibration	measured monotone map	32-entry reliability ROM trained on the fixed chain itself
SNR estimate	decision-directed E_s/N_0	gain-weighted symbol rows, −7.2 dB measured constant; reads low outside the regime it was calibrated in
FFT scaling	unscaled	$1/N$ carried as a per-stage $\gg 1$, with BFP exponents tracked through every energy and LLR comparison

The group delay is the one that bites hardest in new code. Inside `FixedReceiver.receive()`

it cancels — detection and demodulation index the same analytic arrays — but any caller that detects in one space and then slices, bounds or advances in sample space accumulates 31 samples per step. It is the first thing to check when an integer-chain walk drifts.

10.9 Debug Lore

Three integer-domain bugs cost real time and are recorded so an RTL implementation does not repeat them: the ZC metric's Q-scaling must account for *both* 2^{-15} factors the Q15 kernel introduces (a perfect correlation scored 0.006 with one missing); scipy's `remez(..., type="hilbert")` returns taps producing $-\sin$ for a cosine input, so the taps must be negated for a positive-frequency analytic signal; and BFP energies compare only after shifting by $2\,\Delta\text{exp}$.

11 Validation and Results

11.1 Regression Suites

Every layer has a self-verifying suite (Table 12); all pass at the state documented here.

Table 12: Verification suite summary.

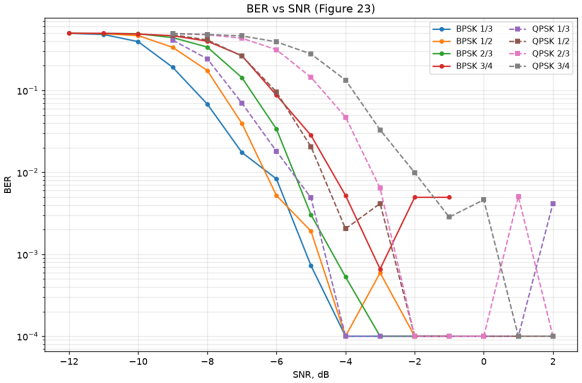
Suite	Checks	Scope
`verify_article.py`	44	bit-exact article worked examples: CRCs, Base38/QTH, conv-code outputs at all rates, scrambler/interleaver sequences, ZC ACF (Appendix A)
`smoke_e2e.py`	10	end-to-end TX→channel→RX incl. CFO, modes, STF, auto-mode detection
`fixed_point.py`	24	fixed-point primitives, TX/RX parity, LDPC and 16-QAM TX and RX, HARQ, calibration, SNR estimator (Appendix B)
`rf_channel.py`	6	SSB transparency, LO-derived CFO to 0.1 Hz, decode through the fading RF chain
`afc_netting.py`	—	netting convergence + trim-budget and anchor guard scenarios (PASS/FAIL)
`simplex_session.py`	—	whole-system two-station test, bit-exact delivery assert (PASS/FAIL)

11.2 Article Reproduction

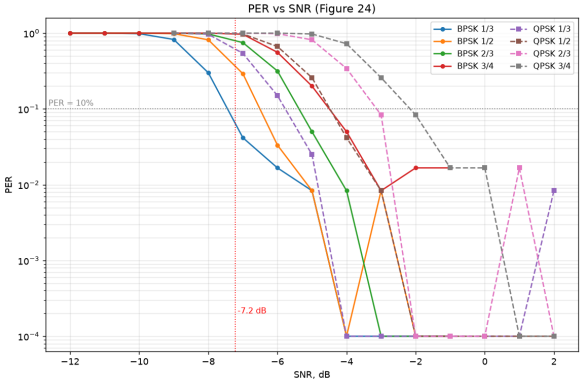
Table 13: Reproduction versus the article.

Metric	Article	This implementation
Worked examples	—	44/44 bit-exact
Sensitivity, BPSK 1/3	≈ -7.5 dB	-7.2 dB raw RX; -7.6 dB genie-synchronized
BER/PER curves (Figs. 23–24)	simulated	reproduced (Figure 28), both preambles

The remaining 0.3 dB between the raw and genie-synchronized receiver is residual synchronization loss at low SNR (occasional missed detections and one-symbol timing slips), not the decode chain — i.e. the reproduction gap is closed to within measurement noise of its diagnosed cause.

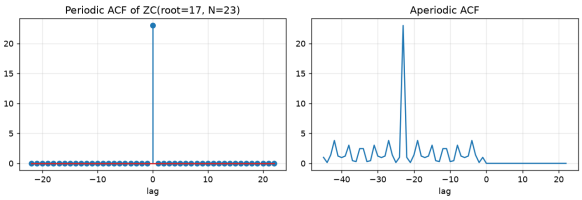


(a) BER versus SNR.

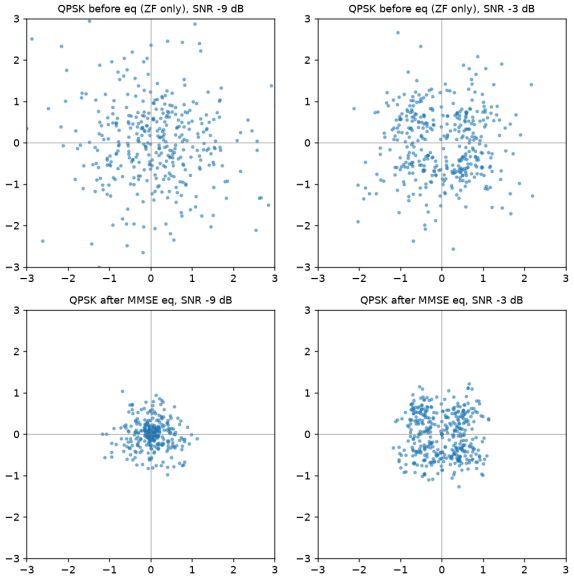


(b) PER versus SNR.

Figure 28: Reproduction of the article's Figures 23–24: eight modulation/rate configurations over the article channel model (multipath $[1, 0, 0.4, 0, 0, 0.2]$, BSC 10^{-3}, BEC $2{\cdot}10^{-2}$, random ± 100 Hz CFO with drift, random timing), 120 packets per point.



(a) Zadoff–Chu autocorrelation.



(b) QPSK constellation, before/after equalization.

Figure 29: Article-style diagnostic figures (`experiments/figures.py`).

11.3 Measured Feature Gains

Table 14 consolidates every A/B-measured improvement. All A/B comparisons use fixed seeds (same channel realizations per arm); sensitivity points use 40–120 trials.

Table 14: Feature gains, A/B measured.

Feature	**Measured effect**
LLR recalibration	+1.5–2 dB on BPSK/QPSK rungs (−9 dB: 6/40 → 29/40); regresses 16-QAM → gated to $\mu \leq 2$
HARQ chase combining	2nd attempt 15/20 vs. 10/20 at −8.5 dB (noise-limited regime)
LDPC (IRA 768/256) vs. conv	+0.7 dB AWGN at BLER 10 %; ≈ parity end-to-end (front-end LLR quality binds)
STF preamble variant	equal to Newman everywhere (±0.14 dB over 8 configs, 2σ agreement per point)
AFC netting	+284 Hz → < 12 Hz in 5 exchanges; steady ≤ 11 Hz under 0.09 Hz/s relative drift
Fixed-point RX vs. float	≤ 0.5 dB loss at −7 dB; *ahead* at −9/ − 10 dB (22 vs. 20, 11 vs. 5 of 30, against float+recal); 16-QAM within 0.3–0.8 dB after per-modulation LLR quantization (8-bit for $\mu = 4$)
Integer SNR estimator	≤1.5 dB error, −17 . . . + 8 dB, all modes; gain weighting essential (equal-weight pooling was 5–15 dB pessimistic on faded EXTREME frames)

11.4 The System Demonstration

The recorded demonstration (`results/system_demo.wav`, Figure 30, transcript in Appendix C) is the whole system at once: two stations over the fading SSB RF chain with realistic LO errors, EXTREME-mode negotiation, rate climb, AFC netting visible in the transcript's CFO column, HARQ, and bidirectional message transfer — blind-decoded from the monitor's audio by `demod_frame_auto` with no side information.

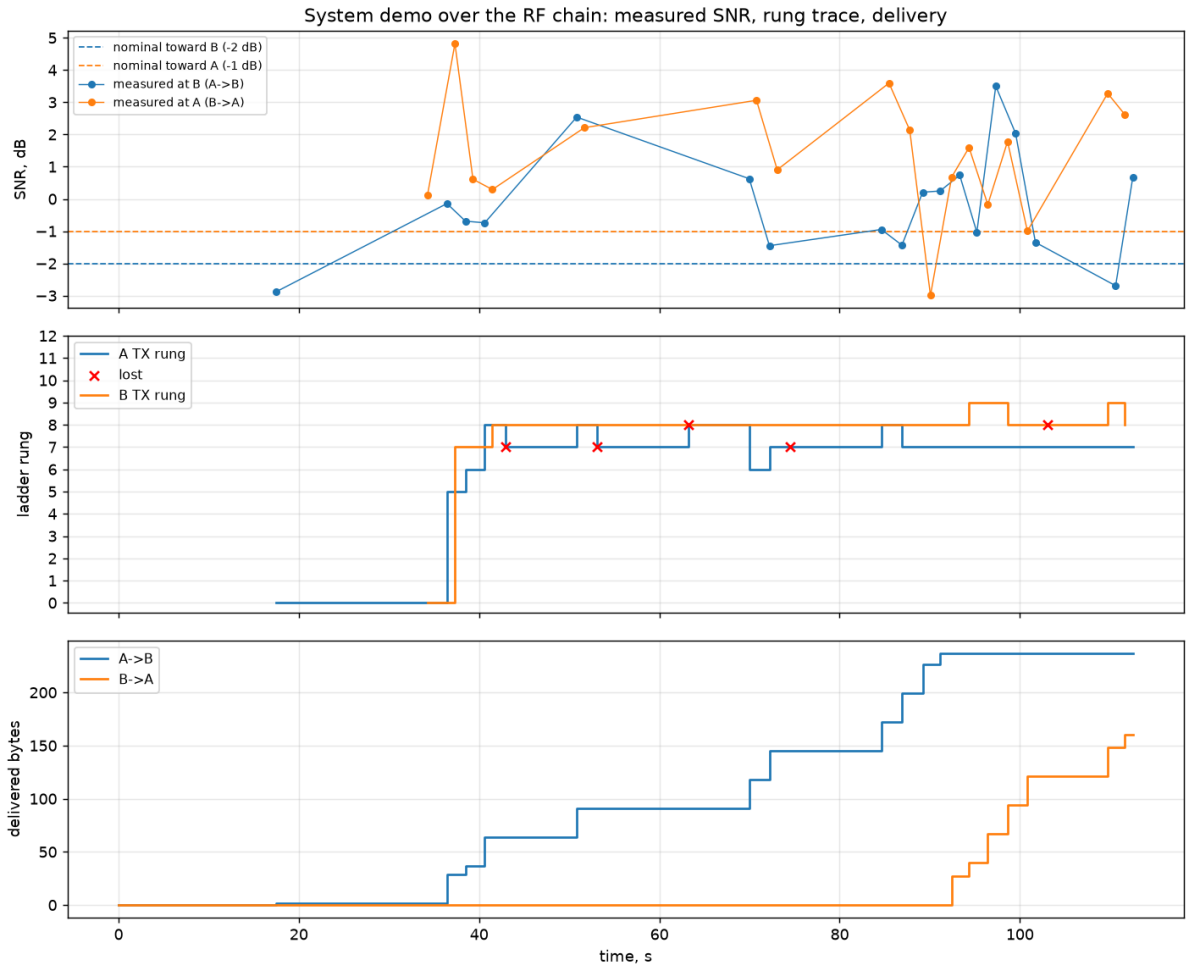


Figure 30: System-demo timeline: per-frame SNR, ladder rungs, and delivery over the session.

The same demonstration also exists on the *fixed-point* pipeline (`--phy fixed`, Section 10): at $-2/-1$ dB (`system_demo_fixed.wav`, 37/47 frames blind-decoded) and at $+5$ dB, where the ladder climbs into 16-QAM (`system_demo_fixed_qam16.wav`, 26/40; Figure 31 and Appendix D).

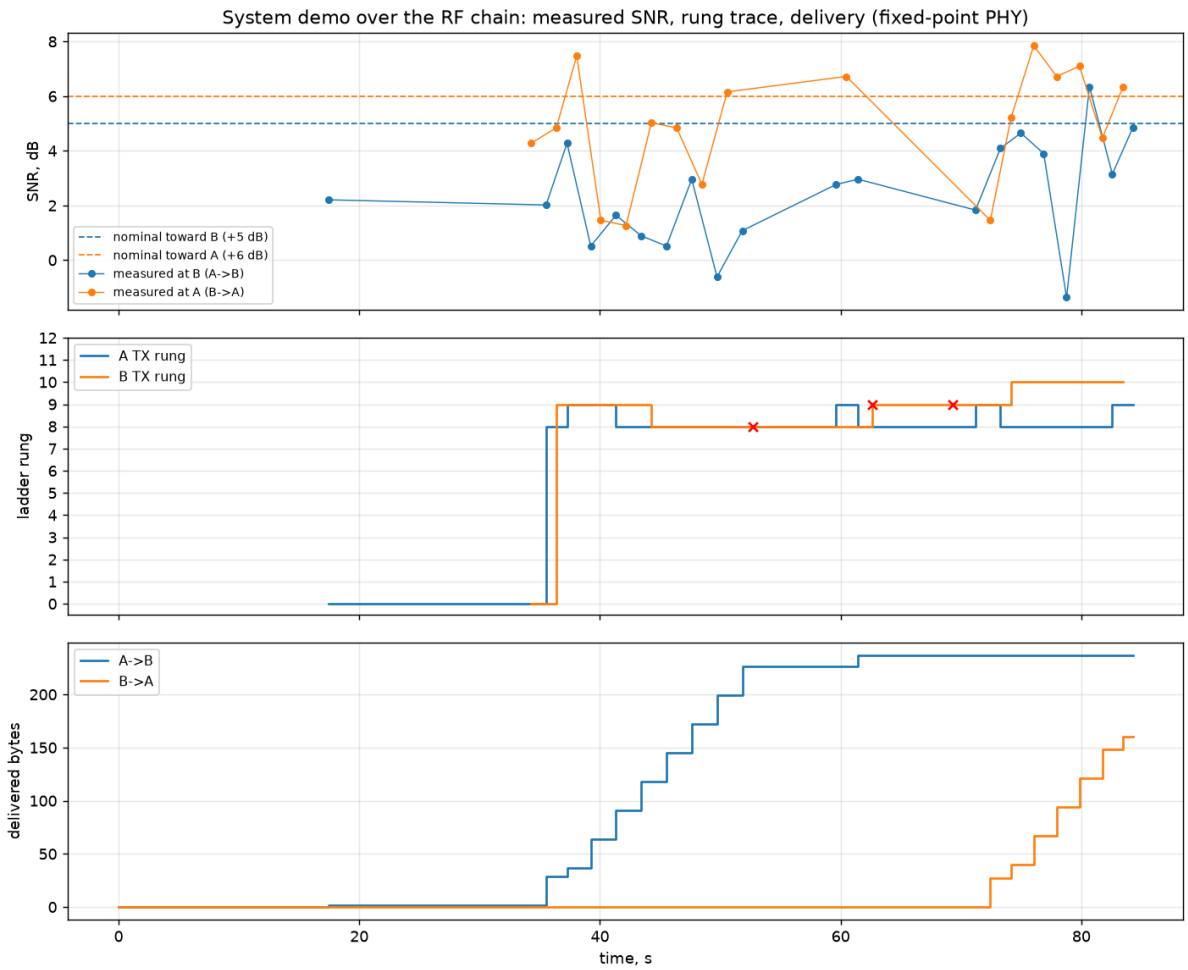


Figure 31: Fixed-point-pipeline demo at +5 dB: the integer SNR estimates drive the ladder into the 16-QAM rungs.

11.5 Link Budget

In radio terms, the EXTREME operating point (−17.5 dB planned, −17.9 dB measured, 6 kHz reference) needs $S/N_0 \approx +20.3$ dB-Hz, i.e. ≈ -13.7 dB in the ham-standard 2.5 kHz bandwidth — about 7 dB less sensitive than FT8 [6] and ∼20 dB better than SSB voice. Against ITU-R P.372 noise levels [9] with 0 dBi antennas: on 80 m at 10 W, 500–2000 km reliably on typical nights and 3000–5000 km from quiet rural sites; on 40 m at 10 W, 300–1500 km all day and worldwide multi-hop on quiet winter nights; 100 W adds roughly one ionospheric hop to everything. A planning caveat: EXTREME's 0.69 s coherent integration assumes $\lesssim$0.1 Hz Doppler spread — on disturbed or auroral paths ($\geq$1 Hz) the coherent gain collapses at any power and ROBUST is the better choice, one more argument for adaptive mode selection.

11.5.1 Measured Against a Reference Source

Everything above is derived from a *relative* sensitivity and an *assumed* noise figure. With the SDR receiver of Section 12.6 on the bench, the assumption can be replaced by a measurement: a tinySA's calibration output, a reference tone at a documented level, was fed into the radio through a 40 dB attenuator and the receive chain characterised end to

end (`demoapp/sdr_rx_calibrate.py`; `results/sdr_rx_calibration.csv`). At 10 MHz with −65 dBm at the antenna port:

Table 15: Receive chain measured against a reference source (HackRF One, LNA 40 dB).

Quantity	**Measured**
Frequency error	−33.1 Hz at 10 MHz = −3.31 ppm, repeatable to 0.0 Hz (−23 Hz at 7 MHz)
Linearity	1.002 dB/dB over 16 dB of gain, 0.03 dB residual
Receiver noise	−95.5 dBm in the 6 kHz reference band (NF ≈ 40 dB)
Absolute sensitivity = measured noise floor + the rung's required SNR:	
rung 0 EXTREME BPSK 1/3 7.8 bit/s	−113.4 dBm
rung 4 NORMAL BPSK 1/3 117.6 bit/s	−103.1 dBm
rung 12 NORMAL 16-QAM 3/4 1059 bit/s	−90.8 dBm

Two results matter for planning. First, the receiver's own noise is *not* what limits this link: against ITU-R P.372 atmospheric noise in the same 6 kHz band, external noise still dominates by +9 dB at a quiet rural site, +17 dB at a typical rural one and +24 dB in residential noise — so the externally-limited assumption behind the ranges above holds, and it holds with about 9 dB of margin at the quietest sites. That margin is precisely where a preselector and a low-noise amplifier begin to pay for themselves, and it is worth noting that the 40 dB noise figure measured here belongs to a wideband, 8-bit, unpreselected SDR — an ordinary HF receiver is far better, so this is close to a worst case.

Second, the frequency error is negligible for this waveform: 3.31 ppm against an acquisition range that tolerates roughly 50 ppm at 7 MHz. A free-running SDR needs no external reference to *acquire* a frame; a disciplined oscillator matters only for EXTREME's 0.69 s coherent integration, where drift *within* a 44 s frame is the constraint rather than absolute accuracy. The absolute levels in Table 15 inherit the calibration output's nominal −25 dBm; the relative results — linearity, frequency error, and the comparison against atmospheric noise — do not.

12 C Implementation and Infrastructure

The fixed-point reference model of Section 10 answers *whether* integer arithmetic preserves the link performance; the C port (`cport/`) answers whether the system *fits* a DSP or microcontroller. It is portable C99 with no `malloc` and no floating point on the signal path (doubles remain on the control plane — dB values and timers — as in the model), and covers the entire feature family: both FEC families, all three modulations and link modes, the gated two-stage frequency search, HARQ combining, calibrated LLRs, the integer SNR estimator, the full link layer, and a streaming receiver architecture for MCU deployment.

12.1 Bit-Exact Porting Methodology

Every module is validated against *golden vectors* generated from the Python fixed-point model (`gen_vectors.py`): inputs and expected outputs are dumped as C headers and

compared with `memcmp` — no tolerances anywhere. Whole transmit frames (up to 260 k samples for EXTREME) are compared through 64-bit FNV-1a hashes over every `int16` sample, with the first 64 samples embedded verbatim for debuggability. Two rules keep the comparison honest:

- **ROMs are dumped, never recomputed.** Twiddles, preamble blocks, pilot symbols, LDPC graph, ladder tables and detection kernels all come from the Python model; a C-side reimplementation of table generation would risk silently diverging in rounding.
- **Width findings are documented, not patched over.** Every intermediate that exceeds a naive width estimate (Viterbi path metrics, BFP energy sums, the ZC metric's Q-scaling) is recorded in `WIDTHS.md` with its measured range, so an RTL implementation inherits the analysis.

The suite currently runs **79 checks** across six binaries (primitives, bit pipeline, TX frames, RX demodulation, streaming receiver, link layer). RX genie vectors are self-hosting: clean frames are rebuilt by the bit-exact C transmitter inside the test, so only the Python detection genie (start sample, CFO word) and expected bits are dumped.

12.2 Streaming Receiver

The frame-at-once reference receiver is impractical on an MCU (it wants the whole capture in memory), so `rx_stream.c` implements a ring-buffer state machine (SEARCH → ZC → HEADER → DATA) fed in arbitrary chunks. The tone-detection stage is necessarily causal and diverges from the model in two documented ways: block exponents and the noise floor are windowed (a streaming receiver cannot know the whole capture's minimum exponent, and the model's whole-capture *median* floor is acausal), and detection commits on a local metric peak — when the above-threshold region ends or the argmax stays stable for a full window span — instead of a global argmax. On the golden corpus the streaming receiver lands on the identical tone anchor for clean frames, making everything downstream bit-identical to the frame-at-once path.

Memory is dominated by sample history, and two measurements shaped the final architecture:

- **The ring only needs to cover the Zadoff–Chu re-anchor lookback, not the preamble.** The tone stage is incremental (each 512-sample block folds into a ≈540-byte summary; raw tone samples are never re-read). A permanent high-water-mark diagnostic (`rxs_ring_hwm`) measured the deepest lookback over the corpus at 8 510 / 32 318 / 124 478 samples per mode.
- **One shared raw ring serves every instance.** All mode-instances listen to the same audio (the preamble root self-labels the mode), so concurrent instances write identical samples and a single `int16` ring of 147 456 samples (288 KB) replaces per-instance analytic rings. The analytic signal is reconstructed on extraction (Hilbert-on-read), bit-identical to the former write-time FIR — and measured *faster*, because the 63-tap filter now runs only on consumed regions rather than on every sample in every instance.

12.3 Protocol Extensions

Three extensions beyond the Python reference address the fixed per-frame overhead (preamble + header ≈0.64 s in NORMAL mode) that dominates air time at the top rungs:

- **Selective-repeat burst ARQ.** Bulk transfers send up to a window of frames back-to-back, answered by one bitmap acknowledgment (NACK-by-omission). The two flag combinations impossible in the legacy protocol serve as burst frame types, so the legacy path is untouched.
- **Extended frames.** The article's header `len` field counts packet *bits* (255 max → 27-byte payloads). An unused header type codepoint reinterprets `len` as payload *bytes*, allowing 255-byte payloads (2 076-bit packets) behind a single preamble. Burst fragment size scales with the engage-time rung (200 B at rung ≥10, 100 B at ≥7, else 25 B).
- **Streamed burst windows.** A whole selective-repeat window is transmitted behind one preamble and header (Sections 2.3 and 7.5) instead of one preamble per fragment. `tx_build_burst` waveforms are bit-exact against the fixed-point model, and both receivers can follow a burst: the frame-at-once path re-runs the recording through `rxd_receive_burst` and re-locks on each Zadoff–Chu block, while the streaming receiver steps to the next block at its deterministic offset via `rxs_continue_burst`.

A streamed burst carries only full-size fragments; the short tail of a transfer always travels as its own frame, because the receiver derives the message length from that final fragment's own length. The acknowledgment request rides on the burst's *first* block as well as its last, so a peer that cannot follow the stream still answers and the sender learns by bitmap rather than by waiting out a timeout.

That receive-side continuation is also where the sharpest bug of this work appeared. While the streaming receiver is stepping through blocks it is not running the preamble detector, so a continuation that does not know where to stop makes the receiver deaf for one block-time per phantom block. An early version re-armed its block counter on every successful block and therefore never learned that a burst had ended: it chased up to seven blocks that were never sent, ≈12 s of deafness landing squarely on the peer's next burst, which timed out and retried indefinitely. The transmitter marks the last block of a burst with the acknowledgment request, which is the stop signal — qualified as "set *and* not the first block of this stream", since the first carries it too.

Measured on the virtual channel: a 250-byte transfer completes in 3 frames instead of ≈20 for stop-and-wait, and a 5 000-byte file drops from 211 transmitted frames (legacy) to 45, holding the top QAM16 rung throughout. Streaming the window compounds this: a 250-byte transfer at 25-byte fragments takes 4 transmissions streamed against 13 after falling back to per-frame bursts, and a 14 kB file over a +20 dB channel takes 76 transmissions streamed against 187 per-frame.

A fourth extension sits above the protocol rather than inside it. Nothing in the chain compressed, while every comparable HF system does — Winlink's B2F, VARA, and PACTOR all compress before transmitting. Files are now DEFLATEd whole and the compressed stream is what gets split into parts; compressing each part separately would discard most of the ratio, since the dictionary never warms up. A distinct envelope

magic marks the compressed form, so a peer that predates it sees an unknown message rather than misparsing a structurally valid one, and the transmitter falls back to sending raw whenever compression fails to shrink the payload. Compression lives at the application layer precisely so the C port stays dependency-free: an embedded build substitutes miniz or heatshrink, with plain DEFLATE on the wire either way. Measured on a 14 kB configuration file: 14 162 bytes to 5 015 on air, 2.82×. The honest qualification is that transmissions fell only 1.4× and wall-clock delivery 1.6×, because acknowledgments and the rate-ladder bootstrap do not compress — the ratio applies to the part of an exchange that is payload, not to the exchange. Supporting endpoints added alongside: a diagnostic event stream (every TX/RX, timeout, rung change annotated with the controller inputs that caused it, burst state transitions) plus a controller-decision snapshot — which immediately located a real bug (burst acknowledgment windows systematically timing out and poisoning the rate controller toward its hard rung-0 fallback; fixed by forgiving a window's first miss and budgeting the reply air time for the actual bitmap at a conservative rung) — and an external oscillator fine-tune endpoint (VCTCXO DAC / CAT clarifier actuator driven by the AFC netting loop, with budget clamping, an anchor role, and a manual override).

12.4 Memory and Flash Engineering

Table 16 shows the measured flash footprint (`arm-none-eabi-gcc -mcpu=cortex-m7 -O2`, const tables count into `.text`). Three reductions brought the total from an initial 242 KB to ≈57 KB with zero runtime cost and bit-exactness enforced by the golden tests:

- **Preambles stored as their unique periodic blocks** (183 KB → 1.3 KB): each mode's preamble is exactly a 32-sample tone block tiled $2T\times4$ times, a 64-sample tone block tiled $T\times2$ times, and one 128-sample ZC tile (whose tail doubles as the cyclic prefix) repeated `sym_tile` times — 224 unique samples per mode, re-expanded by modulo indexing.
- **Single-table trigonometry** (18.2 KB → 8 KB): the NCO sine table and all six FFT twiddle arrays are exact index-arithmetic views of one 4096-entry cosine table ($\sin[i] = \cos[(i + 3N/4) \bmod N]$; twiddles at stride N/n); the generator asserts the identities at dump time.
- **Packed Viterbi traceback** (1 bit per state per step instead of a predecessor byte — the predecessor is $(s\gg1) + (\text{bit}\ll5)$), and `int32` depuncture buffers: extended-frame Viterbi state fell from 181 KB to 42 KB, bit-identically.

Table 16: Measured Cortex-M7 flash footprint (all three modes).

Object	Flash	Dominant tables
`fft.o`	9.4 KB	8 KB shared cosine ROM
`rx_detect.o`	10.5 KB	ZC kernels + detection masks
`rx_stream.o` + `rx_demod.o`	16.9 KB	frequency-search tables
`ldpc.o`	7.2 KB	code graph
`link.o` + `station.o`	7.2 KB	rate-ladder ROM
`tx.o`	3.5 KB	1.3 KB preamble blocks
`dsp.o` + `conv.o` + others	3.0 KB	—
Total	**≈57 KB**	

RAM, with the shared-ring architecture: 288 KB shared raw ring + 8/35/69 KB per-mode summaries and scratch + 53 KB decoder state ≈ **453 KB for the full three-mode station** (a single-mode NORMAL build, with the ring shrunk to its measured lookback, needs ≈40 KB). Against the chosen target — STM32H723 (Cortex-M7 at 550 MHz, 1 MB flash, ≈496 KB usable data RAM) — the full three-mode station fits with margin, the code fits in the 64 KB ITCM alone, and the CPU projections give ≤10 % worst-case load (EXTREME acquisition, amortized), with everything else below 3 %.

12.5 Demonstration Infrastructure

The C stack runs end-to-end in an interactive demonstration (`demoapp/`): a virtual channel daemon exposes two station devices and a configuration device over Unix sockets, moves 12 kHz `int16` audio in paced ticks, and applies propagation delay, AWGN sized against the delivered-signal RMS, one-pole Rayleigh fading, and half-duplex muting; an optional audio mode plays each station's receive signal on one stereo channel with the sound card as the pacing clock, making the protocol audible (the EXTREME bootstrap's 21-second warble adapting up to sub-second QAM16 chirps). Two console station applications (three streaming receivers each, one per mode, over the shared ring) provide messaging, multi-part file transfer over the burst protocol, channel statistics, the diagnostic event stream, and the oscillator fine-tune command. A scripted smoke test drives the whole stack — text plus a 5 000-byte file across the impaired channel, byte-compared on arrival — at 25× time scale; the protocol clocks itself from received samples, so behaviour is identical at any scale. The stop-and-wait inefficiency, the burst protocol's window bugs, the carrier-sense threshold and the rung-0 fallback were all found by this infrastructure rather than by inspection.

12.6 From Simulation to a Radio

The demonstration apps talk to a *driver* over a socket rather than to hardware, which turns out to be the right seam for real radio: an SDR driver (`demoapp/sdr_driver.py`) presenting the same devices puts the entire C stack on the air with nothing in `cport/` changed. It reproduces the SSB model of Section 9 sample by sample — audio → analytic signal (the receiver's own 63-tap Hilbert design) → interpolation to the SDR rate → I/Q, and back, where taking the real part of the decimated baseband *is* a product detector. Tuning the radio to the suppressed carrier makes the two stations' LO error appear as exactly the audio-band CFO the modem already tracks. The DSP costs 4.4 % of one

core to transmit and 3.1 % to receive per audio-second at 2.4 Msps, so a laptop runs it comfortably in real time.

Figure 32 shows the arrangement that needs no radio at all: the driver cross-connects two stations through the entire SSB chain — Hilbert transform, interpolation, int8 quantisation, decimation and product detection — so every line of the signal path is exercised, and the console apps above it cannot tell the difference. That is what the regression test uses.

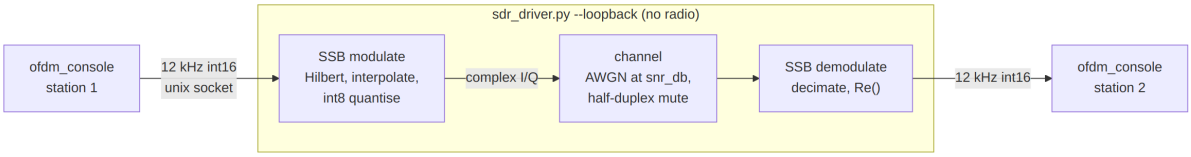


Figure 32: Hardware-free loopback: two stations cross-connected through the full SSB modulation and demodulation chain.

Bringing this up on a HackRF One produced four corrections that no amount of loopback testing would have found: the local oscillator must be *offset tuned* (with it on the carrier, the DC leakage lands inside the 3.5 kHz passband and the recovered audio was 98 % DC — the device has neither hardware DC correction nor AGC); the aggregate gain control is inert and the LNA/VGA/AMP stages must be driven individually (worth 32 dB of working level); `writeStream` only *queues* samples, so switching back to receive when it returns truncated the tail of every frame; and a killed process left the radio *keyed* until a receive stream was opened.

12.6.1 Instrumented Measurements

The two measurements that follow share the arrangement of Figure 33: a tinySA Ultra on USB, used in both directions. As an analyser it measures what the radio transmits; as a source — through its calibration output, which is a reference tone at a documented level — it measures what the receiver makes of a known input. Both are scripted, so the numbers below are reproducible rather than read off a screen.

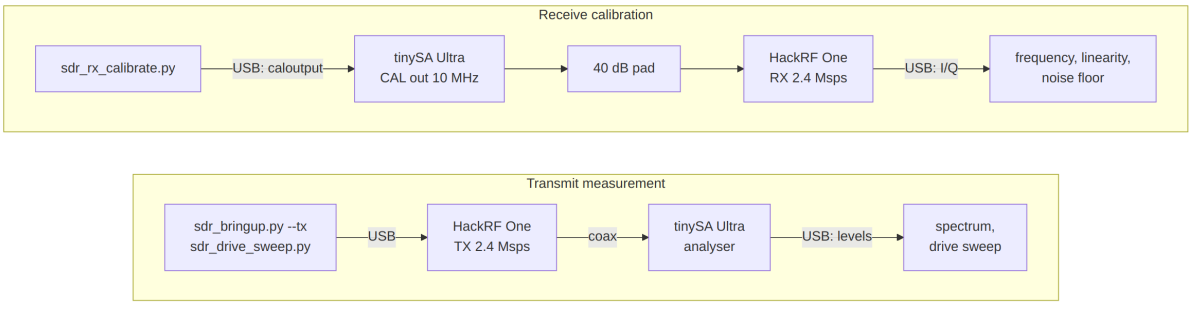


Figure 33: Instrumented bench, both directions: as an analyser the tinySA measures what the radio transmits; as a source its calibration output drives the receiver through a 40 dB pad.

12.6.2 The Transmitted Spectrum

Figure 34 is the waveform measured on a tinySA Ultra wired to the antenna port, captured by `demoapp/tinysa.py` (which sweeps with the transmitter off, keys it, and max-holds

several sweeps). The occupied band matches the design: a −3 dB width of 2.42 kHz against the 2.1 kHz the numerology predicts, once the 3 kHz resolution bandwidth is allowed for, sitting exactly where the carrier and 300–2400 Hz audio band put it. That single measurement validates the absolute tuning, the offset-tuning arithmetic, the drive scaling into the 8-bit DAC and the single-sideband modulation at once — an error in any of them shows up as splatter or a mirror image. Visible 50 kHz below the signal is the LO leakage at −35 dBc, the price of the offset tuning that keeps DC out of the *receive* passband.

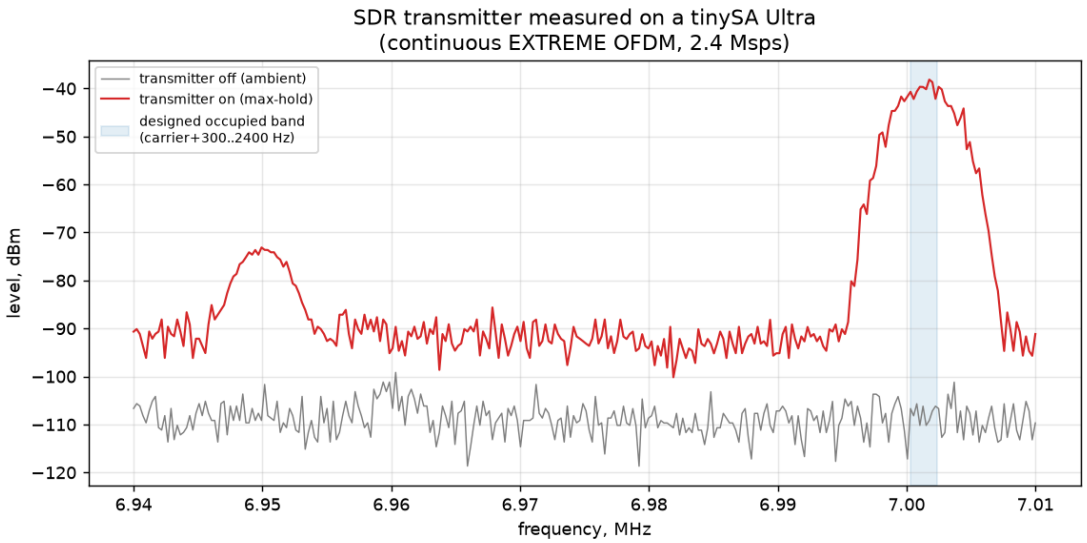


Figure 34: The transmitter measured on a tinySA Ultra: continuous EXTREME OFDM at 2.4 Msps, carrier 7.000 MHz, 3 kHz RBW. The shaded band is the designed occupancy (carrier + 300. . . 2400 Hz); the spur at 6.95 MHz is the offset-tuned local oscillator.

12.6.3 How Hard the Radio Can Be Driven

Figure 35 sweeps the transmit gain and measures, at each step, the fundamental, the shoulders 5–10 kHz outside the occupied band, and the harmonics. It should be read from the middle outwards: below VGA 16 the reported shoulders are the *analyser's* noise floor rather than the transmitter's, because the fundamental is only −58 dBm. Above VGA 32 the degradation is real compression, provoked by the waveform's ≈8 dB PAPR — at full drive the shoulders reach −22 dBc and the second harmonic −38 dBc, both worse than the −43 dBc usually demanded of spurious emissions.

The asymmetry between the two impairments is the practical result: a low-pass filter removes the harmonics but does nothing about the shoulders, which are spectral regrowth a few kilohertz from the carrier and inside any filter's passband. The usable window with this waveform is therefore about VGA 16–32, i.e. −42 to −24 dBm, and real power has to come from a *linear* external amplifier rather than from winding this one up — the same linearity requirement the OFDM waveform imposes everywhere else in the chain.

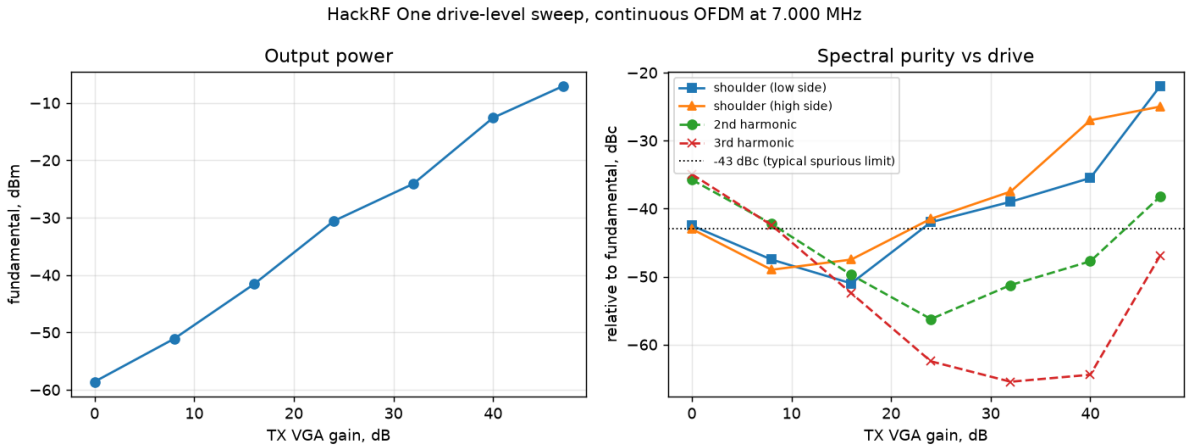


Figure 35: Drive-level sweep on the HackRF One (`demoapp/sdr_drive_sweep.py`): output power, and spectral purity relative to the fundamental, against transmit gain. Points below VGA 16 are limited by the analyser's noise floor, not by the transmitter.

12.6.4 Decoding Off the Air

A half-duplex radio cannot receive its own transmission, so the last question — does a receiver actually *decode* this over a real RF path? — needs a separate transmitter. Figure 36 shows the one used: a laptop's sound card plays a generated file into an AM modulator carried by an external generator at 7.000 MHz, and the result reaches the SDR through a 40 dB pad.

AM demands a different detector, for a reason worth stating because it is a property of the waveform rather than an inconvenience. The receiver is single-sideband: it tunes to a *suppressed* carrier and takes the real part of the baseband. Present it with a double-sideband signal and the lower sideband folds onto the upper one; worse, any frequency error between the generator and the SDR multiplies the audio by a slow cosine, splitting every subcarrier in two. The test therefore recovers the carrier that AM conveniently provides, derotates by it so the carrier sits at exactly 0 Hz and zero phase, and only then takes the real part. That is classical synchronous detection, and it is why the decoded CFO in Table 17 reads zero: the carrier lock has already absorbed the error the modem would otherwise estimate.

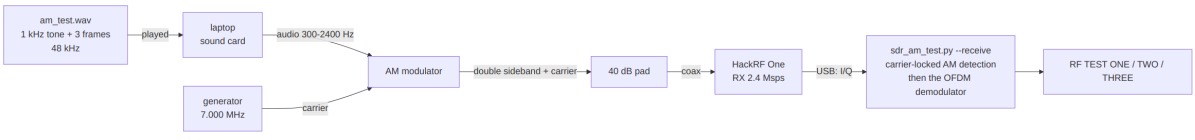


Figure 36: Off-air receive test: an external AM transmitter feeds the SDR through a 40 dB pad.

Table 17: All three transmitted frames recovered over a 7 MHz RF path (`results/am_rx_offair.wav`).

Frame	Payload	Length	SNR
NORMAL QPSK 1/2	`RF TEST ONE`	1.0 s	+13.6 dB
NORMAL BPSK 1/3	`RF TEST TWO`	1.8 s	+14.4 dB
ROBUST BPSK 1/3	`RF TEST THREE`	7.3 s	+8.5 dB

Figure 38 is what those frames look like as the demodulator received them. The analysis FFT is 128 bins at 12 kHz, which is exactly the modem's own subcarrier grid, so every row of the waterfall is one OFDM bin and the structure of Section 2 is directly legible: the Newman tone comb striping the preamble, then the header, then the data block filling 300–2400 Hz. The two NORMAL frames last about a second; the ROBUST frame occupies 7.35 s for the same 27 bytes, which is what 16× tiling costs and buys. The region boundaries are not fitted — they are computed from the decoded header's own modulation, code rate and length fields, and land on the measured burst to within 15 ms.

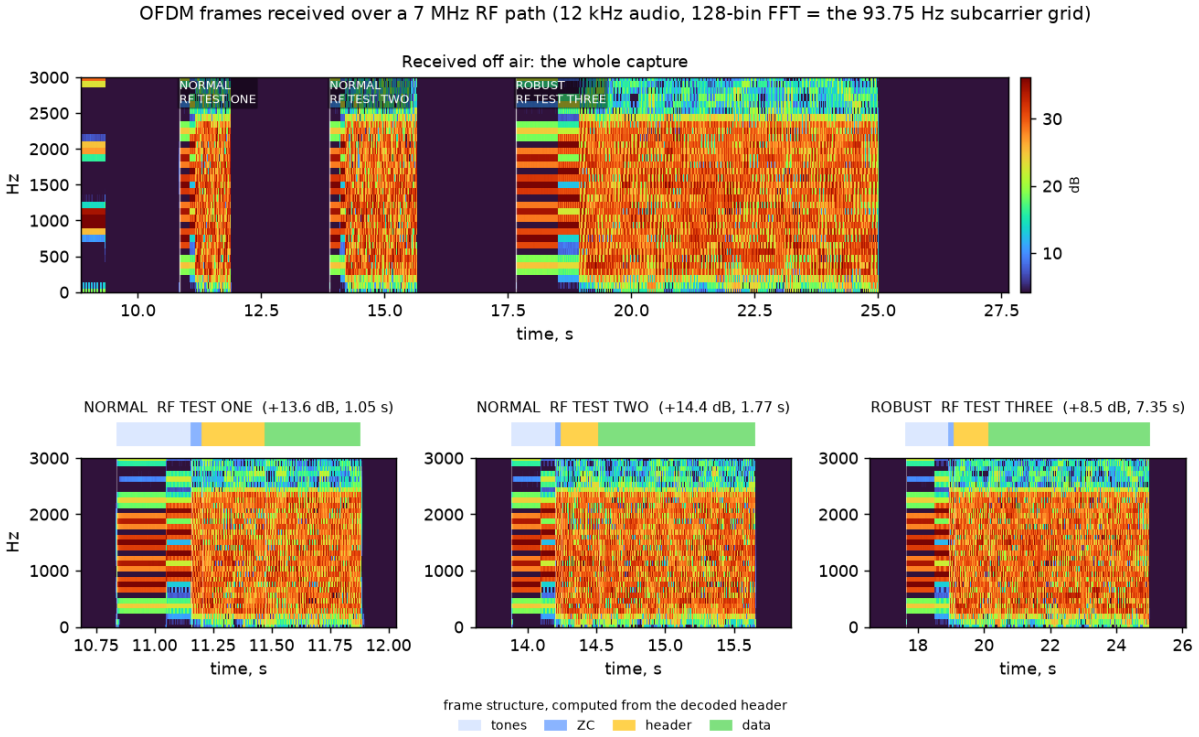


Figure 37: Frames received over the air (`demoapp/sdr_waterfall.py` on `results/am_rx_offair.wav`): the whole capture above, each frame close up below with its preamble, header and data spans marked.

Figure 38 shows those frames as the demodulator received them. The analysis FFT is 128 bins at 12 kHz, exactly the modem's own subcarrier grid, so each row is one OFDM bin and the frame structure of Section 2 is directly legible: the Newman tone comb lighting only a few bins, the Zadoff–Chu symbol closing the preamble, then the header and the data block filling 300–2400 Hz. The spans above each waterfall are computed rather than fitted — from the mode's tone-field geometry and the decoded header's own modulation, code rate and length — and they land on the measured burst to within 15 ms. The comb's

end is measurable independently: bin occupancy sits at 0.17 through the tone fields and jumps to 0.9 at 0.321 s, against a predicted 0.320 s. It is worth noting that the ZC symbol occupies all 23 carriers, so on a waterfall it resembles data even though it belongs to the preamble — which is why it is drawn as its own span.

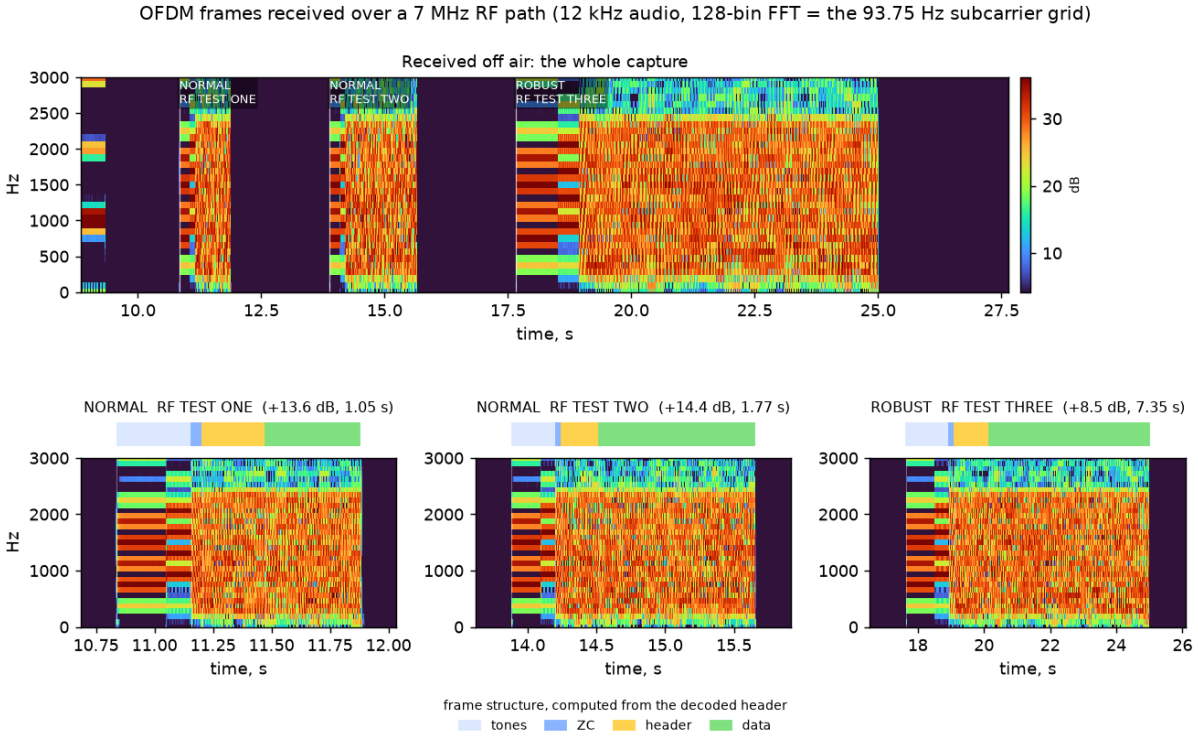


Figure 38: Frames received over the air (`demoapp/sdr_waterfall.py` on `results/am_rx_offair.wav`): the whole capture above, each frame below with its measured structure. The ROBUST frame carries the same 27 bytes as its NORMAL neighbours and occupies 7.35 s doing it — what 16× tiling costs, and buys.

Every frame transmitted was recovered, across two link modes and two modulations, with the mode identified from the preamble root alone — the mechanism the whole adaptive ladder depends on, here exercised over real RF rather than in simulation. ROBUST reports the lowest SNR by design: its 16× tiling spreads the same energy over a longer symbol, so the per-symbol estimate falls while the decoded margin grows.

Two honest qualifications. The transmitter is an AM modulator rather than the SDR of the preceding sections, so this validates the receive chain over RF and not a complete transmit-to-receive loop, which still wants a second radio. And a first, coarser scan of the same recording found only two of the three frames, because it advanced four seconds after each hit while the demodulator returns one frame per window — the miss was in the measurement harness, not the link, which is exactly the kind of result worth re-checking before blaming the hardware.

13 Discussion

13.1 What Binds Performance Now

The front end, not the code. The clearest lesson of the coding work is that FEC choice stopped mattering before code quality did: LDPC's 0.7 dB AWGN advantage compresses to parity end-to-end because the demodulator's LLR quality is the binding constraint. The monotone recalibration recovered 1.5–2 dB, but a standards-grade LDPC matrix (802.11n/NR-style [8]) *plus* fully calibrated LLRs would be needed to realise the remaining ∼2 dB. Any further sensitivity work should start at the LLR front end, not the decoder.

Synchronization is the sensitivity frontier. Both the original reproduction gap and the fade-loss analysis point the same way: at the edge, frames are lost to detection and timing, not to decoding. HARQ helps only in the noise-limited regime for exactly this reason — a sync-level loss leaves no LLRs to combine.

Physics caps EXTREME. The 64× mode's 0.69 s coherent integration assumes a channel coherent over that window ($\lesssim$0.1 Hz Doppler spread). This is a physical ceiling, not an implementation one, and it is why the mode ladder — rather than ever-longer integration — is the right architecture.

Feedback latency at the floor. Once a direction falls to EXTREME, the adaptation loop period is bounded by 25–45 s frame air times. The controller is designed around this (fade-aware filtering, loss fallback, silence decay), but no tuning can make a 7.8 bit/s control loop fast.

13.2 Limitations

The channel model, while faithful to the article (and extended with Rayleigh fading and a physical LO model), is still a simulation; the measured sensitivities should be planned around with ∼3 dB of fading margin (−14 . . . −15 dB for EXTREME). The 16-QAM rungs run without LLR recalibration (the map is BPSK-trained); a modulation-specific map is straightforward but unmeasured. The LDPC matrix is a from-scratch IRA construction, not a standards matrix. The project also has no over-the-air validation yet — the WAV recordings are transceiver-ready precisely to enable it.

13.3 Future Work

In rough order of expected value: over-the-air testing with real SSB rigs using the recorded WAVs; a standards-grade LDPC matrix plus QAM-specific LLR recalibration; porting the fixed receiver's slew-limited tracker back to the float model (it is measurably better); pilot reduction or 2-D channel estimation to reclaim carrier capacity; per-carrier bit-loading on the measured channel estimate; frame aggregation in the ARQ to amortise preamble overhead at high rungs; a 46.875 Hz-spacing variant for very stable channels; and shrinking EXTREME's frequency-search range after AFC netting converges (compute reduction on embedded targets).

14 Conclusion

The project set out to reproduce a published amateur-radio OFDM PHY layer and ended with a complete adaptive communication system. The reproduction itself is closed: 44/44 worked examples bit-exact, the BER/PER figures reproduced, and the sensitivity gap traced to three specific synchronization defects and eliminated to within 0.3 dB of a genie-synchronized bound. On that foundation, the waveform was extended an order of magnitude in each direction — down to −17.9 dB and 7.8 bit/s (78 % of Shannon capacity at −20 dB) and up to +4.7 dB and 1059 bit/s with 16-QAM — and wrapped in a link layer that measures everything it can observe and feeds all of it back: receiver-driven rate adaptation over a 13-rung measured ladder, CRC-gated HARQ from exported LLRs, learned per-rung sensitivity offsets, and closed-loop AFC netting of the stations' carrier frequencies.

Two results deserve emphasis beyond the feature list. First, the measured LLR recalibration — 1.5–2 dB from correcting the *shape* of the demodulator's confidence, at zero over-the-air cost — illustrates how much sensitivity can hide between a correct demodulator and a correct decoder. Second, the fixed-point receiver matching and then beating the floating-point chain at the sensitivity edge shows that an RTL-oriented implementation need not be a degradation; the discipline it forces (bounded scales, division-free estimation, a tracker instead of a polynomial fit) produced the better algorithm.

Everything claimed here is regenerable from the repository: the regression suites are self-verifying, every sensitivity figure has its sweep script and JSON dump, and the final demonstration is a WAV file any copy of the receiver decodes blind. The natural next step is the one simulation cannot take: the same frames over a real HF path.

References

[1] VARA HF — signal identification wiki. https://www.sigidwiki.com/wiki/VARA_HF. accessed 2026-08-17.

[2] bashkirtsevich. Development of a digital amateur-radio protocol based on OFDM: The PHY layer (in Russian). Habr, 2024. https://habr.com/ru/articles/1070804/.

[3] David Chase. Code combining — a maximum-likelihood decoding approach for combining an arbitrary number of noisy packets. *IEEE Transactions on Communications*, 33(5):385–393, 1985.

[4] Jinghu Chen, Ajay Dholakia, Evangelos Eleftheriou, Marc P. C. Fossorier, and Xiao-Yu Hu. Reduced-complexity decoding of LDPC codes. *IEEE Transactions on Communications*, 53(8):1288–1299, 2005.

[5] David C. Chu. Polyphase codes with good periodic correlation properties. *IEEE Transactions on Information Theory*, 18(4):531–532, 1972.

[6] Steven Franke, Bill Somerville, and Joe Taylor. The FT4 and FT8 communication protocols. *QEX*, pages 7–17, July/August 2020.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[8] IEEE. *IEEE Std 802.11-2020: Wireless LAN MAC and PHY Specifications*. IEEE, 2021.

[9] ITU-R. Recommendation P.372-16: Radio noise. Technical report, International Telecommunication Union, 2022.

[10] Hui Jin, Aamod Khandekar, and Robert McEliece. Irregular repeat–accumulate codes. In *Proceedings of the 2nd International Symposium on Turbo Codes and Related Topics*, pages 1–8, 2000.

[11] Donald J. Newman. An L^1 extremal problem for polynomials. *Proceedings of the American Mathematical Society*, 16(6):1287–1290, 1965.

[12] Timothy M. Schmidl and Donald C. Cox. Robust frequency and timing synchronization for OFDM. *IEEE Transactions on Communications*, 45(12):1613–1621, 1997.

[13] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.

[14] Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967.

Appendices

The four appendices below collect verbatim tool output that would interrupt the main narrative. Appendix A reproduces the output of the bit-exact article-reproduction suite (`verify_article.py`) referenced in Section 11; Appendix B reproduces the output of the fixed-point validation suite (`fixed_point.py`) referenced in Section 10; Appendix C reproduces the blind-decoded transcript of the recorded two-station demonstration session (`demo_wav.py`) referenced in Section 11; and Appendix D reproduces the transcript of the same demonstration running entirely on the fixed-point pipeline at +5 dB, where the ladder climbs into 16-QAM.

A Article-Reproduction Suite Output

Verbatim output of `./venv/bin/python experiments/verify_article.py`: the 44 bit-exact checks against the article's worked examples — numerology constants, Base38 callsign and QTH-locator encodings, CRC-8/CRC-16 values, convolutional-code outputs at all four rates, interleaver/scrambler sequences, and Zadoff–Chu sequence properties.

```
[PASS] subcarrier spacing = 93.75 Hz
[PASS] channel bins = 3..25 (23 carriers)
[PASS] center bin = 14 (1312.5 Hz)
[PASS] pilot bins = [3, 6, 10, 14, 17, 21, 25]
[PASS] data carriers = 16
[PASS] cyclic prefix = 32 samples (25%)
[PASS] tiled symbol = CP + 4*128 samples
[PASS] newman tone bins = [8, 12, 16, 20]
[PASS] newman shifted bins = [10, 14, 18, 22]
[PASS] callsign_to_int('R9FEU') = 4671407026402144
[PASS] int_to_callsign round-trip
[PASS] qth6_to_int('LO88CA') = 12261936
[PASS] int_to_qth6 round-trip
[PASS] crc8_lte example = 0b10101101
[PASS] crc16 example input is 236 bits
[PASS] crc16_ccitt example = 0b1001001100011111
[PASS] crc16 example payload decodes to article text
[PASS] crc16 example equals Data(reserved=12375, ...).encode() body
[PASS] Header(ver=1, BEACON, BPSK, R13, len=90) bits
[PASS] Header round-trip
[PASS] Beacon packet size = 90 bits
[PASS] Beacon callsign field bits
[PASS] Beacon QTH field bits
[PASS] Beacon round-trip
[PASS] Data packet size = 20+216+16 bits
[PASS] Data round-trip
[PASS] CC impulse response (rate 1/3)
[PASS] CC codec constants
[PASS] header CC rate 1/3 (93 bits)
[PASS] header CC rate 1/2 (62 bits)
[PASS] header CC rate 2/3 (47 bits)
[PASS] header CC rate 3/4 (42 bits)
[PASS] Viterbi round-trip CCLTEBPSK
[PASS] Viterbi round-trip CCLTEBPSK_12
[PASS] Viterbi round-trip CCLTEBPSK_23
[PASS] Viterbi round-trip CCLTEBPSK_34
[PASS] Viterbi corrects 8/93 hard errors (rate 1/3)
[PASS] interleave 32 bits over 16 carriers
[PASS] deinterleave inverts interleave
[PASS] scrambler example sequence
[PASS] scramble is an involution
[PASS] descramble (soft) recovers source
[PASS] ZC sequence constant amplitude
[PASS] ZC periodic ACF is a delta function

44 passed, 0 failed
```

B Fixed-Point Validation Suite Output

Verbatim output of `./venv/bin/python experiments/fixed_point.py`: primitive quality checks (FFT, Hilbert, NCO, CORDIC, integer Viterbi), the fixed/float TX×RX cross-matrix, the PER comparison table against the float receiver, and the LDPC / 16-QAM / calibration / HARQ / ROBUST / EXTREME decode checks referenced in Section 10.

```
[PASS] fixed FFT SQNR > 55 dB  (59.1 dB)
[PASS] Hilbert FIR SQNR > 55 dB  (60.7 dB)
[PASS] NCO derotation residual < -60 dBc  (-67.5 dBc)
[PASS] CORDIC atan2 error < 0.001 rad  (1.00001)
[PASS] CORDIC magnitude error < 0.1%  (19999)
[PASS] integer Viterbi corrects noisy 6-bit LLRs
[PASS] fixed TX waveform correlation > 0.999  (0.99998)
[PASS] clean fixed TX -> fixed RX
[PASS] clean float TX -> fixed RX
[PASS] clean fixed TX -> float RX

NORMAL PER, fixed RX vs float RX, 30 packets/point:
  -6 dB: fixed 26/30   float 30/30
  -7 dB: fixed 27/30   float 29/30
  -8 dB: fixed 26/30   float 23/30
[PASS] fixed RX within ~1 dB of float at -7 dB  (see table above)
[PASS] fixed RX decodes LDPC (ver=2) frames
[PASS] fixed RX decodes 16-QAM frames
[PASS] fixed TX LDPC (ver=2) -> fixed RX
[PASS] fixed TX LDPC -> float RX
[PASS] fixed TX 16-QAM -> fixed RX
[PASS] fixed TX 16-QAM -> float RX
[PASS] integer SNR estimate within 2 dB  (errs 0.6/0.8 dB at -7/0)
[PASS] calibrated mode (alpha fit + reliability ROM) decodes
[PASS] HARQ chase combining (complementary erasures)
[PASS] repacked LC word roundtrips fixed chain
[PASS] fixed TX emits ver=1 (conv) headers by default
[PASS] fixed ROBUST decode @ -11 dB  (cfo +29.4 Hz)
[PASS] fixed EXTREME decode @ -17 dB  (cfo +33.8 Hz)

24 passed, 0 failed
```

C System-Demonstration Transcript

Verbatim output of `./venv/bin/python experiments/demo_wav.py`: the two-station session over the fading SSB RF chain (Section 11). The first block is the simplex scheduler's air-time log as heard by the passive monitor (per-frame carrier snapshots include each station's LO drift and AFC trims); the second block is the *blind-decoded* transcript recovered from the monitor's WAV audio alone — mode, modulation and rate from the frame itself, measured CFO, and the unpacked link-control word (seq/ack/rung-request/SNR-report) for every decodable frame. A's and B's distinct carrier fingerprints (≈ -18 and ≈ -22 Hz at the monitor after netting) identify the transmitter of every frame. Encrypted-looking payloads are mid-message fragments of the bulk transfer; `b'\x00'` frames are ACK-only.

```
expected CFOs at monitor: A frames +28.4 Hz, B frames -71.0 Hz
expected station-to-station CFO A->B: +99.4 Hz (AFC netting active)
```

```
t=  91.1s  A->B phase complete, B starts its reply traffic
final trims: A -64.0 Hz, B +36.0 Hz; residual A->B offset +5.0 Hz
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_timeline.png
saved /home/ivan/projects/ofdm_transceiver_proto/results/system_demo.wav
     (113.6 s, 2663 KiB, 40 frames)
  t=   0.0s  A transmits  17.5 s (carrier  +42.6 Hz)
  t=  17.5s  B transmits  16.8 s (carrier  -57.1 Hz)
  t=  34.3s  A transmits   2.1 s (carrier   -4.4 Hz)
  t=  36.5s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  37.4s  A transmits   1.1 s (carrier  -32.3 Hz)
  t=  38.5s  B transmits   0.8 s (carrier   -5.6 Hz)
  t=  39.4s  A transmits   1.2 s (carrier  -20.2 Hz)
  t=  40.7s  B transmits   0.8 s (carrier  -21.6 Hz)
  t=  41.5s  A transmits   1.4 s (carrier  -12.2 Hz)
  t=  49.6s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  50.9s  B transmits   0.8 s (carrier  -21.8 Hz)
  t=  51.7s  A transmits   1.4 s (carrier  -19.8 Hz)
  t=  62.0s  A transmits   1.2 s (carrier  -19.5 Hz)
  t=  68.2s  A transmits   1.8 s (carrier  -19.4 Hz)
  t=  70.0s  B transmits   0.8 s (carrier  -22.2 Hz)
  t=  70.8s  A transmits   1.4 s (carrier  -19.3 Hz)
  t=  72.3s  B transmits   0.8 s (carrier  -22.2 Hz)
  t=  73.1s  A transmits   1.4 s (carrier  -19.2 Hz)
  t=  83.4s  A transmits   1.2 s (carrier  -18.9 Hz)
  t=  84.7s  B transmits   0.8 s (carrier  -22.5 Hz)
  t=  85.5s  A transmits   1.4 s (carrier  -18.8 Hz)
  t=  87.0s  B transmits   0.8 s (carrier  -22.5 Hz)
  t=  87.8s  A transmits   1.4 s (carrier  -18.8 Hz)
  t=  89.3s  B transmits   0.8 s (carrier  -22.6 Hz)
  t=  90.1s  A transmits   1.0 s (carrier  -18.7 Hz)
  t=  91.2s  B transmits   1.2 s (carrier  -22.6 Hz)
  t=  92.5s  A transmits   0.8 s (carrier  -18.6 Hz)
  t=  93.4s  B transmits   1.0 s (carrier  -22.7 Hz)
  t=  94.4s  A transmits   0.8 s (carrier  -18.6 Hz)
  t=  95.3s  B transmits   1.1 s (carrier  -22.7 Hz)
  t=  96.5s  A transmits   0.8 s (carrier  -18.5 Hz)
  t=  97.4s  B transmits   1.2 s (carrier  -22.7 Hz)
  t=  98.7s  A transmits   0.8 s (carrier  -18.4 Hz)
  t=  99.6s  B transmits   1.2 s (carrier  -22.8 Hz)
  t= 100.9s  A transmits   0.8 s (carrier  -18.4 Hz)
  t= 101.8s  B transmits   1.2 s (carrier  -22.8 Hz)
  t= 108.6s  B transmits   1.1 s (carrier  -23.0 Hz)
  t= 109.8s  A transmits   0.8 s (carrier  -18.1 Hz)
  t= 110.7s  B transmits   1.0 s (carrier  -23.0 Hz)
  t= 111.7s  A transmits   0.8 s (carrier  -18.0 Hz)

   t, s mode     rate           cfo seq ack req_rung -> speed selection
     snr rpt  payload
    5.8 EXTREME  BPSK 1/3    +28.4Hz   1   0 rung 0 (8 bit/s)
      -24.0dB  b'CQ'
   23.3 EXTREME  BPSK 1/3    -71.4Hz   0   1 rung 6 (235 bit/s)
       -2.0dB  b'\x00'
   34.7 NORMAL   BPSK 1/2    -18.6Hz   2   0 rung 8 (471 bit/s)
       +0.0dB  b'MSG DE R9FEU: QTH...'
   36.9 NORMAL   QPSK 1/2    -20.1Hz   0   2 rung 6 (235 bit/s)
       -2.0dB  b'\x00'
```

```
   37.8 NORMAL   QPSK 1/3    -46.3Hz   3   0 rung 8 (471 bit/s)
        +0.0dB  b' IVAN 73'
   38.9 NORMAL   QPSK 1/2    -19.8Hz   0   3 rung 8 (471 bit/s)
        +0.0dB  b'\x00'
   39.8 NORMAL   QPSK 2/3    -34.7Hz   0   0 rung 9 (529 bit/s)
        +0.0dB  b'!\x87\xeak\xde\xa6\xc8\xde \x04\x07\xf6D\xdalI\xbe...'
   50.0 NORMAL   QPSK 2/3    -26.4Hz   1   0 rung 9 (529 bit/s)
        +0.0dB  b'\xaaAy\x0c\xc7\x85*\x83\xa4\xbb\xaaP\xb7l\x1bO\xea...'
   51.3 NORMAL   QPSK 2/3    -35.6Hz   0   1 rung 8 (471 bit/s)
        +0.0dB  b'\x00'
   52.1 NORMAL   QPSK 1/2    -33.9Hz   2   0 rung 9 (529 bit/s)
        +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   62.4 NORMAL   QPSK 2/3    -34.1Hz   2   0 rung 9 (529 bit/s)
        +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   68.6 NORMAL   QPSK 1/3    -30.4Hz   2   0 rung 9 (529 bit/s)
        +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   70.4 NORMAL   QPSK 2/3    -33.9Hz   0   2 rung 8 (471 bit/s)
        +0.0dB  b'\x00'
   71.2 NORMAL   QPSK 1/2    -35.6Hz   3   0 rung 9 (529 bit/s)
        +0.0dB  b'\xef\x88\xbf\xb8O%|\xf0\x84\xd8\xe6\n\xf2\xad\x10;\x1d
   ...'
   72.7 NORMAL   QPSK 2/3    -36.5Hz   0   3 rung 8 (471 bit/s)
        +0.0dB  b'\x00'
   73.5 NORMAL   QPSK 1/2    -33.5Hz   0   0 rung 9 (529 bit/s)
        +0.0dB  b'C\x9b\xa3\xd4.\x08\xe3J%\xb2\xb9\xfb4\xba9Y\xf9...'
   83.8 NORMAL   QPSK 2/3    -32.7Hz   0   0 rung 9 (529 bit/s)
        +0.0dB  b'C\x9b\xa3\xd4.\x08\xe3J%\xb2\xb9\xfb4\xba9Y\xf9...'
   85.1 NORMAL   QPSK 2/3    -37.5Hz   0   0 rung 8 (471 bit/s)
        +0.0dB  b'\x00'
   88.2 NORMAL   QPSK 1/2    -31.7Hz   2   0 rung 9 (529 bit/s)
        +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
   ...'
   91.6 NORMAL   QPSK 2/3    -36.3Hz   1   3 rung 8 (471 bit/s)
        -2.0dB  b'R R TNX MSG DE UB...'
   92.9 NORMAL   QPSK 1/2    -31.2Hz   3   1 rung 9 (529 bit/s)
        +0.0dB  b'\x00'
   93.8 NORMAL   QPSK 3/4    -37.0Hz   2   3 rung 8 (471 bit/s)
        +0.0dB  b'9 QTH KP50 73'
   94.8 NORMAL   QPSK 1/2    -33.2Hz   3   2 rung 9 (529 bit/s)
        +0.0dB  b'\x00'
   95.7 NORMAL   QPSK 3/4    -35.6Hz   3   3 rung 8 (471 bit/s)
        -2.0dB  b'N\xcc@"\x10\xfa\xe9 gz\r\xcc\x8a\xac\xd0\x7f|...'
   97.8 NORMAL   QPSK 2/3    -36.5Hz   0   3 rung 8 (471 bit/s)
        +0.0dB  b'\xb6\xb0)X\x07\x8d\xf1I|\xdf{\xde%T7\xc8f...'
   99.1 NORMAL   QPSK 1/2    -32.5Hz   3   0 rung 9 (529 bit/s)
        +0.0dB  b'\x00'
  109.0 NORMAL   QPSK 3/4    -36.7Hz   2   3 rung 8 (471 bit/s)
        -2.0dB  b'$K\xdeVUv^;9O%\xdbt\xea\x82\xd5\x8a...'
  110.2 NORMAL   QPSK 1/2    -32.8Hz   3   2 rung 9 (529 bit/s)
        +0.0dB  b'\x00'
  111.1 NORMAL   QPSK 2/3    -36.9Hz   3   3 rung 8 (471 bit/s)
        -2.0dB  b'w\xa3zF\xffY\x10\xdb(\xe0Q#'
  112.1 NORMAL   QPSK 1/2    -30.7Hz   3   3 rung 9 (529 bit/s)
        +0.0dB  b'\x00'

decoded 30/40 frames from the noisy/fading WAV (fade dropouts are
   expected -- the stations' ARQ recovered them)
```

D Fixed-Point-Pipeline Demonstration Transcript

Verbatim output of `./venv/bin/python experiments/demo_wav.py --phy fixed --snr 5`: the +5 dB two-station session with both stations and the passive monitor running the *full fixed-point pipeline* (Section 10) — fixed transmitters, per-mode fixed receivers with blind mode auto-detection, the integer SNR estimator driving rate adaptation, and integer HARQ. The blind transcript shows the climb from the EXTREME bootstrap through the QPSK rungs into 16-QAM (rungs 9–10 on the air, requests up to rung 11), with the integer SNR reports in the link-control words and the AFC-netted carrier fingerprints in the CFO column.

```
expected CFOs at monitor: A frames +28.4 Hz, B frames -71.0 Hz
expected station-to-station CFO A->B: +99.4 Hz (AFC netting active)

t=  61.5s  A->B phase complete, B starts its reply traffic
final trims: A -56.0 Hz, B +40.0 Hz; residual A->B offset +7.6 Hz
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_fixed_qam16_timeline.png
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_fixed_qam16.wav  (85.3 s, 1999 KiB, 40 frames)
  t=   0.0s  A transmits  17.5 s (carrier  +42.6 Hz)
  t=  17.5s  B transmits  16.8 s (carrier  -57.1 Hz)
  t=  34.3s  A transmits   1.2 s (carrier   -4.4 Hz)
  t=  35.6s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  36.4s  A transmits   0.9 s (carrier  -32.3 Hz)
  t=  37.3s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  38.1s  A transmits   1.1 s (carrier  -20.3 Hz)
  t=  39.3s  B transmits   0.8 s (carrier  -17.6 Hz)
  t=  40.1s  A transmits   1.2 s (carrier  -12.2 Hz)
  t=  41.4s  B transmits   0.8 s (carrier  -17.6 Hz)
  t=  42.2s  A transmits   1.2 s (carrier  -12.1 Hz)
  t=  43.5s  B transmits   0.8 s (carrier  -17.7 Hz)
  t=  44.3s  A transmits   1.2 s (carrier  -12.1 Hz)
  t=  45.6s  B transmits   0.8 s (carrier  -17.7 Hz)
  t=  46.4s  A transmits   1.2 s (carrier  -12.0 Hz)
  t=  47.7s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  48.5s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  49.8s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  50.6s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  51.9s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  58.5s  A transmits   1.1 s (carrier  -11.6 Hz)
  t=  59.7s  B transmits   0.8 s (carrier  -18.0 Hz)
  t=  60.5s  A transmits   1.0 s (carrier  -11.6 Hz)
  t=  61.5s  B transmits   1.1 s (carrier  -18.0 Hz)
  t=  68.2s  B transmits   1.1 s (carrier  -18.2 Hz)
  t=  70.3s  A transmits   0.9 s (carrier  -11.3 Hz)
  t=  71.3s  B transmits   1.1 s (carrier  -18.2 Hz)
  t=  72.5s  A transmits   0.8 s (carrier  -11.2 Hz)
  t=  73.3s  B transmits   0.9 s (carrier  -18.3 Hz)
  t=  74.2s  A transmits   0.8 s (carrier  -11.2 Hz)
  t=  75.0s  B transmits   1.0 s (carrier  -18.3 Hz)
  t=  76.1s  A transmits   0.8 s (carrier  -11.1 Hz)
  t=  76.9s  B transmits   1.0 s (carrier  -18.3 Hz)
  t=  78.0s  A transmits   0.8 s (carrier  -11.1 Hz)
  t=  78.8s  B transmits   1.0 s (carrier  -18.4 Hz)
  t=  79.9s  A transmits   0.8 s (carrier  -11.0 Hz)
  t=  80.7s  B transmits   1.0 s (carrier  -18.4 Hz)
```

```
t=  81.8s  A transmits   0.8 s (carrier  -10.9 Hz)
t=  82.6s  B transmits   0.9 s (carrier  -18.5 Hz)
t=  83.5s  A transmits   0.8 s (carrier  -10.9 Hz)

 t, s mode     rate           cfo seq ack req_rung -> speed selection
   snr rpt  payload
 23.3 EXTREME  BPSK 1/3    -71.4Hz   0   1 rung 9 (529 bit/s)
     +2.0dB  b'\x00'
 34.7 NORMAL   QPSK 2/3    -18.6Hz   2   0 rung 10 (706 bit/s)
     +4.0dB  b'MSG DE R9FEU: QTH...'
 38.5 NORMAL   QPSK 3/4    -33.7Hz   0   0 rung 10 (706 bit/s)
     +4.0dB  b'!\x87\xeak\xde\xa6\xc8\xde \x04\x07\xf6D\xdalI\xbe...'
 39.7 NORMAL   QPSK 3/4    -32.7Hz   0   0 rung 9 (529 bit/s)
     +2.0dB  b'\x00'
 48.1 NORMAL   QPSK 2/3    -31.6Hz   0   0 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 48.9 NORMAL   QPSK 2/3    -26.7Hz   1   0 rung 9 (529 bit/s)
     +2.0dB  b'P\xce>m\xfa\xe9\xbf\x90\xb4[\x93\xf9\xf7|!\x97L...'
 50.2 NORMAL   QPSK 2/3    -31.6Hz   0   1 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 51.0 NORMAL   QPSK 2/3    -25.0Hz   2   0 rung 9 (529 bit/s)
     +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
 ...'
 52.3 NORMAL   QPSK 2/3    -32.0Hz   0   2 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 58.9 NORMAL   QPSK 3/4    -26.9Hz   2   0 rung 9 (529 bit/s)
     +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
 ...'
 60.1 NORMAL   QPSK 2/3    -33.5Hz   0   2 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 60.9 NORMAL   QPSK 2/3    -26.0Hz   3   0 rung 10 (706 bit/s)
     +4.0dB  b'!z\xfczI\x03\x1d\xfcG\x03\xe2'
 61.9 NORMAL   QPSK 3/4    -32.4Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 68.6 NORMAL   QPSK 3/4    -31.5Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 70.7 NORMAL   QPSK 3/4    -26.3Hz   3   0 rung 10 (706 bit/s)
     +4.0dB  b'!z\xfczI\x03\x1d\xfcG\x03\xe2'
 71.7 NORMAL   QPSK 3/4    -32.6Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 72.9 NORMAL   QPSK 2/3    -25.3Hz   3   1 rung 10 (706 bit/s)
     +2.0dB  b'\x00'
 73.7 NORMAL   QAM16 1/2   -32.7Hz   2   3 rung 9 (529 bit/s)
     +2.0dB  b'9 QTH KP50 73'
 74.6 NORMAL   QPSK 2/3    -24.0Hz   3   2 rung 10 (706 bit/s)
     +2.0dB  b'\x00'
 76.5 NORMAL   QPSK 2/3    -24.9Hz   3   3 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 77.3 NORMAL   QAM16 1/2   -31.8Hz   0   3 rung 9 (529 bit/s)
     +2.0dB  b'\xb6\xb0)X\x07\x8d\xf1I|\xdf{\xde%T7\xc8f...'
 80.3 NORMAL   QPSK 2/3    -23.6Hz   3   1 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 81.1 NORMAL   QAM16 1/2   -32.0Hz   2   3 rung 10 (706 bit/s)
     +4.0dB  b'$K\xdeVUv^;9O%\xdbt\xea\x82\xd5\x8a...'
 82.2 NORMAL   QPSK 3/4    -25.0Hz   3   2 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 83.0 NORMAL   QAM16 1/2   -31.6Hz   3   3 rung 10 (706 bit/s)
     +4.0dB  b'w\xa3zF\xffY\x10\xdb(\xe0Q#'
```

```
   83.9 NORMAL    QPSK 3/4     -26.6Hz    3    3 rung 11 (941 bit/s)
       +6.0dB  b'\x00'

decoded 26/40 frames from the noisy/fading WAV (fade dropouts are
   expected -- the stations' ARQ recovered them)
```